

Annex XVI - WR Code® - Transport-Agnostic Reference and Resolution Architecture

Version 1.95 - Consolidated Technical Disclosure - 30 August 2026

Part of the Optirando™ Canon. Consolidated technical architecture for U.S. patent-specification preparation. All figures are to be newly produced; the drawing plan is in §XVI.19.

Reference profile summary.

Reference forms. Every human-enterable textual reference has three blocks plus a check: a class prefix, two address blocks, and a terminal error-detection character. Parent-entry class P (Publisher) with an attached check; internal class I and cross-organizational class C, whose third block is addressed (an internal combination code or the responder identifier) with a separate check block; addressable sub-handshake classes SP/SI/SC and the ephemeral, session-scoped sub-handshake class SE, whose third block is a six-symbol combination code of entry and receiving-party identifier with the check rendered as a separate block; class C names the ordered pair of initiator and responder Publisher Identifiers with a separately rendered check block and carries a PENDING lifecycle state while it is an invitation. Prefix parsing is case-insensitive with an extensible terminal/non-terminal grammar; the check is class-aware and validated locally before any lookup.

Carriers. Any WR Code® reference of any class may be offered on any admitted carrier — printed or displayed text, QR/Data Matrix, NFC, BLE pointer, website discovery, email, messaging, documents, spoken audio (live speech, phone/conference audio, the audio track of a video or broadcast), on-screen text in video, or any carrier admitted later (§XVI.9.6). A spoken or visual wake cue may precede the reference on any carrier. Carriers differ in payload capacity, discovery role, and creditable channel provenance; they do not differ in what they carry, and none of them is a trust source.

Resolution and state. Namespace resolution through a signed Namespace Directory; recipient-bound release of pre-consent material; independent handshake and session lifecycles; a privacy-preserving, optionally anchored Establishment Commitment with end-to-end-confidential party nonces and principal-key delegation; and simultaneous UI indication of the active umbrella/parent handshake and active sub-handshake.

License Notice. This Annex forms an integral part of the Optirando™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™.

Trademark Notice. WR Code® is a trademark registered in the European Union (EUIPO). Optirando™, BEAP™, pBEAP™, qBEAP™, PoAE™, PoAC™, WR Pointer™, WR Connect™, WR Store™, WR Guard™, and WR Expert™ are project designations used for technical terminology.

Third-Party Marks. Bluetooth® is a registered trademark of Bluetooth SIG, Inc.; NFC-related marks are trademarks of NFC Forum, Inc.; QR Code® is a registered trademark of DENSO WAVE Incorporated. Data Matrix, AprilTag, ArUco, GS1 Digital Link, and other third-party names are referenced descriptively and without implication of affiliation or endorsement.

Drafting convention. Figures and example identifiers are conceptual and non-normative unless expressly stated otherwise. The term "reference profile" denotes one enabled embodiment of the disclosed architecture; features described for the reference profile are embodiments, not architectural limitations, unless the text identifies a functional dependency. MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY define the reference profile where used normatively. Concrete glyphs, field lengths, delimiters, block counts, block order, check placement, colors, carrier technologies, UI labels, hash functions, and example data structures are embodiment or profile choices unless the text identifies a functional dependency. Wherever this annex states a reference-profile rule in normative language, §XVI.5.11 and §XVI.17.1 state the generalized principle the rule instantiates; the generalized principle is the disclosure, and the rule is one enabled example of it.

Self-containedness note. Companion annexes of the Optirando™ Canon describe handshake establishment (Annex VII), consent and execution/finalization (Annex IX), and designation/spatial selection (Annexes XI and XV) in greater depth. This annex includes, in §XVI.2 and §XVI.13, a summary of every element of those mechanisms on which the WR Code® reference layer functionally depends, so that the disclosed reference-layer methods are understandable and practicable from this document alone.

§XVI.1 Technical Definition, Role, and Scope

§XVI.1.1 Technical definition of WR Code®

WR Code® provides a typed, transport-agnostic entry and reference layer for a governed handshake and resolution architecture. A WR Code® reference gives an external system, user, scanner, browser integration, mail integration, or other conformant client a compact address from which the responsible namespace, referenced entry, applicable handshake class, object or relationship binding, and current authoritative state can be determined through resolution.

In the human-enterable reference profile, every WR Code® reference, of every class, consists of four parts: (i) a class prefix; (ii) a publisher or namespace identifier; (iii) a second body block; and (iv) a terminal error-detection character (check character). The second body block (the third block of the code) is: for P a publisher-local entry reference; for C the counterparty's Publisher Identifier; and for I and for every sub-handshake class (SP, SI, SC, SE) a six-symbol combination code that binds the entry or context and the receiving party's identifier into one block. The check character is present in all classes without exception; it is computed over the prefix and both body blocks and validated locally before any lookup. It is attached to the local block for P — the one class whose third block carries no counterparty — and rendered as a separate fourth block for I, C, and all sub-handshake classes, whose third block is addressed. The first symbol of every human-enterable textual reference in the reference profile is a class-prefix symbol; the parser therefore always knows that classification precedes the address body. The visible reference is intentionally smaller than the complete handshake state. It need not encode participant policy, capabilities, assurance level, object semantics, relationship state, protected content, or execution authority; those properties remain authoritative resolver and handshake data.

A WR Code® reference may be transported directly where a carrier can reliably carry the complete normalized identifier. Where the carrier has limited payload or is intentionally used only for discovery, it may instead convey a registered token, tag identifier, index, or other pointer that resolves to the same WR Entry. The logical reference and the transport mechanism are therefore separate layers.

After capture, the reference is normalized, classified, and checked locally. Only a syntactically admissible reference is submitted to the applicable resolver or request-state service. Resolution and validation then determine the verified entry or request state and whether an applicable handshake already exists, must be established, requires re-authentication, or is unavailable. Consent, acceptance, entitlement, session creation, protected-data release, and execution remain later state transitions.

Accordingly, WR Code® is not defined by a barcode symbology, radio format, fiducial, or visual style. A standardized visual WR Code® rendering is one presentation of the reference layer; QR, Data Matrix, NFC, BLE/radio, email, printed text, speech transcription, fiducials, and other admitted mechanisms are capture or transport paths with different operational capabilities.

§XVI.1.2 One reference form for every carrier; identifiers that are never carried

Every carrier — printed text, QR/Data Matrix, NFC, BLE pointer, website discovery, and email — conveys the same kind of reference: a WR Code® Entry reference (or a pointer to one). An email offering a cross-organizational handshake carries a C-class entry reference exactly as a printed sign carries a P-class Publisher reference; the email differs only in the channel provenance that may be credited to it (§XVI.7.4). A handshake request is therefore itself human-enterable: the issuer emits the C-class WR Code®, and the receiver can read it, type or dictate it into the runtime, verify it, and consent (§XVI.7.5). What does not exist is a second, different code format for requests — the request *is* the WR Code® of the entry, in the same grammar and with the same check as every other class.

Identifiers that serve idempotency or evidence — request_instance_id in a request capsule, and the Establishment Commitment of an established handshake (§XVI.13.6) — are internal protocol values. They are never rendered on a carrier, never typed, dictated, or scanned, and therefore carry no check character. Throughout this annex, "WR Code® reference" or "textual reference" denotes an entry-addressing reference of class P, I, C, SP, SI, or SC (or a registered extension class).

§XVI.1.3 End-to-end technical sequence

capture / discovery → candidate normalization → class-prefix parsing (terminal / non-terminal / invalid) → local error-detection validation → resolver branch selection by class → namespace directory lookup: the Publisher Identifier is resolved to a signed directory record → publisher verification: the record identifies a verified account holder AND a valid DNS entry under the publisher's domain proves control of that domain AND the SSO-verified email address of the acting publisher principal lies in that same DNS-verified domain (or a domain the directory record registers for the publisher); only if all three hold is the namespace treated as verified. An account whose verified email is outside the publisher's DNS-verified domain cannot act for that publisher. For class C this step is

performed for BOTH Publisher Identifiers — the initiator's (first block) and the responder's (second block) — and each side additionally verifies the other's account-holder and DNS status before acting; a C reference is verified only when both namespaces pass → entry verification at the verified publisher's resolver: the entry context must exist in the repository and match exactly the registered assignment (binding) and version referenced; for sub-handshake classes the combination code is expanded and both its entry constituent and its receiving-party constituent are verified against the repository → counterparty verification for the non-publisher side: an individual (B2C) party has no DNS; it is verified as an authenticated account holder with a verified email address, and both parties' verified email addresses are bound into the handshake → recipient-bound retrieval through the Relay: the signed request capsule for the entry is obtained from the Relay named in the publisher's directory record, released only to a principal that authenticates as its intended recipient; capsule signature, delegation chain, Party Bindings, freshness, and request instance are verified locally → existing-state determination: an ESTABLISHED handshake within its grant is reused; otherwise the result is a verified invitation → explicit consent / acceptance by the user for every new umbrella/parent handshake (never implicit), producing a signed acceptance record that is returned to the initiator through the Relay; a sub-handshake may be entered without a new consent step only if its automation, data categories, and context were already consented to in the governing umbrella handshake's grant and the applicable policy permits inherited entry → optional session binding → authorized data use or execution

No umbrella/parent handshake is ever formed without the user's explicit consent. A consent-free path exists in exactly two cases, and both rest on consent already given: (a) reuse of an existing handshake within its grant, and (b) entry into a sub-handshake whose automation, context, and data use are fully covered by the umbrella handshake's consented grant, where the applicable policy permits inherited entry (§XVI.13.2). A sub-handshake that would require any automation, context, or data category not already consented to in the umbrella requires its own explicit consent. No resolution result is presented as verified unless the publisher has been identified as the account holder of the namespace, its DNS-based domain control has been confirmed, and the acting principal's SSO-verified email belongs to that DNS-verified domain. Email, domain, and namespace must therefore agree: the verified email proves the person, DNS proves the domain, the directory proves the namespace, and a mismatch between any two is a verification failure. For a cross-organizational (C) reference this applies to both organizations: the responder verifies the initiator's namespace before considering the invitation, and the initiator has verified the responder's namespace before issuing it; neither side relies on the other's verification.

The sequence separates recognition from authority. A code, carrier, transcript, page marker, email field, check result, or existing relationship record may identify what state should be examined, but none of those artifacts alone grants a new permission or effect.

§XVI.1.4 Scope

This annex defines the WR Code® entry/reference layer, textual grammar and error detection, namespace resolution, carrier mapping, resolver interface boundaries, website and email capture flows, cross-organizational invitation entries, existing-handshake recognition, visual OCR assistance, lifecycle rules, the interaction between parent entries, addressable sub-handshakes, and sessions, and the privacy-preserving Establishment Commitment used to evidence an established handshake. Handshake establishment detail, grants, signed request capsules, acceptance/countersignature, relationship state, revocation, consent, and execution/finalization are summarized in §XVI.2 and §XVI.13 to the extent this annex depends on them and are described in greater depth in companion annexes. This annex describes how a captured reference enters those mechanisms without making the carrier itself authoritative.

§XVI.2 Core Data Objects and Definitions

Term	Technical meaning
WR Entry	A resolver-registered referent addressed by a canonical WR Code® reference or by a registered carrier token. The entry may represent a publisher-level offering, internal entry (including a Pairing Slot), established cross-organizational relationship, object/binding context, or addressable sub-handshake.
Party Identifier	The identifier of one party to a handshake: a Publisher Identifier for a publisher/organization; a Principal Identifier for an individual user at principal level (any of the user's paired devices); or a Device-Scoped Principal Identifier for an individual user on one specific device. Every party additionally has a verified email address that is bound into the handshake (see Party Binding).
Party Binding	The set of verified identity facts for one party recorded in the capsule, acceptance record, and Handshake State Record: Party Identifier, the party's SSO Identity (its one verified email address — for a publisher, the verified email of the acting principal under the

Term	Technical meaning
	publisher's delegation), key generation, and, for a publisher, its account-holder and DNS domain-control verification result at binding time. Both parties' bindings are covered by both signatures and, through the capsule and acceptance hashes, by the Establishment Commitment.
SSO Identity	The single authenticated identity of a principal in the system. It is the principal's verified email address, established once by email-ownership verification and authenticated at the required assurance level on every login. There is exactly one SSO Identity per principal; every Party Binding, every handshake, every device pairing, and every Principal Key of that principal is bound to this same verified email address. No handshake, binding, or pairing may be formed under a different or additional email identity.
Principal Identifier	The fingerprint (hash) of a user principal's long-term public key, held in the user's Orchestrator. The key, and therefore the identifier, is issued under and bound to the principal's SSO Identity (verified email); a principal cannot hold Principal Keys under two different email identities. It is never broadcast for discovery (P10) and is disclosed only inside a signed capsule or acceptance record to the counterparty.
Device-Scoped Principal Identifier	The fingerprint of the Principal Key held on one specific paired device of a principal. Each device paired by an I pairing handshake generates its own Principal Key under the principal's SSO Identity during the pairing and presents it to the issuer, which registers it; the fingerprint of that key identifies "this principal on this device" and exists only from the pairing onward. It is the receiving-party identifier used when a sub-handshake, message clone, or resource must resolve on exactly one device of a principal. Revoking a device's key (for example, a lost device) invalidates every reference bound to that Device-Scoped Principal Identifier without affecting the principal-level identifier or the other devices.
Device Name	A human-readable name assigned to a device at pairing, chosen by the user or defaulted from Device Class and counter ("laptop-2"), and unique within the principal. The full addressing path is Publisher Identifier → Principal (SSO Identity) → Device Name. The name is the handle by which users select a device in lists; it never appears in a reference and never on a carrier. Renaming does not affect issued references (they bind the Device Record, not the name); after revocation the name may be reused for a new device while old references still point at the revoked record.
Device Record	The signed record a publisher or tenant holds for each paired device: Device Name, Device Class, Device-Scoped Principal Identifier (key fingerprint), owning principal, status, generation. It is created by the pairing handshake and signed by the tenant or publisher under the umbrella handshake in which the device was onboarded, so that a name resolves to exactly one key and the binding is cryptographically verifiable by every party to that umbrella. The receiving-party constituent of an SI combination code issued at device granularity — or of an SC combination code bound to a Registered Counterpart Device — binds the Device Record; the resolver expands it to the fingerprint, and the device proves possession of the corresponding key.
Device Pass	An optional high-assurance registration by which one specific device of the responder organization makes itself addressable to the initiator beneath an established C umbrella: the device presents its tenant-signed Device Record over the sealed channel and proves possession of the device key; the initiator verifies the tenant signature against the responder's Directory Record and thereafter holds the record as a Registered Counterpart Device of the relationship, enabling SC references at device granularity that resolve only on that device (§XVI.13.7). Opt-in per device with consent on the device, revocable by either side, subject to expiry and renewal by policy.
Handshake Alias	A user-assigned local name for an established handshake or sub-handshake (equivalently, for the WR Entry it rests on) — for example "Calculus" for a particular SP sub-handshake. An alias exists only in the assigning party's own Orchestrator (or, by tenant policy, shared within the tenant); it never appears on a carrier, is not part of any reference, carries no check, and confers no authority. Speaking or entering an alias resolves it locally to the underlying established handshake and its canonical reference; everything after that local lookup — gates, grant, consent, assurance — is identical to entering the reference itself. Aliases are unique within their scope, freely renamable and removable, and dangle harmlessly when the handshake ends (the alias then resolves to the terminal state).
Displayed	The one normalized domain a publisher declares for visual display on its WR Code®

Term	Technical meaning
Responsible Domain	composites, so that a user can see, before any capture, which domain is responsible for the offering. Normalization is strict: the registrable base domain only — no scheme, no "www." or any other subdomain, no path, port, or query — lowercased, with internationalized names compared in their ASCII (punycode) form. The value is declared as <code>display_origin</code> in the publisher's Namespace Directory Record and must correspond to one of the publisher's DNS-verified origins. On a carrier it is plain display text: never a link, never tappable, not part of any reference, and contributing nothing to the check; its presence is verified after resolution (§XVI.9.4), and its absence — on textual, audio, transcribed, or other non-visual carriers — has no effect on validity.
Device Class	A registered classification of a paired device — smartphone, laptop, workstation, smart glasses, wearable, sandbox PC, server, gateway, machine, robot, and further classes registered later — recorded in the Device Record at pairing and available to policy (for example, "clone BEAP messages only to devices of class sandbox PC", "smart glasses may open SE contexts only").
Principal Key	A signing/encryption key pair held by an individual principal. For a principal acting on behalf of a publisher, the principal's public key is certified by the publisher's directory-registered key (a Delegation Certificate), so that counterparties and the Relay verify the delegation chain rather than requiring the publisher root key to sign every capsule.
Delegation Certificate	A publisher-signed statement binding a Principal Key, together with the principal's SSO-verified email address, to the publisher's Publisher Identifier, with scope (initiate, accept, revoke), generation, and expiry. Valid only while the email's domain is one of the publisher's DNS-verified domains (§XVI.6.5). Included by value or reference in every capsule, claim, and acceptance record signed by a delegated principal.
Publisher Identifier	A namespace identifier associated with a verified publisher or responsible namespace. The reference profile uses six symbols of the reference alphabet. The fixed length is a profile choice; the architecture requires deterministic namespace resolution rather than a specific number of symbols.
Local/Object/Binding Reference	The third block of a WR Code® reference (prefix, publisher block, third block, check). For P it is a publisher-local entry reference (five symbols by default). For C it is the responder's Publisher Identifier, so that the reference names the ordered pair of organizations. For I and for every sub-handshake class — SP, SI, SC, SE — it is a six-symbol combination code issued by the publisher that binds two constituents into one identifier: the WR Entry the sub-handshake is about and the identifier of the receiving party at the granularity the class permits: the Publisher Identifier of a responding organization (SC), the Principal Identifier of an individual on any of their paired devices (SP, SI), or — for SI, and for SC where a Device Pass exists — the Device Record of one specific paired device (§XVI.2, §XVI.13.7). Device granularity otherwise exists solely inside the tenant, because only the tenant holds the signed Device Records of its principals; a publisher addresses a user as a principal, and the initiator of a C relationship addresses the responder organization, whose Orchestrator routes internally by its own policy, unless a device of the responder has registered itself with a Device Pass. The granularity is fixed in the issuer's entry record and applied by the resolver on expansion; it is not visible in the code. The per-class meaning of the entry constituent: I — the tenant-internal umbrella context (workspace, department, document room) bound to the principal, or the specific paired device, for whom the internal umbrella is issued; every I code is thereby addressed, there being no public internal offering; SP — the specific object, resource, action, or purpose offered beneath the publisher's P offering; SI — the internal object, resource, workflow, or automation within the tenant context, with the receiving party at principal or device granularity as the issuer chooses; SC — the object, resource, action, or purpose shared within the C relationship (receiving party = the responder organization), so that each sub-handshake beneath a C umbrella has its own entry; SE — the session-scoped object, view, or action exposed under the parent handshake (§XVI.5.7). How the publisher derives the combination code is publisher-internal (§XVI.5.10); the resolver expands it back into entry and receiving party and verifies each (§XVI.6.5), and the receiving party's runtime can confirm from the expansion that the code is addressed to it before anything is retrieved (Gate 4, §XVI.7.6).
Reference Alphabet	The symbol set of the address body. The reference profile uses Crockford Base32 (digits 0-9 and letters A-Z excluding I, L, O, U; case-insensitive; input aliases I/i/L/l → 1 and O/o →

Term	Technical meaning
	0). Each symbol has an integer value 0-31.
Prefix Grammar Registry	A versioned table, trie, deterministic state machine, or functionally equivalent definition that marks consumed class-prefix sequences as TERMINAL, NON_TERMINAL, or INVALID, and assigns each terminal prefix a resolver/request-state branch and a class value sequence for error detection.
Terminal Prefix	A complete class-prefix sequence that determines the branch for the active grammar version.
Non-Terminal Prefix	A valid but incomplete class-prefix sequence requiring one or more further class/subclass symbols. In the reference profile, S is non-terminal.
Class Value Table	The registered mapping of each class-prefix symbol to an integer value 0-31 used as error-detection input (§XVI.5.4).
Check Character	The terminal error-detection element of a human-enterable textual reference. It detects capture/transcription error and supplies no authentication, entitlement, consent, relationship proof, or execution authority.
Carrier Token	A registered machine-readable token, tag identifier, index, or pointer mapped by the resolver to a WR Entry. A machine-only token is not required to masquerade as a textual WR Code® reference.
Namespace Directory	A signed, distributable directory mapping a Publisher Identifier to its resolver endpoint(s), publisher verification key(s), declared reference-profile parameters (grammar version), and namespace status (§XVI.6.4). Records are cacheable, but a cached record is valid only while its generation matches the current generation; any change to a record (key rotation, endpoint change, status change, domain change) increments the generation and invalidates every cached copy, so that no client ever acts on a superseded record within its nominal expiry.
Namespace Directory Record	The signed record for one Publisher Identifier as returned by the Namespace Directory.
Invitation Entry	A C-class WR Entry created by an initiator in its own namespace to offer a cross-organizational handshake to one or more named responders. Its lifecycle begins in PENDING and, on acceptance, the same identifier becomes the durable relationship entry (§XVI.7.5). Resolution of an Invitation Entry is recipient-bound: pre-consent material is released only to a principal whose delegation chains to a named responder.
Request Instance	A signed request_instance_id or equivalent unique identifier attached to a pending handshake request so that repeated processing is idempotent. It is an internal protocol value, never rendered as a textual reference.
Pending Handshake-Request Capsule	A signed data structure attached to an Invitation Entry, created by the initiator, containing request_instance_id, participant binding, requested profile/scope, the initiator nonce encrypted to each responder (§XVI.13.6), a commitment H(nonce_I) in the clear, freshness, and expiry data, and the initiator's Delegation Certificate where applicable. It is stored by the initiator's resolver (or an admitted Relay) for recipient-bound retrieval; no intermediary sees a nonce in the clear.
Relay	The sealed store-and-forward transport of the architecture. It holds Pending Handshake-Request Capsules and acceptance records for delivery between parties and releases each item only to a principal that authenticates as its intended recipient. The Relay carries only signed, recipient-bound, end-to-end-encrypted material; it cannot read nonces or protected payloads, is not a trust root, and grants nothing. In the reference profile the Relay is operated as part of the publisher's or the platform's infrastructure and is named in the Namespace Directory Record; a publisher's own resolver may embody the Relay function for its namespace.
Orchestrator	The local governed runtime that owns the trusted UI, the local principal's authentication state, the Handshake State Index, session management, and consent surfaces (the host Orchestrator). A second instance, the sandbox Orchestrator, is where original external content (attachments, non-WR links) is opened via the BEAP clone mechanism; it runs as a VM on the host in the standard tier (opening after an explicit warning) or in a hardware-isolated sandbox in the high-assurance tier; the recommended default is to open all links

Term	Technical meaning
	and attachments there, and deviations such as whitelisted verified URLs that open directly are explicit super-admin policy decisions (§XVI.7.4). It consumes resolver and Relay results; it is not required to mediate basic public namespace verification (§XVI.4.3).
Handshake	A durable, mutually accepted relationship/consent context between identified parties, defined by a profile, a grant scope, and lifecycle state. Established by acceptance/countersignature of a request; ended only by explicit revocation, dissolution, expiry, or supersession.
Grant	The scope of capabilities, data categories, and effects that a handshake authorizes. Operations outside the grant require fresh consent or a new handshake.
Acceptance / Countersignature	The explicit, assurance-gated act by which the responder accepts a verified request, producing a signed acceptance record over the request material that carries the responder nonce encrypted to the initiator and H(nonce_R) in the clear. Both parties hold both signatures after establishment.
Handshake State Record	Authenticated state describing a handshake or relationship, including both parties' Party Bindings (with verified email addresses), profile, lifecycle state, generation/freshness information, grant scope, the Establishment Commitment (§XVI.13.6), and, where applicable, session/login epoch information.
Handshake State Index	The Orchestrator's local, authenticated cache of Handshake State Records, keyed as described in §XVI.7.3.
Establishment Commitment (EC)	A domain-separated hash over all parties' Party Identifiers, the request instance, a nonce contributed by each party, and the signed request/acceptance material, computed identically by both parties at establishment and optionally anchored (§XVI.13.6).
Session	A temporary runtime state established under an applicable handshake. Ending a session does not by itself dissolve or revoke the underlying handshake.
Umbrella/Parent Handshake	The governing higher-level relationship or publisher context beneath which object-, action-, or purpose-specific sub-handshakes may operate.
Addressable Sub-Handshake	An independently text-addressable subordinate handshake. The reference profile uses SP, SI, or SC for durable sub-handshakes and SE for ephemeral, session-scoped ones; the second prefix symbol exposes the parent/domain class (or, for SE, the ephemeral lifecycle) while the exact parent relationship and object binding remain resolver data.
WR Site Discovery Record	Validated metadata binding a visited origin to a publisher/namespace, manifest, generation, expiry, and offered entry references.
Signed Site Manifest	A publisher-signed page-to-entry map for one origin: it lists the pages or page patterns of the origin and, for each, the WR Entry reference(s) offered there, with generation and expiry. It answers only "which entry does this page offer" and is used by the browser integration to turn a page visit into an entry candidate (§XVI.7.2). It is not the entry's repository context (LoRA adapters, agent instructions, capability declarations, policies — those are resolver data behind the entry, released only after verification and within the grant), and it is not the WR Link declaration (see WR Link). Its signing key is validated through the Namespace Directory and the origin binding (§XVI.6.5).
WR Link	A site-owner-declared anchor point on a page at which registered WCR recordings, arranged under a consented handshake, attach. WR Links are the navigation graph of a site for governed automation: when a user under an applicable handshake reaches a page, the runtime routes along the declared WR Links according to the available WCR recordings and the consented handshake context, so that predefined AI automation paths can run on that site. WR Links are declared in a separate, publisher-signed WR Link declaration that references the entry(ies) under whose grant the automations are permitted; they are consumed only after the entry has been verified and the handshake is active, never during discovery. WR Links are not confined to the page: depending on the active handshake context, the same previously agreed automation links may be surfaced by the trusted runtime in the WR Overlay (rendered over the page or the physical scene) or in the adaptive WR Menu, which lists, for the current umbrella/sub-handshake context, exactly those WR Links whose automations the user has consented to and which are applicable here. The overlay and the menu are runtime-rendered from validated state (P12); they

Term	Technical meaning
	show WR Links, they never receive them from page content. This annex covers only the point at which a WR Link declaration is admitted (entry verified, handshake active); the automation model itself is described in the companion annexes.
Email Capture Record	A typed local handoff from a mail integration to the runtime containing message context, provenance result, and extracted/derived reference candidates. It is capture metadata, not authority.
Capture Record	The typed local candidate-processing structure into which every capture path converges (§XVI.7.1).

§XVI.3 Architectural Invariants

P1 - Transport agnosticism, not payload equivalence. The same WR Entry may be reachable through different admitted carriers, including visual, textual, acoustic, radio, and document-based ones and carriers admitted after this disclosure, but carrier technologies are not assumed to have equal payload capacity. A profile may use direct identifier transport or a registered pointer.

P2 - Reference, not authority. A WR Code® reference, carrier token, transcript, discovery record, check result, or anchored commitment identifies a candidate or state to validate. It does not itself create a credential, relationship, entitlement, consent, session, or execution right.

P3 - Namespace and reference-kind determinability. The responsible publisher/issuer/namespace and the applicable reference class are determined before authoritative use. The class is determined from the prefix; the namespace is determined from the Publisher Identifier through the Namespace Directory or, for pointer carriers, through the registered token-to-entry binding.

P4 - Recognition and syntax validity are not execution. Successful capture, prefix parsing, and error-detection validation establish only syntactic admissibility. Trusted use requires authenticated resolution and the applicable handshake/consent state.

P5 - Extensible prefix-state parsing. Every human-enterable textual reference begins with a class prefix. A parser distinguishes complete, incomplete, and invalid class sequences. A UI closes a prefix block only after a terminal prefix is recognized, allowing future classes and subclasses without changing the address body.

P6 - Universal class-aware textual error detection. Every human-enterable or human-transcribable textual reference defined by the reference profile includes an error-detection character computed over its class values and its normalized address body and validated before remote resolver or request-state lookup.

P7 - Common capture boundary. Keyboard, paste, OCR of images and video frames, speech/audio transcription, email and messaging extraction, website discovery (page-to-entry mapping and on-page textual references), documents, and direct textual machine payloads converge on the same normalization, prefix parsing, and check-validation boundary. Reference kind is determined by the prefix, not by which capture path supplied the candidate. Candidate detection on all of these surfaces is always on; only insertion of a detected candidate into the visible entry field is user-toggleable (§XVI.7.7).

P8 - Resolver separation. Object identity, relationship state, capabilities, policy, access scope, assurance, and execution rights are authoritative resolved/handshake data rather than properties inferred from code syntax.

P9 - Pointer carriers and proximity are candidates. Fiducials, beacons, NFC tags, and other pointer carriers may identify what to resolve but do not prove who is entitled to use the result.

P10 - Privacy by non-emission. Discovery does not require a user device to broadcast a stable public user identifier. Public identity/repository checks should avoid unnecessary user-identifying telemetry where entitlement is not required.

P11 - Durable identifier permanence. A durable published WR Entry reference is not reassigned to an unrelated referent after activation/publication. Terminal or superseded state remains associated with the original identifier.

P12 - Trusted re-rendering. Externally received artwork — a code graphic on a page, in an email, in an attachment, in a screenshot, on a sign, or on any carrier — is capture input: the runtime may OCR-scan it, decode it, or capture it directly (including from screenshots the Orchestrator takes itself), and the textual reference so obtained enters the ordinary capture pipeline. What the artwork never is, is a source of trust. Every visual WR

Code® the runtime presents to the user in a trusted surface (the code itself with its Outer Shape and optional QR/Data Matrix element, the verified publisher identity, the offering, the handshake context, overlay and menu) is generated locally from state the runtime has itself verified — directory record, entry state, capsule — and externally supplied artwork is never displayed inside the trusted surface, so nothing painted onto a carrier (a logo, a fake verification mark, an imitation of the trusted UI) can borrow the runtime's credibility. The locally rendered visual is also what other devices scan in a cross-device handoff. Whether a captured reference is a valid handshake offering is decided by resolution and the gates of §XVI.7.6, never by the carrier: a code read from a video, spoken in a podcast, photographed on a sign, or OCR-scanned from a screenshot is exactly as valid as any other once its namespace and entry verify. For **websites and email** one additional rule applies, and it concerns only automation of capture, not validity: automatic detection and (where enabled) automatic insertion of an offering from a page or a message require that the reference was placed there through the secure WR Connect integration — the signed WR script block (§XVI.7.4a) — so that the runtime can verify publisher signature and origin/domain binding before it surfaces anything on its own initiative. A reference on a page or in a message that lacks such a block is not auto-surfaced; the user can still capture it manually (select, OCR, dictate), and it then goes through the same verification as any other capture and, if it passes, is the same valid offering. Because these two carriers are generated by the publisher's own tooling, the integrated proof costs the publisher nothing and adds a signature-backed provenance that a graphic cannot provide.

P13 - Invitation reference is not invitation proof. Knowledge of a C-class reference does not prove that an invitation addressed to the holder exists, that the named initiator created it, or that it is still pending. Recipient-bound release, signed request material, verification, and required acceptance establish the relationship.

P15 - No external links. A WR Code® reference is never a URL, never embedded in a URL, and never delivered as a clickable link on any carrier. The reference is resolved only by a conformant runtime through the Namespace Directory; it does not point to a web location, and no carrier profile may define a link-based transport. Repository links (§XVI.4.3) that let a user inspect a publisher's record on the WRC are not affected: they are constructed by the runtime from a pinned endpoint and the validated identifier, never taken from a carrier, and lead only to the WRC publisher record in the runtime's verification view. A Bootstrap Payload for non-WR scanners (§XVI.11) is likewise not a WR transport: a conformant runtime never reads it before verifying the textual reference and never follows it afterwards. Links contained in incoming email or documents are not opened from the host Orchestrator's trusted surface; if a user chooses to open one, the BEAP clone mechanism redirects it by default to the sandbox Orchestrator. Policy may whitelist verified URLs to open directly — an explicit super-admin deviation from the recommended out-of-band default (§XVI.7.4). This removes the phishing, redirect, and look-alike-domain attack surface that link-based invitations carry: there is nothing to click, nothing to hover, and no destination to spoof — only a checked reference that the runtime resolves against signed directory state.

P14 - Evidence without disclosure. Proof that a handshake was established is a domain-separated commitment over Party Identifiers, request instance, and nonces contributed by every party. Nonces are confidential to the parties end to end; no intermediary (Relay, directory, commitment service, ledger) ever holds a nonce in the clear. Only the commitment may be stored outside the parties or anchored; the commitment is unlinkable to the parties without their nonces, and neither its existence nor its anchoring confers or preserves any authority.

§XVI.4 Resolution and Validation Model

§XVI.4.1 Processing stages

Stage	Function
1. Discovery / selection	Identifies what entry, object, page, message, or nearby candidate may be intended. Examples include browser navigation, vision, OCR region detection, pointer/gaze selection, BLE proximity, manual initiation, or speech.
2. Capture / transport	Produces either a complete textual reference or a carrier token according to the admitted carrier profile.
3. Local parsing / error detection	Normalizes presentation, determines class, validates prefix completeness and the terminal check, and rejects invalid candidates before network effects.
4. Namespace resolution and publisher verification	For a textual reference, obtains the Namespace Directory Record for the Publisher Identifier(s), validates its signature and status, confirms that the record identifies a verified account holder, and confirms domain control through a valid DNS entry under the publisher's domain (§XVI.6.5); the namespace is verified only if all of these hold. For class C the stage is performed for both Publisher Identifiers in the reference. for a pointer carrier, obtains the registered token-to-entry binding.

Stage	Function
5. Entry verification	Maps the validated reference or token, via the resolver endpoint identified in Stage 4, to the entry record in the publisher's repository. The entry context must exist there and match exactly the assignment (binding) and version the reference designates; a reference whose entry is absent, whose binding differs, or whose version is superseded without an explicit successor is not verified. For sub-handshake classes the combination code is expanded and its entry constituent and its receiving-party constituent are each verified against the repository. Returns lifecycle state (including PENDING for an Invitation Entry) and profile-specific context.
5a. Recipient-bound retrieval	Obtains the signed request capsule for the entry from the Relay named in the publisher's directory record. The Relay releases it only to an authenticated intended recipient; the runtime verifies signature, delegation chain, Party Bindings, freshness, and request instance locally. Acceptance records travel back to the initiator by the same sealed path.
6. Trust, counterparty, and entitlement validation	Validates publisher/domain bindings, signatures, recipient/relationship identity, freshness, revocation, grants, attestation, session/login state, the authentication assurance the profile demands of either or both parties (for example passkey, hardware-key, biometric, or multi-factor confirmation, or any other proof the issuer or the handshake profile makes mandatory), and other profile requirements. Verifies the non-publisher counterparty where present: an individual party has no DNS and is verified as an authenticated account holder with a verified email address.
7. Verified offer / state and deliberate decision	Presents a trustworthy invitation, existing relationship state, active context, re-authentication requirement, or terminal state. Explicit consent or acceptance is obtained for every new umbrella/parent handshake. Depending on the applicable policy, reuse within an existing grant and entry into a sub-handshake fully covered by the umbrella's consented grant may proceed without a new consent step; policy may equally require a fresh consent or a re-authentication for either.
8. Authorized use / execution	Protected information is released or an effect is authorized only within the validated grant and the applicable execution and finalization rules. The grant is the concrete content of the handshake: it enumerates the governed assets and actions the parties have consented to — such as hashed model weights and adapters (for example safetensors files identified by content hash), configurations, AI instructions and prompt profiles in text form, WCR workflow recordings, semantic and spatial context (Publisher Semantic Layer, semantic graph, Spatial Anchor Registry, Virtual Replica), normalized media, capability requests and offers, and automation triggers — the exact set depends on the case — together with the Proof-of-Authoritative-Execution (PoAE™) and Proof-of-Authoritative-Capability (PoAC™) requirements that apply to each. Every release and every effect travels as a BEAP™ capsule beneath the handshake, never as a side channel (§XVI.4.2a).

§XVI.4.2a Content of a grant and governed release

A grant is where the handshake becomes valuable: it turns a verified relationship into an enumerated, consented set of assets and actions, each with the proof regime under which it may be used. The grant is established in the acceptance step of the umbrella handshake (§XVI.13), may be narrowed by the responder at acceptance and by any party's policy afterwards, and is inherited or refined by sub-handshakes beneath the umbrella. Its content is open and depends on the case. Every grant item is a repository asset that is cryptographically versioned and hash-anchored: each version carries a content hash, versions are chained in signed, Merkle-anchored manifests, and no version is ever silently replaced. Nothing in a grant is executed, rendered, loaded, or applied before two conditions have been checked locally on the consuming device: the artifact's hash equals the hash the grant names for it, and the grant itself is valid at the time of use (signed, unrevoked, unexpired, within scope and use limits). A failed hash check or an invalid grant is a hard stop, never a fallback to an older or unverified version. The following inventory collects the asset classes the corpus defines; it is illustrative, and any data that can be transported securely under the invariants of this specification — recipient-bound, sealed, text-pure or normalized and hash-pinned — may be made available as a grant item, whether or not it belongs to a class listed here:

- **Parametric artifacts** — model weights, LoRA and other adapters, specialized models, tokenizers, vision adapters of a Recognition Package, identified by content hash (for example a safetensors file addressed by its hash) so that what was consented to is exactly what is released and executed; a changed artifact is a new grant item.
- **Procedural artifacts** — skills, WCR workflow recordings (deterministic at tool boundaries, adaptive on the path between them), workflows, pre-declared action chains, validators, tools, prompt profiles, AI

instructions and briefing templates in text form (text-pure by construction, therefore carried as BEAP™ text, never as code), pointer actions, and WR Link™ declarations binding recordings to declared anchor points.

- **Declarative sources** — retrieval corpora and their index segments, semantic chunks, FAQ and reasoning units, documents and data sets, templates; and the **context** itself: a Context Graph contributed from a party's vault, Context Frames, and the **Publisher Semantic Layer** with its structural declarations (semantic regions, entities and relations, navigation graph and transitions, workflow states, capabilities and preconditions, Semantic Anchors and positional references, permitted overlay declarations).
- **Spatial context** — Scope Manifests, the Spatial Anchor Registry (fiducial records, mapped positions, verified associations, pose updates), Virtual Replicas of environments and objects, and recognition descriptors — the spatial semantic graph by which physical objects are designated and navigated.
- **Presentation assets** — normalized media (images, graphics, and, where admitted, video, converted at ingest to pinned formats and referenced by hash), infographics and composed frames, declarative Mini-App compositions from pre-admitted primitives, overlay asset classes requested by type, derived composition assets such as icons and charts, and layout candidates.
- **Capability requests and offers** — a party's request that the counterpart provide, or offer to provide, any of the above, governed by PoAC™ (below).
- **Automation triggers and effects** — actions the counterpart may cause to be executed, and the conditions under which they execute, governed by PoAE™ (below).
- **Configurations and policy** — orchestrator, sandbox, tool, connector, and policy configurations, attestation tiers, and the scope, use limits (§XVI.8.4), expiry, revocation, and assurance terms (§XVI.13) that apply to each grant item.

Objects exchanged *within* a relationship — messages and their clones, submitted forms, handed-over files — are not repository assets and not grant items; they are payloads that flow under the grant, subject to the sandbox and clone rules of §XVI.7.4.

Two proof regimes are attached to grant items:

- **PoAE™ (Proof of Authoritative Execution)** governs effects. An effect authorized by a grant is executed only after a human authorization of the grade the grant and the risk class demand — from simple confirmation to passkey, hardware-key, or attested confirmation — and the authorization, the exact artifact hashes and instructions involved, and the outcome are recorded as cryptographic proof. AI prepares; a human authorizes; the system records. A grant may fix the PoAE™ grade per effect, per asset class, or per counterpart.
- **PoAC™ (Proof of Authoritative Capability)** governs capability acquisition. A capability — a skill, a WCR recording, a source, a validator, a tool, an adapter, a prompt profile — enters a party's orchestrator only under a grant, backed by evidence of the need it fills, and by the rule *propose, never silently apply*: the capability is proposed, its provenance (hash, issuer, grant) is verified, and it is applied only after authorization. A grant may declare which capability classes the counterpart may request or offer and which PoAC™ evidence is required.

Paid offerings and the WR Store™. A publisher's WR Code® entry may equally offer commercial, entitlement-gated items — for example a pro version of a library, a pro LoRA adapter, a premium capability, or a subscription tier — distributed through the WR Store™ or any other governed channel. Purchase, entitlement, and billing are handled as part of the handshake and its grant like any other condition of release; the mechanisms of payment and entitlement are open and out of scope here. A paid item remains an ordinary grant item in every other respect: hash-anchored, released only under a valid grant, and subject to the same proofs and transport as everything else in this section.

All grant items, proofs, requests, and releases are transported as BEAP™ capsules beneath the handshake: text-pure, recipient-bound, sealed end-to-end, verified on every read path, with binary originals inert until opened under the sandbox rules. No grant item is released by any other channel, and nothing outside the grant is released at all. The textual reference that started the handshake carries none of this; it is the pointer to the relationship within which the grant lives (§XVI.4.2).

§XVI.4.2 Resolver output and protected-data boundary

Resolution may be deliberately staged. Before consent or entitlement, the resolver returns only the information required to identify the responsible namespace, validate the candidate, determine lifecycle/request state, and

present an appropriate trusted offer or status. Internal, device, cross-organizational, or otherwise protected material may remain entirely within publisher-controlled or federated infrastructure. Global textual routability therefore does not imply public disclosure or public hosting.

A check-valid reference may still be inapplicable to the current user, tenant, relationship, device, or target. Such a context mismatch (for example, a "not for me" outcome, or "not for this device" where the issuer bound the entry to one specific device of the principal) is a policy/state determination distinct from capture-error detection.

§XVI.4.3 Independent namespace and identity verification

The Namespace Directory and the publisher resolver/repository are logically independent from the Orchestrator. Where the referenced metadata is public and no entitlement is required, a conformant client may perform namespace and entry-status verification directly against the Namespace Directory and the applicable resolver without first establishing an Orchestrator session and without disclosing a stable user identity merely to ask whether the namespace/reference exists. An Orchestrator may consume the same results as part of a richer handshake flow, but it is not required to be the sole trust intermediary for basic namespace or publisher-identity verification.

Completed publisher blocks may be rendered by the trusted runtime as **repository links**: runtime-generated links that open the authoritative publisher record for that Publisher Identifier on the Optirando™ WRC (Workflow Ready Cloud), where the public publisher and namespace information is hosted, so that the user can verify who the namespace belongs to before completing the reference. The destination is external to the device but not arbitrary: the runtime constructs the link deterministically from the validated Publisher Identifier and a pinned WRC endpoint that ships with the runtime — never from the carrier, the captured text, page content, or message content — and opens it in its own verification view. What P15 excludes are content-derived links whose destination an issuer or attacker controls; a repository link has a fixed, runtime-owned destination whose only variable part is the already-validated identifier. Repository links are a verification affordance for the user, not part of the identity model.

§XVI.5 Textual Reference Grammar, Parsing, and Error Detection

§XVI.5.1 Reference-profile textual forms

Reference class	Reference-profile form	Purpose
Main entry	P-PPPPPP-LLLLLX	Publisher entry; X is the terminal check character attached to the local block.
Internal umbrella	I-IIIIII-BBBBBB-X	Tenant namespace; third block = combination code of internal context + receiving principal or device; check rendered as a separate block. Every I code is addressed to a known principal or device of the tenant.
Cross-organizational relationship	C-IIIIII-RRRRRR-X	Ordered pair initiator/responder Publisher Identifiers; check rendered as a separate block. PENDING while an invitation, ESTABLISHED once accepted.
Sub-handshake (all)	ST-PPPPPP-BBBBBB-X; ST ∈ {SP,SI,SC,SE}	Independently addressable sub-handshake beneath the corresponding parent/domain class. BBBBBB is the six-symbol combination code of entry + receiving-party identifier; X is the check, rendered as a separate block. SE additionally has a bounded session lifecycle (§XVI.5.7).

PPPPPP, IIIIII, and RRRRRR are Publisher Identifiers of six reference-alphabet symbols. The local field LLLLL of P is five symbols in the reference profile; a profile may use a different fixed length (§XVI.5.11). BBBBBB is always six symbols. X denotes the check-character position, always the last symbol. For P the check is attached to the local block; for I, C, and every sub-handshake class the check is rendered as its own fourth block, so that the six-symbol third block (a Publisher Identifier or a combination code) stays clean for reading, dictation, OCR, and comparison. Presentation separators (hyphens) are not identifier material; all groupings and the ungrouped form MUST be accepted on input.

All reference classes share one structure: block 1 prefix, block 2 publisher identifier, block 3 entry / counterparty / combination code, block 4 check. Two rendering families follow from what block 3 contains: an unaddressed local entry (P — check attached) or an addressed block naming or binding a counterparty (I, C, and all sub-handshake classes — check separate). No reference class carries request identifiers or selectors; those are resolver and capsule data.

§XVI.5.2 Variable-length prefix grammar and UI block formation

Every textual reference begins with a class prefix. The prefix is parsed as a variable-length sequence. A Prefix Grammar Registry classifies each partial sequence as `TERMINAL`, `NON_TERMINAL`, or `INVALID`. The reference profile (grammar version 2) defines:

`P` → `TERMINAL` (entry, Publisher)
`I` → `TERMINAL` (internal umbrella, addressed)
`C` → `TERMINAL` (entry, cross-organizational relationship)
`S` → `NON_TERMINAL`
`SP / SI / SC` → `TERMINAL` (addressable sub-handshake)
`SE` → `TERMINAL` (ephemeral, session-scoped sub-handshake)
any other first symbol, or `S` followed by any other symbol → `INVALID`

A grammar version may register further terminal classes or subclasses. Unregistered first symbols, and `S` followed by an unregistered symbol, remain `INVALID`.

The interactive field follows parser state. If `P`, `I`, or `C` is received first, the UI may immediately insert the presentation hyphen because the prefix block is complete. If `S` is received, no hyphen is inserted; the parser waits for a second class symbol. Only after `SP`, `SI`, `SC`, or `SE` is complete may the UI close the prefix block. A future grammar version may register longer class/subclass paths, and the same rule repeats until a terminal state is reached.

Because the first symbol is always a prefix, no textual reference of the reference profile begins with a Publisher Identifier. A Publisher Identifier may therefore contain any reference-alphabet symbol, including `P`, `I`, `C`, or `S`, without ambiguity.

This state-driven block formation prevents premature formatting from being confused with classification and permits versioned extension of the class hierarchy without changing the publisher/local/check body or the resolver interface.

§XVI.5.3 Case normalization and address-space extension

Textual capture is case-insensitive. Alphabetic material is normalized to upper case before prefix parsing and check validation. Prefix parsing is performed before payload-alias normalization: the first one or two symbols are consumed by the Prefix Grammar Registry as class symbols, and only the subsequent address body is subjected to the Crockford input aliases (`I/i/L/l` → `1`, `O/o` → `0`). A class symbol `I` is therefore never rewritten to the digit `1`.

Class `I`, class `C`, and all sub-handshake classes have a fixed body of twelve symbols (publisher identifier plus a six-symbol third block); the local block of `P` is five symbols in the reference profile. Five reference-alphabet local/object symbols provide $32^5 = 33,554,432$ publisher-local references per applicable class namespace. Each additional local symbol multiplies that space by 32. The five-symbol local field is a compact default, not an architectural maximum. Six-symbol Publisher Identifiers provide $32^6 = 1,073,741,824$ namespaces; Publisher Identifiers are issued once and never reassigned (§XVI.8.2).

§XVI.5.4 Terminal check character for all human-enterable textual references

Every textual reference exposed to keyboard entry, paste, OCR, dictation, speech transcription, email copy, or another human-enterable path includes a terminal error-detection character. The covered categories are WR Code® Entry references of all classes, including `PENDING` `C`-class invitation entries and `SE` ephemeral sub-handshakes.

Check input. The check is computed over the sequence of integer values

`v = [ class values of the consumed terminal prefix ] || [ values of the normalized address body symbols ]`

excluding presentation separators and excluding the check character itself. Class values are taken from the Class Value Table of the active grammar version; address-body values are the reference-alphabet values (0-31). The reference profile Class Value Table equals the Crockford input-mapping value of each class symbol, so that class symbols need no second alphabet:

Class symbol	Value
P	22
I	1 (Crockford input alias of I)
C	12
S	25
E (second symbol of SE only)	14

For SP the class value sequence is [25, 22]; for SE it is [25, 14]. Because the class values participate in the check, a prefix substitution (SP for P, C for I, or a transposed pair in a C reference) changes the expected check symbol and is detected locally. Since the reference-profile Class Value Table coincides with the Crockford input mapping, the computation is equivalent to running the Damm algorithm over the complete ungrouped, alias-normalized string; the class-awareness of the check resides in the fixed prefix position and the per-class body length, not in a second alphabet. A grammar version that adds classes outside the reference alphabet assigns them Class Value Table entries explicitly.

Check algorithm. The reference profile uses the Damm algorithm over a totally anti-symmetric quasigroup of order 32, defined algebraically in the finite field $GF(2^5)$ with reduction polynomial $x^5 + x^2 + 1$ (0x25): $x * y = (\alpha \cdot x) \oplus y$, where $\oplus$ is bitwise XOR on the five-bit symbol values, $\cdot$ is field multiplication, and α is the field element 2 (the polynomial x). Total anti-symmetry follows from a single identity: both $(c * x) * y \oplus (c * y) * x$ and $x * y \oplus y * x$ equal $(\alpha \oplus 1)(x \oplus y)$, which is non-zero for $x \neq y$; the full 32×32 table (Appendix A.1) is derived from a one-line rule and needs no lookup storage. The interim is computed from initial state 0 as $\text{interim} \leftarrow \text{interim} * d$ for each value d of v ; the check symbol is the reference-alphabet symbol whose value equals $\alpha \cdot \text{interim}$ (the unique k with $\text{interim} * k = 0$); a candidate verifies if the interim over $v \parallel [k]$ equals 0. Test vectors, including negative vectors, are reproduced in Appendix A.2. This construction detects every single-symbol substitution and every transposition of two adjacent symbols anywhere in v , including across the class/body boundary and between the final body symbol and the check symbol; the fixed length of each class form additionally detects omitted or inserted symbols. A later grammar version may register another class-aware error-detection algorithm without changing the resolver, handshake, or consent architecture. A valid check indicates capture consistency only and is never authentication or authorization.

§XVI.5.5 Illustrative examples

The following are complete, registry-derived references of the reference profile (grammar version 2), one per class; the C-class example is the relationship initiated by ABC123 toward XYZ789. Check values are computed per §XVI.5.4 and reproduced with their check inputs in Appendix A.2.

Reference	Class values	Body values (Crockford)	Check input length	Check
P-WR7X4K-9B2M3C	[22]	W R 7 X 4 K 9 B 2 M 3	12	C
I-K4T9M2-7QFA3X-S	[1]	K 4 T 9 M 2 7 Q F A 3 X	13	S
C-ABC123-XYZ789-H	[12]	A B C 1 2 3 X Y Z 7 8 9	13	H
SP-WR7X4K-H2N5V8-7	[25, 22]	W R 7 X 4 K H 2 N 5 V 8	14	7
SI-K4T9M2-3ZDQ7E-H	[25, 1]	K 4 T 9 M 2 3 Z D Q 7 E	14	H
SC-ABC123-M8W2R4-0	[25, 12]	A B C 1 2 3 M 8 W 2 R 4	14	0
SE-WR7X4K-6TCJ9F-P	[25, 14]	W R 7 X 4 K 6 T C J 9 F	14	P

All of the following denote the same candidate after normalization: P-WR7X4K-9B2M3C, PWR7X4K9B2M3C, p-wr7x4k-9b2m3c, P WR7X4K 9B2M3C.

§XVI.5.6 Cross-organizational invitation references

A cross-organizational relationship is addressed by a C-class reference whose body is the ordered pair of Publisher Identifiers — initiator first, responder second — followed by a separately rendered check block, for example C-ABC123-XYZ789-H. There is no separate local block: the ordered pair itself identifies the umbrella entry. The second block names the namespace whose repository holds it (the initiator); the third block names the responder, so that the responder sees its own identifier directly in the code and can run its checks (Gate 4, §XVI.7.6) before anything is retrieved. Individual entries beneath the umbrella are addressed by SC references,

whose third block is the combination code of entry and responder identifier (§XVI.2, §XVI.5.1). It is captured, parsed, and checked exactly like every other reference and delivered on any admitted carrier, including linkless email (§XVI.7.5). Both Publisher Identifier blocks are verified independently against the Namespace Directory — account holder and DNS domain control for each (§XVI.6.5) — as they complete (§XVI.5.8), which is why the check is not attached to the second block. A C reference is treated as verified only when both namespaces pass.

There is exactly one C entry per ordered pair: the entry is created in PENDING state as the invitation and becomes the ESTABLISHED umbrella relationship on acceptance under the same identifier. A renewed invitation after termination re-uses the pair's identifier with a new request instance; the identifier itself is never reassigned. Because the reference names the responder, recipient-bound resolution releases pre-consent material only to a principal chaining to the second block's publisher. $A \rightarrow B$ and $B \rightarrow A$ are distinct references and distinct relationships. Multi-organization relationships are not expressed by additional body blocks; they are modeled as an umbrella with SC sub-handshakes or by the group-session mechanism of the Canon. Class C and class SC both have a fixed twelve-symbol body with the check as a separate block.

§XVI.5.7 Ephemeral sub-handshakes (SE)

A short-lived, session-scoped handshake context that is displayed, dictated, or otherwise made human-enterable is an SE reference: it is a sub-handshake like SP/SI/SC, with prefix SE, the issuing publisher's namespace, the six-symbol combination-code third block and separate check, and the same parsing, error-detection, directory, entry-verification, and consent rules. What distinguishes SE is lifecycle, not form: an SE entry exists only under an established parent handshake, is bound to a session or a bounded time window, is not durable, and resolves only while its session and expiry are valid; afterwards it is EXPIRED and its identifier is not reissued (P11). Its combination code binds the session-scoped object, view, or action being exposed to the receiving principal under whose established handshake the session runs. Promotion of an SE context to a durable SP/SI/SC sub-handshake creates a new durable binding (§XVI.13.2). Expiry, replay, session binding, and authorization remain separate state and are not replaced by the check.

§XVI.5.8 Candidate framing, submission, and class-dependent progressive verification

Three things are kept apart in the entry path: **completeness** of a candidate, **submission** of a candidate, and **progressive verification** of a namespace while the candidate is still being entered. They are decided by different mechanisms.

Completeness is determined by the grammar alone. The class prefix is parsed first; the class fixes the block lengths; therefore, once the prefix is terminal, the parser knows exactly when block 2 is complete, when block 3 is complete, and which symbol is the check:

Class	After the prefix, the parser expects	Complete when
P	6 (Publisher Identifier) + 5 (local) + 1 (check, attached)	12 body symbols received
I	6 (tenant Publisher Identifier) + 6 (combination code) + 1 (check, separate block)	13 body symbols received
C	6 (initiator) + 6 (responder) + 1 (check, separate block)	13 body symbols received
SP, SI, SC, SE	6 (Publisher Identifier) + 6 (combination code) + 1 (check, separate block)	13 body symbols received

No directory lookup, no policy, and no user action is needed to establish completeness or to locate the check; the UI inserts presentation hyphens at the block boundaries the grammar defines, and the check validates locally the moment the last symbol arrives. An incomplete candidate is simply incomplete; an over-long one is INVALID.

Submission is an explicit act. Reaching completeness does not trigger any network access. The first contact with a directory, resolver, or Relay for the full candidate happens only on submission: Enter or a submit control in the entry field, the end of a dictated segment confirmed by the user, the end of an OCR region confirmed in the capture indicator, or — for automatically detected candidates from a verified signed script block (§XVI.7.4a) — the auto-insertion event itself where the user has enabled auto-insertion. Submission is recorded as the SUBMITTED state (§XVI.7.7). Until then the user can correct the candidate, and a mistyped code causes no lookup anywhere. A check failure blocks submission and is shown as a capture error.

Progressive verification is a class-dependent convenience, never a requirement. Progressive verification means: after block 2 (and, for C, block 3) is complete but before submission, the runtime looks up the Namespace Directory Record for the Publisher Identifier and shows the verified organization and domain next to the block, so

the user can see that the code points where they expect before submitting. Whether this is done depends on the class, because it depends on whether the namespace record is public and on what a pre-submission lookup would reveal:

Class	Progressive verification	Reason
P, SP	Yes, by default; may be disabled by user or tenant policy.	The publisher's directory record is public, the lookup is namespace-keyed without any user identity (P10, §XVI.4.3), and the preview lets the user catch a wrong Publisher Identifier before submitting.
C, SC	Yes, for both Publisher Identifier blocks, by default; may be disabled by policy.	Both blocks are organization namespaces with public directory records; the responder immediately sees "invitation from Organization A to my organization" and can spot a look-alike identifier before submitting. The preview shows only public directory data — never whether an invitation exists or is pending (§XVI.6.3).
I, SI, SE	No.	The namespace behind these classes is not public; a lookup before submission would show nothing meaningful to the user and would reveal to an observer of directory or resolver traffic that someone holds a code from that tenant. The candidate is framed and checked locally, and every lookup happens only after submission, inside the gates of §XVI.7.6, immediately before the consent dialog.

Where progressive verification is performed, it is candidate verification only: an unfinished reference is never promoted to a trusted entry, no pre-consent material is fetched, and the preview is rendered by the runtime from the directory record (P12). Where it is not performed, the user loses nothing in assurance: namespace, entry, and counterparty verification are always carried out in full after submission and before anything is offered for consent (§XVI.7.6), regardless of class.

Per-class entry experience. For the avoidance of doubt, the resulting behaviour per class is:

- **P offering typed or dictated by a user:** prefix P → hyphen; six symbols → hyphen, and (unless disabled) the publisher's verified name appears; five symbols + check → the check validates locally; the user presses submit → gates 2 to 6 run → consent dialog.
- **SP sub-handshake (e.g. a specific offering under a known publisher):** as for P, with the combination code as block 3 and the check as a separate block; the preview shows the publisher only, never the expanded entry or receiving party.
- **I pairing handshake (device onboarding):** prefix I → hyphens at the grammar boundaries; no preview; the context constituent of the combination code is a Pairing Slot and the receiving party is the principal. The user enters the code on the new device, which authenticates as the same SSO Identity, generates its device key, reports its Device Class, and is given its Device Name; submit → gates → pairing consent → Device Record created, slot consumed.
- **Device-directed SI sub-handshake (or SC to a Registered Counterpart Device):** the issuer selects the target device by name from its device list (the tenant's own devices, or the counterpart devices registered by Device Pass in the C relationship); the prefix stays SI or SC and reveals nothing about the granularity; submit → gates → the request resolves only on the named device (NOT_FOR_THIS_DEVICE elsewhere) → consent on that device.
- **I internal umbrella or SI internal sub-handshake:** I-IIIIII-BBBBBB-c / SI-IIIIII-BBBBBB-c; no preview; local check only; submit → gates (including tenant policy and, for device-bound codes, Device-Scoped Principal Identifier match) → consent dialog or inherited entry (§XVI.13.2).
- **C invitation:** prefix C → hyphen; initiator block → preview of the initiator organization (unless disabled); responder block → preview of the responder organization, which should be the user's own; separate check → validates locally; submit → gates 2 to 6 including recipient-bound release → consent dialog.
- **SC sub-handshake under a C relationship:** initiator block preview as for C; combination code as block 3, no expansion shown; check; submit → gates → consent dialog or inherited entry.
- **SE ephemeral sub-handshake:** no preview; local check; submit → gates → session-scoped context, subject to expiry.
- **Automatically detected candidates (signed script block on web or email, sensor capture):** the same framing and check rules apply; the candidate is shown in the capture indicator, is inserted into the field

only if auto-insertion is enabled, and is submitted either by the auto-insertion event (for verified script-block candidates) or by the user's explicit action.

§XVI.5.9 Voice, audio, transcriber, OCR, and manual capture

Keyboard input, paste, OCR of still images or video frames, and speech transcription — from live microphone input, a phone or conference call, a voice assistant, or the audio track of a recorded or streamed video or broadcast — feed the same parser/check boundary. A transcriber may emit characters, spoken separator tokens, and confidence metadata. The runtime removes non-semantic separators, normalizes case, advances the prefix grammar, applies payload aliases to the address body, and validates the final check. An incomplete prefix, failed check, or insufficient recognition confidence causes correction/repetition or explicit confirmation rather than silent guessing.

In a speech-driven implementation, recognition of the spoken WR Code® activation phrase ("WR Code" followed by the reference, or a profile-defined equivalent) acts as a **wake cue**: it wakes the local WR processing surface and causes the following speech segment to be transcribed as a candidate reference. The same wake-cue mechanism applies to audio that is not addressed to the device at all — a presenter reading a reference aloud, a podcast, a video's sound track, an announcement — because the runtime listens for the cue, not for the speaker. Wake-up does not itself insert, resolve, accept, or execute the candidate; it only opens the capture window described in §XVI.7.7. A speech-driven implementation may equally accept a **Handshake Alias** (§XVI.2) after its assistant wake phrase — "OK Opty, load Calculus" — to recall an *established* handshake or sub-handshake: the alias is resolved in the local alias table, never against the network, and yields the canonical reference and relationship state, from which the runtime proceeds exactly as if that reference had been entered — for example, presenting the entry's WR Code® composite for the WR Pointer™, or opening the session surface the grant permits. An alias can never establish a new handshake, reach a foreign namespace, or bypass a gate; an unknown alias fails locally with no network access. The same command surface docks and undocks sub-handshakes into the current context (§XVI.13.4): "OK Opty, add Calculus" docks the aliased sub-handshake alongside those already active, "OK Opty, remove Calculus" undocks it, and "OK Opty, load Calculus" recalls it as the primary context — all subject to the docking rules and the assurance the grant demands.

§XVI.5.10 Reference-profile class semantics

Class	Resolved role
P / SP	P identifies a Publisher entry — an offering made by a publisher to any resolving party (the "public" handshake class), in the form P-PPPPPP-LLLLLX with the terminal check character X attached to the local block and validated locally before any lookup, exactly as for every other class. SP identifies an independently addressable sub-handshake beneath a Publisher parent (SP-PPPPPP-BBBBBB-X, check separate; BBBBBB = combination code of entry + receiving-party identifier). Global addressability does not require unrestricted disclosure. I identifies a tenant-internal umbrella context (I-IIIIII-BBBBBB-X, check separate): a workspace, department, workflow, automation, document room, a server or machine of the tenant, or a Pairing Slot for onboarding a device. Block 2 is the tenant's Publisher Identifier; block 3 is the combination code of the internal context + the receiving party, which is a principal of the tenant or one specific paired device of that principal (Device Record). There is no unaddressed internal offering: every I code is issued for a known principal or device, which is why I uses the addressed form with a separate check rather than the P local form. Device onboarding is an I pairing handshake. A device joins a principal's context through an I code whose context constituent is a Pairing Slot: an entry the tenant creates in its own namespace meaning "one device of this principal may pair here", optionally with an expected Device Class and an expiry; the receiving party is the principal, who is known — the device is not, so the code carries no device constituent. The new device enters the code, authenticates as the same principal (the same SSO Identity, i.e. the same verified email address), generates its own Principal Key under that SSO Identity, presents the key together with its Device Class, and receives its Device Name (chosen by the user, or defaulted); the tenant checks the class against the slot's expectation, registers key, class, and name, and at that moment the Device Record and the Device-Scoped Principal Identifier come into existence (§XVI.2). The slot is single-use by construction (§XVI.8.4, consume_at = acceptance) and is superseded by the resulting Device Record, which is signed by the tenant under this I umbrella — that signature is what makes a Device Name cryptographically unambiguous to every holder of the umbrella. A pairing never introduces a second identity; a device presenting a different email identity cannot be paired and would require its own handshake under its own SSO Identity. A private user without a tenant pairs devices in the same way beneath the P handshake of the publisher with which the user registers, the publisher acting as issuer of the pairing; the resulting Device Records serve the user's own device management and receiving-side binding (§XVI.13.7), not device-level addressing by the publisher. A machine, server, gateway, robot, or sandbox PC of the tenant is simply a paired device with a Device Record of the
I / SI	

Class

Resolved role

corresponding Device Class; offered externally, it is a P or SP entry of the publisher. Devices therefore need no reference class of their own. An earlier draft used the tenant identifier twice; the second occurrence is now the combination code. SI identifies an internal sub-handshake beneath an I umbrella (SI-III-III-BBBBBB-X, check separate; BBBBBB = combination code of the internal entry + receiving party at principal or device granularity). Access *from* a device *to* an internal resource is therefore an SI sub-handshake addressed to that device; delivery *to* a named device is an SI sub-handshake issued at device granularity (§XVI.13.7); SP has no device granularity, and SC has it only toward a device that has registered itself by Device Pass. Protected knowledge may remain entirely within company infrastructure even though the textual reference can be routed globally.

C identifies a cross-organizational umbrella/relationship entry named by the ordered pair initiator/responder Publisher Identifiers: PENDING while it is an invitation, ESTABLISHED once accepted, and terminal thereafter; the identifier does not change across these states (P11). SC identifies an individual subordinate handshake beneath such a pair (SC-III-III-BBBBBB-X, check separate); its

C / SC namespace block is the initiator and its third block is the combination code of that sub-handshake's entry and the responder's identifier, so that every sub-handshake under the umbrella has its own entry and the responder's runtime can confirm from the expansion that the code is addressed to it before anything is retrieved. The local reference may resolve the counterparty, relationship, object binding, access policy, capabilities, and hosting context, released recipient-bound.

SE identifies an ephemeral, session-scoped sub-handshake beneath an established parent handshake of any class (SE-PPPPPP-BBBBBB-X, check separate). It has the same combination-code third block as the other sub-handshake classes but a bounded lifecycle: it resolves only while its session and expiry are valid, is never durable, and is never reissued (§XVI.5.7).

The same physical object may have different SP, SI, or SC bindings for different governing contexts. The compact reference identifies the binding; the authoritative resolver data identifies the actual object and parent relationship.

Third block of sub-handshake references. For SP, SI, SC, and SE the third block does not identify a free-standing entry. It is a six-symbol combination code issued by the publisher that jointly encodes (a) the WR Entry — the specific object, resource, action, or purpose the sub-handshake governs — and (b) the identifier of the receiving party (responder organization or individual principal). Both are combined into one six-symbol block so that a sub-handshake is addressed with exactly the same four-block form as every other reference; no additional block and no second identifier are transported. How the publisher derives the block (sequential issue, keyed hash of parent-id and object-id, or a lookup table) is publisher-internal; the resolver alone expands the block into its parent binding and object, and only releases that expansion within the applicable grant. Two sub-handshakes for the same entry toward different receiving parties, or for different entries toward the same party, therefore have different combination codes, and the code alone reveals neither the entry nor the receiving party to anyone without resolver entitlement.

Device targeting. A principal may hold several paired devices — for example a workstation, a laptop, smart glasses, a sandbox PC, and a wearable — under one SSO Identity. The reference layer addresses a specific device not by adding a third constituent to the combination code but by choosing the granularity of the receiving-party constituent: the issuer binds the entry either to the principal (any paired device may resolve it) or to the Device-Scoped Principal Identifier of one device (only that device may resolve it). Examples: cloning a BEAP™ message to the principal's sandbox PC is an SE or SI sub-handshake whose combination code binds the message to the sandbox PC's Device-Scoped Principal Identifier; a workstation issuing an internal code that only a particular other workstation of the same principal may use binds the SI entry to that workstation's identifier. On any other device of the same principal the reference resolves to NOT_FOR_THIS_DEVICE (a refinement of NOT_FOR_YOU, §XVI.4.2). Policy may constrain targeting by Device Class — for example, permitting message clones only to devices of class sandbox PC, or confining smart glasses to SE contexts. Because device targeting rides on device-scoped keys, revocation of a device's key invalidates all references bound to it while leaving the principal-level identifier and other devices untouched.

Known-identifier rule for the receiving-party constituent. An issuer can bind only what its own repository knows at the time the code is generated. The receiving-party constituent of a combination code may therefore reference: (a) any Publisher Identifier — always, through the Namespace Directory; (b) a Principal Identifier / SSO Identity — only where a handshake between the issuer and that principal already exists (a P handshake with the publisher; membership of the tenant); the counterpart organization in a C relationship is addressed only as an organization; (c) a Device Record (and thereby a Device-Scoped Principal Identifier) — for SI where the device has been paired by an I pairing handshake *under this tenant*, since the pairing is what creates the device key, the name, and the record the issuer selects from; and for SC only where the counterpart device has registered itself with a Device Pass (§XVI.13.7), the initiator holding the tenant-signed record the device presented and proved.

Per class this means: **I / SI** — the tenant is the issuer, its principals and their paired devices are known, so device-level binding is available (a workstation issuing a code that only another workstation of the same principal may use is exactly this case); **P / SP** — a publisher addresses a user at principal level (any device of the user) or not at all (public offering); which device receives an SP is the user's own choice at acceptance; **C / SC** — the receiving party of an SC code is the responder organization; the initiator never selects a principal or device of the counterpart on its own. Which principal and which device inside the responder organization handle an SC is the responder's own role management, defined by the responder's policy and executed by its Orchestrator (routing by role, department, or named principal is internal to the responder and invisible to the initiator). The single exception is a Registered Counterpart Device: a device that has registered itself by Device Pass may be bound as the receiving party, and the resulting SC resolves on that device alone; **I pairing** — neither the Device Record nor the fingerprint of a device that is not yet paired exists, so a pairing code binds a Pairing Slot + the principal and nothing about the device; the device presents its freshly generated key, Device Class, and Device Name during pairing, the issuer registers them, and only from then on is that device selectable and addressable. **Device granularity (SI, and SC by Device Pass)** — the issuer binds a Device Record it already holds; a device-directed code for a device not in its list cannot be generated. An I code for a principal whose intended device is not yet paired is therefore principal-bound; the device is paired first and becomes addressable thereafter.

§XVI.5.11 Generalized reference structure

The reference-profile forms of §XVI.5.1 instantiate a general structure. A textual reference of the disclosed kind consists of an ordered sequence of typed fields: (1) a **classification field** that is parsed first and determines how the remaining fields are interpreted and which resolution branch applies; (2) one or more **address fields**, at least one of which is a namespace identifier that deterministically selects the responsible resolver; and (3) an **error-detection field** computed over the classification and address fields and validated before any remote lookup. The following generalizations are part of the disclosure and are not limited by the reference profile:

- **Number and role of address fields.** The reference profile uses exactly two address fields. A profile may use one address field (namespace only, addressing a namespace-level default entry), two (namespace plus local/binding block, or namespace plus counterpart namespace), or more than two (for example, namespace, counterpart namespace, and a local/binding block; or an ordered list of several participant namespaces). Each role assignment is declared per class in the grammar registry; the parser reads the class first and then frames the address fields according to the class's declared field list.
- **Meaning of an address field.** Any address field may be a namespace identifier, a publisher-local entry identifier, a collapsed binding identifier that the resolver expands into two or more constituents (§XVI.5.10), a counterpart identifier, a request or session discriminator, or a registered pointer/token, as the class declares. The reference profile's choice that class C's second field is the responder's namespace and that sub-handshake classes combine entry and receiving-party identifier into one six-symbol field are class declarations, not structural constraints.
- **Field lengths.** Each field has a length that is fixed by the class or delimited by explicit end-of-candidate framing, with the error-detection field always last; the reference profile uses six-symbol namespace identifiers and a five-symbol default local field. Any length or mix of length rules that leaves the check position determinable is within the disclosure.
- **Placement and number of error-detection fields.** The reference profile attaches the check to the last address field except where the last address field is itself a namespace identifier that benefits from being displayed unmodified (class C), in which case the check is rendered separately. A profile may always render the check separately, always attach it, place per-field checks on individual address fields, or add a second reference-wide check; the functional requirement is that at least one error-detection value covers the classification field and every address field and is validated locally before resolution.
- **Classification field.** The reference profile uses one or two leading alphabetic symbols with a terminal/non-terminal grammar. A profile may use a different number of classification symbols, a different symbol set, a positional or numeric class code, or a machine-only classification field for non-human carriers; and a grammar version may add, deprecate, or subdivide classes without altering already-issued references of existing classes.
- **Alphabet and normalization.** Crockford Base32 with its aliases is a choice of human-transcribable alphabet. Any alphabet with a deterministic normalization from human input to a canonical symbol sequence, and a compatible class-value mapping for the error-detection algorithm, may be substituted.
- **Error-detection algorithm.** Damm over an order-32 quasigroup is a choice that detects all single substitutions and adjacent transpositions for this alphabet. Any algorithm whose input is the ordered class and address values and whose output is one or more symbols of the reference alphabet (or a separately rendered symbol set) may be substituted, with corresponding statements of detected error classes.

- **Relationship references.** Naming a relationship by an ordered pair of party identifiers is one embodiment of a relationship address. Alternatives within the disclosure include an unordered pair with a canonical ordering rule, a pair plus a relationship discriminator to allow several parallel relationships between the same parties, a pair-derived collapsed identifier issued by the initiator's resolver, or a participant list for multi-party relationships; in each case the resolver, not the string, determines the relationship's participants, state, and grant.
- **Use limitation.** An entry, context, or automation may be declared single-use or n-use as authoritative resolver/grant state (§XVI.8.4); the reference itself does not change. Variants within the disclosure include per-party limits, time-boxed limits, and limits per carrier instance where a carrier binding registry distinguishes instances.
- **Lifecycle binding.** That the invitation and the established relationship share one identifier (P11) is a property of the reference profile's lifecycle model. A profile may instead issue distinct pre- and post-establishment identifiers linked by the resolver; the disclosed invariant is that identifier reuse for an unrelated referent never occurs and that terminal states remain resolvable.

What is functionally required across all of these variations is the ordered separation of classification, address, and error detection; classification before address interpretation; local error detection before network effect; deterministic namespace resolution from an address field; and the rule that no field of the reference carries authority (P2).

§XVI.5.12 Design rationale and encompassed simplified variants

This section records why the reference profile makes the choices it makes, and states expressly that simplified variants of the structure remain within the generalized disclosure of §XVI.5.11. The variants below are alternative embodiments encompassed by that generalized structure; each trades away a named property of the reference profile. Nothing in this section limits the scope of §XVI.5.11.

Why the namespace field is a fixed-alphabet identifier and not a domain name. A profile may carry a registrable domain name in the namespace field; the generalized structure encompasses it. The reference profile deliberately does not, for three technical reasons. First, error behaviour: a mistyped or mis-recognized fixed-alphabet identifier fails the check symbol locally, before any network access, whereas a mistyped domain is a different valid domain that a third party can register and back with genuine DNS records — the transcription-error class would become an impersonation class. Second, lifetime: domains expire, transfer, and rebrand; binding references to a domain would either invalidate every issued carrier on a change of domain or, worse, transfer outstanding references to the domain's next registrant. The stable identifier decouples reference lifetime from domain lifetime: the Namespace Directory Record binds the *current* DNS-verified origins to the identifier, and a domain change is a record update that leaves issued references valid. Third, trust structure: with a domain as the identifier, the domain-name system would become the sole root of trust for addressing; with a stable identifier, DNS remains one verification channel among several (Directory signature, web manifest, email alignment, SSO identity). Human-readable accountability is provided independently by the Displayed Responsible Domain (§XVI.9.4), which is display text cross-checked against the verified record, not identifier material.

Encompassed simplified variants and the properties they trade away.

(a) *Omitting the classification field.* A profile may omit the class prefix and leave classification to the resolver. Such a profile forgoes local framing and completeness detection before network access, class-dependent preview policy before the first network access, and the binding of the class into the check input, under which a class-substitution error is detected locally.

(b) *Omitting the error-detection field.* A profile may omit the check symbol and rely on resolver-side validation. Such a profile forgoes local rejection of transcription and recognition errors; every mistyped candidate becomes a network query, with the accompanying privacy exposure of failed candidates and loss of immediate capture feedback.

(c) *A flat opaque token without field structure.* A profile may use a single undifferentiated token resolved against one central table. Such a profile forgoes namespace delegation: the two-stage resolution — namespace identifier to publisher, publisher namespace to entry — distributes authority and caching to publisher namespaces under dual-signed Directory Records, whereas a flat token concentrates every resolution and every relationship at a single resolver.

(d) *Machine-readable-only presentation*. A profile may present references exclusively as machine-readable payloads without a visible textual form. Such a profile forgoes transport agnosticism (voice, dictation, manual entry, print without a scanner) and the text-first verification order of §XVI.9.4, under which the decoded payload must agree with the independently verified visible text.

(e) *Larger alphabets with shorter bodies*. A profile may use a larger, case-sensitive alphabet to shorten the body. Such a profile forgoes the capture properties of the reference alphabet: case-insensitivity, alias normalization of visually confusable symbols, dictation robustness, and OCR robustness.

(f) *A namespace-only short form for a default entry*. A profile may resolve a prefix-plus-namespace form (without a local block) to a publisher-declared default entry. The reference profile does not, because a short form whose body is a proper prefix of the long form makes completeness locally undecidable — a truncated long-form capture could present as a valid short form — and because two visible presentations of the same class weaken the single learned pattern on which user recognition rests. The same convenience is available without a second form: a publisher designates an ordinary entry as its default, and alias, menu, and auto-insertion mechanisms supply the reference.

(g) *Reusing an existing identifier system*. A profile may carry an identifier of an existing system (a decentralized identifier, a name-service handle, a telephone-style number) in the namespace field. Such profiles inherit the entry properties of the chosen system — length and enterability, dependence on that system's root of trust, and that system's reassignment and squatting characteristics — in place of the properties engineered here.

What §XVI.5.11 requires functionally — ordered separation of classification, address, and error detection; classification before address interpretation; local error detection before network access in profiles that include the error-detection field — is preserved or consciously traded in each variant above, and every such variant is within the present disclosure.

§XVI.6 Namespace Resolution, Repository Resolution, and Carrier-Token Mapping

§XVI.6.1 Direct-identifier and pointer modes

An admitted carrier profile declares whether it can transport the complete normalized textual reference, a registered pointer/token, or both. Direct mode exposes the complete identifier. Pointer mode exposes a compact token, tag identifier, or index that the resolver maps to the same authoritative WR Entry. Transport agnosticism therefore preserves entry semantics across carriers without pretending that all carriers have the same payload budget.

§XVI.6.2 Namespace determination and supplementary fields

A direct textual reference exposes its Publisher Identifier(s) and allows deterministic routing through the Namespace Directory. A pointer carrier may omit the textual namespace and instead rely on a globally or scope-uniquely registered token-to-entry binding maintained by the resolver that issued the token; the token's issuing namespace is recorded in the binding. Both modes converge on the same authenticated resolution and state-validation layer.

Where capacity permits, a carrier may also include manifest identifiers or hashes, profile/version data, signatures, or other declared supplementary fields. Supplementary fields may accelerate validation but do not replace the authoritative entry record, relationship state, object binding, access policy, or consent state.

§XVI.6.3 Pre-establishment boundary for invitation entries

A PENDING C-class entry is a WR Entry like any other for the purposes of namespace resolution, check validation, and lifecycle lookup, but its pre-consent material (capsule, initiator nonce ciphertext, offered scope) is released only recipient-bound. To any principal that does not chain to the responder named in the second block, the resolver returns at most a status such as NOT_FOR_YOU (§XVI.4.2), without confirming whether the entry exists or is pending. Knowledge of the reference therefore does not prove that an invitation to the holder exists. A pending invitation becomes authentic for the responder only after recipient-bound release and independent verification of the signed capsule. On acceptance the same identifier transitions to ESTABLISHED and subordinate contexts may be addressed through SC.

§XVI.6.4 Namespace Directory

Deterministic namespace resolution is provided by a Namespace Directory that maps each Publisher Identifier to a signed Namespace Directory Record containing at least: the publisher's current verification key(s) and key generation; one or more resolver endpoints; one or more Relay endpoints for capsule and acceptance delivery; the

grammar version; the registered WR Connect script version and, where the publisher customizes the connector, the hash of its connector code (§XVI.7.4a); the bound origin(s) or domain(s), where the publisher operates websites; namespace status (ACTIVE, INACTIVE, REVOKED, SUPERSEDED, COMPROMISED); and record generation and expiry.

Directory records are signed by the directory operator's key and countersigned by the publisher. The directory operator's public key is the trust anchor of the reference layer: it is distributed with the runtime (pinned), and key rollover is performed by a rollover record signed by both the outgoing and the incoming operator key, which clients verify before accepting records signed by the new key. Initial registration of a Publisher Identifier and its first publisher key is an out-of-band, operator-vetted act (organizational identity verification per operator policy); the countersignature requirement applies to every subsequent record update, so that neither the operator alone nor the publisher alone can alter an active record. Records are cacheable so that repeated references to the same namespace do not require a full directory query each time, but a cache entry is never authoritative on its own: every record carries a generation number, and any change to the record — key rotation, resolver or Relay endpoint change, status change, domain registration change, — increments the generation. On each resolution the client performs a lightweight generation check (or receives a signed change notification where the profile supports push), and a cached record whose generation no longer matches the current one is invalidated immediately, regardless of its remaining nominal expiry. Expiry is therefore an upper bound on cache life, not a promise of validity; a changed record is never served from cache. In one embodiment the directory is a queryable service; in another it is published through DNS beneath a dedicated zone (for example, a record set at `<publisher-id>.<directory-zone>`), so that any DNS-capable client can resolve a Publisher Identifier to its endpoints and key material; in another the directory is federated, with the root directory delegating Publisher Identifier ranges to regional or sector directories under the same record format. The directory answers "which resolver and which key belong to this namespace"; it does not answer "what does this entry mean" and does not hold entry content.

Directory lookups are namespace-keyed and do not require a user identity. Directory endpoints apply anti-enumeration controls (rate limiting, no wildcard listing) so that the directory cannot be used to enumerate the namespace population.

§XVI.6.5 Publisher key binding for manifests and origins

Every publisher-signed artifact consumed by the reference layer — Signed Site Manifests, Pending Handshake-Request Capsules, acceptance records, Carrier Binding Registry records, and entry status records — is verified against a publisher verification key obtained from the Namespace Directory Record, or against a Principal Key whose Delegation Certificate chains to that directory-registered key, and never against key material carried by the artifact or the visited page alone. Verifiers check the Delegation Certificate's scope, generation, and expiry and treat a revoked or out-of-scope delegation as an invalid signature.

Account holder and domain control. A Publisher Identifier is treated as verified only when two independent facts hold: the directory record identifies the publisher as a vetted account holder of that namespace (§XVI.6.4), and a valid DNS entry under the publisher's own domain — a profile-defined record naming the Publisher Identifier — proves that the party holding the namespace also controls the domain. A namespace whose DNS proof is missing, expired, or names a different Publisher Identifier is not verified, regardless of the directory record; a DNS record without a matching directory record is likewise not verified. Both are re-checked within their caching limits on every resolution.

Email-domain agreement. A principal may act for a publisher — initiate, accept, revoke, issue Delegation Certificates, or sign on the publisher's behalf — only if the domain part of the principal's SSO-verified email address is the publisher's DNS-verified domain or a domain that the publisher's directory record explicitly registers as its own (each such domain carrying its own DNS proof). The Delegation Certificate records the principal's verified email, and every verifier checks that this email's domain is among the publisher's DNS-verified domains at the time of the signature. An email outside those domains — a personal address, a mail provider's domain, a look-alike domain, or a domain whose DNS proof has lapsed — does not chain to the publisher, even if the account itself is otherwise authenticated. This closes the gap between "someone with a verified mailbox" and "someone who is this organization": the person is proven by the email, the organization by DNS, and the two are bound by requiring the email to live inside the DNS-proven domain. For a C-class reference the account-holder and domain-control verification is performed for each of the two Publisher Identifiers, by whichever party is resolving: the initiator verifies the responder's namespace before issuing the invitation, the responder verifies the initiator's namespace on receipt and again before acceptance, and every later party resolving the established relationship verifies both. A C reference with one unverified side is not verified.

Entry context, assignment, and version. Namespace verification says who the publisher is; entry verification says that the referenced thing exists as the publisher registered it. The resolver verifies that the entry context designated by the local block exists in the publisher's repository, that it is assigned (bound) exactly as the

reference implies — same parent context, same object, same relationship, same offering — and that the repository version the reference designates is current or explicitly superseded. A mismatch in existence, assignment, or version is a verification failure, not a soft warning. For sub-handshake references the combination code (§XVI.5.10) is expanded by the resolver and both constituents are verified: the entry constituent must resolve to an existing entry binding under a currently governing parent (for SC: an ESTABLISHED C relationship whose two namespaces have themselves passed account-holder and DNS verification), and the receiving-party constituent must resolve to the verified party the code is addressed to — at principal level, or, where the issuer bound it to one device, to the Device-Scoped Principal Identifier of that device. The verifications do not disappear because two identifiers were collapsed into one block; they are performed on the expanded form.

Individual (B2C) parties. A party that is an individual user rather than a publisher — the customer side of a P-class handshake, the user side of a device-directed SP sub-handshake, and similar — has no domain and therefore no DNS proof. Such a party is verified by its SSO Identity: authentication at the required assurance level of the one account whose verified email address is that party's identity, recorded in the party's Party Binding. The verified email address and the SSO Identity are the same thing; there is no separate login identity that could differ from the email bound into the handshake, and no handshake can be formed, resumed, or paired under an email other than the one the account was verified with. The publisher side of the same handshake is still verified by account holder plus DNS; the asymmetry is in the proof available, not in whether verification occurs.

Email binding of both parties. In every handshake, of every class, both parties' SSO Identities — their one verified email address each — are part of their Party Bindings and are therefore bound into the capsule, the acceptance record, the Handshake State Record, and — through the capsule and acceptance hashes — the Establishment Commitment. The email address is an identity fact and a notification channel; it is never itself a carrier of authority and never a link (P15).

An origin (website) is bound to a namespace when both directions agree: the Namespace Directory Record lists the origin, and the origin publishes a discovery hint (§XVI.7.2) naming the Publisher Identifier. A manifest signed by the directory-registered key and served from, or referenced by, an origin listed in the directory record is a validated manifest; either direction alone is insufficient.

§XVI.7 End-to-End Capture, Discovery, and State Flows

§XVI.7.1 Common capture record and local validation

All textual capture paths converge on a typed local candidate-processing boundary before remote resolver or Relay use. A reference Capture Record may contain `source_type`, `raw_candidate`, `normalized_candidate`, `reference_class`, `prefix_state`, `terminal_prefix`, `publisher_or_participants`, `local_or_selector`, `check_character`, `check_result`, source origin or message identifier, timestamp, and optional confidence/provenance/carrier-token fields. Field names are illustrative; the technical function is a typed separation between capture evidence and authoritative state.

A failed grammar transition or check is rejected locally. A pointer carrier bypasses textual parsing only because it supplies a different registered token type; it does not receive authority by skipping the textual representation.

§XVI.7.2 Website discovery and page-to-entry binding

1. On top-level navigation or equivalent page commit, the browser integration derives the normalized origin and current page URL.
2. The client checks for a still-valid authenticated WR Site Discovery Record already bound to that origin.
3. If no fresh record exists, a discovery hint may be obtained from a profile-defined DNS record, same-origin well-known resource, signed HTTP/Link hint, HTML link/meta hint, or registry lookup keyed by the verified origin. The hint names a Publisher Identifier and a manifest location. A hint is not trusted until the publisher/domain binding is validated.
4. The runtime obtains the Namespace Directory Record for the hinted Publisher Identifier, confirms that the record lists the visited origin (§XVI.6.5), and verifies the Signed Site Manifest against the directory-registered publisher key, including its signature/hash/generation and publisher declaration.
5. The current URL is matched against the manifest's page-to-entry map. For automatic surfacing the connector code served on the page is first hashed and compared with the canonical or publisher-registered connector hash (§XVI.7.4a); only then is the signed WR script block it emitted verified, and it must agree with the mapped entry. A page carries at most one umbrella offering. A site-wide umbrella may cover declared pages; otherwise only an explicitly mapped entry is offered. WR Link declarations on

the page are not consulted at this stage; they become relevant only once the offered entry has been verified and a handshake under it is active (§XVI.2, WR Link).

6. The resulting entry candidate undergoes only the minimum pre-consent resolution required to identify the offering party and determine the appropriate handshake state.

Failure of domain binding, manifest validation, or page matching prevents the runtime from presenting the page as a verified WR handshake source. Page JavaScript or visual branding alone is not a trust source.

§XVI.7.2a Catalog pages: multiple embedded sub-handshake blocks beneath one umbrella

The page-to-entry rule of §XVI.7.2 is unchanged: a page carries at most one umbrella offering. Beneath that verified umbrella, a page MAY carry any number of signed WR script blocks (§XVI.7.4a) whose payload reference is of a sub-handshake class — SP, SI, SC, or SE — for example one block per item of a course or product catalog. Such blocks are emitted by the publisher's connector from Sub-Handshake Embedding Blocks issued at publication (Annex XVIII §XVIII.3.11.4); their number and placement are page content, not authority.

A sub-handshake block on a page is credited for automatic surfacing only when all of the following hold:

1. the connector code served on the page has passed the registered-hash comparison of §XVI.7.4a — performed once per page, before any data block on that page is read;
2. the block's signature verifies against the directory-registered publisher key, and its Publisher Identifier matches the page's verified origin and the umbrella entry mapped for the page (§XVI.7.2 steps 4–5);
3. the referenced sub-handshake is declared in the currently valid generation of the Signed Site Manifest or catalog commitment governing that umbrella entry. A block whose reference is absent from the current generation — including any block of a superseded generation — is not credited; generation supersession thereby retires embedded blocks without touching the umbrella.

A block failing any condition is not auto-surfaced and is not credited even if its signature verifies in isolation; the reference it carries remains capturable manually as ordinary text and is then verified like any other capture, without script-block provenance. The single-parent rule of §XVI.7.4a is unaffected: sub-handshake-class blocks in any number coexist with at most one credited parent-class reference per page.

Multiple credited sub-handshake candidates on one page do not each open an offer surface. They enter the candidate set of §XVI.7.7a for element-scoped designation; unsolicited bulk surfacing of an entire catalog is not a conforming behavior.

§XVI.7.3 Existing-handshake detection and direct reuse

After a site, entry, or other candidate is verified, the runtime determines whether a usable handshake already exists before initiating a new-connect flow. The lookup key is derived from the authenticated local principal, verified publisher/namespace, handshake profile/class, and, where applicable, relationship or entry context. The local authenticated Handshake State Index may be consulted first; stale or missing state may be refreshed from a verified publisher/relationship authority.

Typical semantic states include NONE, PENDING, ESTABLISHED/ACTIVE, SUSPENDED, EXPIRED, REVOKED, SUPERSEDED, and REAUTH_REQUIRED. Equivalent profile-specific state names are permitted where their effects are unambiguous. A cached record is reusable only while its generation, expiry, revocation/logout epoch, grant scope, and other freshness conditions remain valid.

If an established handshake is valid and the requested operation lies within its existing grant, the runtime may reuse the handshake directly, and may enter a sub-handshake beneath it without a new consent step where the umbrella grant already covers that sub-handshake's automation and context (§XVI.13.2). A new session may still need to be started or rebound. New capabilities, materially expanded scope, a new finalizer/effect, or higher assurance requires the applicable fresh consent or authorization. The fact that a code or page was seen before is not evidence that the handshake remains active; current authenticated state is.

§XVI.7.4 Email as a carrier: extraction and handoff

Email is one admitted carrier among others. It may convey a WR Code® reference of any class — a Publisher offering, a device or internal entry, a sub-handshake, or a cross-organizational invitation — and the runtime treats the extracted reference exactly as it would treat the same reference captured from print, a scan, or a page. What email adds is optional channel provenance; what it never adds is authority.

The Orchestrator is not a conventional mail client. The architecture has two Orchestrator instances: the **host Orchestrator**, which owns the trusted surface, the Handshake State Index, and consent, and the **sandbox Orchestrator**, in which original external content is opened. The host Orchestrator does not render, open, or execute original emails, attachments, or embedded links in its trusted surface. Every incoming message is transformed into the BEAP™ format on receipt; the host Orchestrator's mail surface shows the BEAP representation. Opening an original attachment or following a link other than a verified WR Link (§XVI.2) goes through the **BEAP clone mechanism**, which redirects the item to the sandbox Orchestrator.

Policy-dependent isolation. The recommendation of the architecture is that all external links and all attachments are opened out-of-band — in the sandbox Orchestrator, separated from the host Orchestrator's trusted state. How far a deployment follows that recommendation is a matter of policy, set per deployment tier and, for deviations, by explicit super-admin decision:

- *Recommended default (all tiers):* links and attachments open in the sandbox Orchestrator. In the standard/free tier the sandbox Orchestrator typically runs as a virtual machine on the host and opening is preceded by an explicit warning about the risks of external content; the warning is part of the trusted UI and cannot be suppressed by page or message content. In the high-assurance tier the sandbox Orchestrator runs in a hardware-isolated sandbox, and policy will normally prohibit opening on the host altogether.
- *Whitelisted links:* verified URLs may be whitelisted by policy. A whitelisted link need not be opened in a sandbox; it may open directly, and without the standard warning, if the policy says so. Because this departs from the recommendation to open everything out-of-band, whitelisting is a super-admin decision that must be configured explicitly — it is never on by default, is never derived from message or page content, and is recorded as a policy deviation. Whitelisting applies to links; it is not a mechanism for attachments.
- *Further restriction:* policy may always tighten beyond the default — forbidding specified attachment types, requiring additional confirmation or assurance before a clone is provisioned, or confining clone opening to designated sandbox profiles.

Attachments. Attachments are presented either inside the sandbox environment or as reconstructed, inert representations within BEAP — a rebuilt PDF or a text-only rendering produced by the BEAP transformation. Whether and under what warning an original attachment may be opened in the sandbox is governed by the same policy framework; the host Orchestrator's trusted surface itself does not open originals.

Attachments and third-party binaries. Attachments are not opened, rendered, parsed, or executed by the host Orchestrator in their original form. An attachment reaches the user in one of two ways: (a) it is transformed into the BEAP™ format on receipt — as a reconstructed PDF or text-only representation — and the user works with that representation; or (b) where a BEAP transformation is not sufficient or the user needs the original, the original is provisioned as a BEAP clone to the sandbox Orchestrator and opened there, at the isolation level the applicable policy prescribes, separated from the host Orchestrator's trusted state and from any handshake context. Third-party binaries — office documents with active content, archives, executables, installers, scripts, media containers with parsers of their own — are treated as potentially hostile regardless of sender or provenance; a valid email provenance result does not make an attachment safe. Because the WR runtime targets high-assurance environments, direct processing of attachments may be further restricted by policy: a publisher, tenant, or user policy may forbid sandbox opening of specified types altogether, may require a BEAP transformation with no clone path, may require an additional confirmation or assurance step before a clone is provisioned, or may restrict clone opening to designated sandbox profiles. By default every open of an original is a redirect to the sandbox Orchestrator; deviations are explicit super-admin policy decisions, and the host Orchestrator's trusted surface itself offers no path that opens an original. WR Code® references are recognized during the BEAP de-packaging step itself — as text, from the body, structured fields, or the signed WR script block — which is the only point at which message content and the WR capture pipeline meet.

1. The mail integration obtains message content, headers, and available provider/MTA authentication results.
2. SPF, DKIM, DMARC, sender-domain/publisher alignment, DNS/WR binding, and other applicable provenance checks determine whether authenticated-email provenance may be credited. Sender-domain alignment is evaluated against the origins listed in the Namespace Directory Record of the Publisher Identifier contained in the extracted reference.
3. The message and every attachment are transformed into the BEAP™ format on receipt (see above). During de-packaging, the transformation produces a typed representation of the message from which the textual WR Code® reference is recognized and extracted as text. Only text is extracted — from the visible body, from structured mail data, or, in the normal case, from the signed WR script block by which the

publisher placed the code in the message (§XVI.7.4a). Original graphics in the message or its attachments are not used for automatic detection; for auto-detection and auto-insertion from email the reference must come from the signed WR script block (§XVI.7.4a). A graphic may still be captured by the user via the sandbox view or a screenshot (P12); such a manual capture goes through the same verification and, if it passes, is an equally valid offering. Case is irrelevant; structured or machine-readable fields are convenience only. 3a. One umbrella offering per carrier instance: a single email (and likewise a single website origin or a single physical carrier) presents at most one umbrella/parent handshake offering. If the transformation yields more than one parent-class reference, the message is treated as malformed and no offering is presented; sub-handshake references beneath the one offered umbrella are permitted. 3b. The trusted runtime rebuilds the visual WR Code® locally from the extracted and verified textual reference — the standardized rendering with Outer Shape and, where the profile includes it, a QR/Data Matrix element — and shows this locally generated visual to the user in place of any artwork the message carried (P12). The user therefore always sees a runtime-rendered code. From this local rendering the reference can be captured again by OCR or scan — for example by a second device (§XVI.10) — which is why OCR remains a valid capture path for email even though message graphics themselves are never OCR-scanned.

4. The integration emits a typed Email Capture Record to the local runtime containing message context, provenance outcome, the BEAP-derived raw and normalized candidates, and the signed-script-block verification result.
5. The runtime independently applies the same grammar and check validation as for keyboard, OCR, and speech input, selects the resolver branch from the class prefix, resolves the entry, and evaluates current Handshake State. A PENDING C-class entry resolves recipient-bound to a verified invitation (§XVI.7.5).
6. An existing valid handshake is reused within its grant; otherwise a verified invitation or terminal/re-authentication state is presented.

If email provenance fails, the message receives no authenticated-channel credit. A still-visible checked reference may nevertheless be manually entered or OCR-captured and processed through the ordinary path as an unauthenticated-source capture.

§XVI.7.4a Signed WR script block: how a publisher places a code in an email or on a website

In the normal case a publisher does not paste a WR Code® into a message or page as loose text or as an image. The publisher's WR tooling (the Generator, §XVI.10, or the publisher connector script on a website) emits a **signed WR script block**: a structured, publisher-signed element that carries the textual reference, the reference class, the entry generation, and the publisher's signature over these fields, verifiable against the directory-registered key (§XVI.6.5). In email the block is a structured part of the message; on a website it is emitted by the publisher's WR Connect script into the page.

Connector integrity (registered script hash). The connector has two parts that are protected differently. The dynamic data block — reference, class, entry generation — changes per entry and is protected by the publisher's signature. The static connector code changes only per version and is protected by a **registered hash**: the platform publishes a signed list of canonical WR Connect versions with their hashes, and a publisher's Namespace Directory Record declares which version it runs; a publisher that customizes the connector registers the hash of its variant in its own directory record, which — like every record update — is dual-signed by publisher and directory operator, so that an attacker holding only the publisher's server key cannot change the registered hash alone. Before the browser integration reads a data block from a page, it hashes the connector code as served and compares it with the canonical or publisher-registered hash; the connector is never executed by the runtime, only compared. A mismatch means the connector was altered on the server (XSS, compromised CMS, plugin or supply-chain modification, downgrade): the page is not auto-surfaced, the data block is not credited even if its signature verifies, and the runtime may report the mismatch to the publisher without user data. This is related to subresource integrity but differs in that the reference value lives in the dual-signed directory, not on the possibly compromised page, and is versioned with its own generation. A receiving integration extracts the reference from this block, verifies the block signature, and records the result in the Capture Record. A block whose signature fails, whose Publisher Identifier does not match the sending domain or origin (§XVI.7.2, §XVI.7.4 step 2), or which carries more than one parent-class reference is not auto-surfaced. Loose textual references without a script block remain capturable manually as ordinary text and are verified like any other capture; they simply are not auto-detected or auto-inserted from these two carriers. For automatic detection and insertion on these two carriers the visual form in the message or page is not used as the source of the reference; the visual shown to the user is rebuilt locally from the verified text (P12). Manual capture of a graphic remains possible and yields a reference that is verified like any other; it simply arrives without script-block provenance and is never auto-surfaced.

§XVI.7.5 Cross-organizational invitation, recipient-bound resolution, and establishment

1. The initiator principal authenticates locally; where it acts for a publisher, its Principal Key carries a Delegation Certificate with initiate scope. The initiator resolves the intended responder's Namespace Directory Record and verifies the responder's account-holder status and DNS domain control (§XVI.6.5); it does not issue an invitation to an unverified namespace.
2. The initiator creates (or, after a terminated relationship, re-opens) the C-class entry for the ordered pair initiator/responder in its own namespace with state PENDING, generates a high-entropy initiator nonce, and attaches a signed Pending Handshake-Request Capsule containing request_instance_id, Party Bindings of both parties (ordered Party Identifiers with verified email addresses), requested profile/scope, nonce_I encrypted to the responder publisher's directory-registered encryption key, H(nonce_I) in the clear, freshness, expiry data, and its Delegation Certificate. The capsule is deposited with the Relay named in the initiator's directory record for recipient-bound release.
3. The WR Code® Generator emits the C-class reference (for example C-ABC123-XYZ789-H) with its separate check block for any admitted carrier — print, display, QR/Data Matrix, NFC, website, or email. Where email is used, the message conveys the reference in visible text, structured mail data, or an attached payload-capable carrier and carries no link (P15).
4. The recipient integration validates channel provenance where applicable (§XVI.7.4), extracts the reference, validates its check locally, and hands a typed Capture Record to the runtime. The runtime obtains the initiator's directory record, verifies the initiator's account-holder status and DNS domain control, and — because the second block names the responder itself — confirms that its own namespace record is current and that the reference names it; only then does it resolve the entry.
5. Because the entry is PENDING, the responder's runtime requests the capsule from the Relay with a recipient-bound claim: the responder principal signs the claim with its Principal Key together with a Delegation Certificate (accept scope) chaining to the responder publisher's directory-registered key. The Relay validates the chain and releases the capsule only to a claim that chains to a Party Identifier named in the capsule, not to mere possession of the reference. The Relay cannot decrypt nonce_I.
6. The resolver returns the authorized entry state. ESTABLISHED surfaces the existing relationship rather than reopening the invitation; DECLINED, EXPIRED, WITHDRAWN, REVOKED, and SUPERSEDED are terminal for that entry, and the identifier is not reused (P11).
7. For a PENDING entry, the responder re-verifies the initiator's account-holder and DNS status at acceptance time, verifies the capsule signature and Delegation Certificate against the initiator's registered key, decrypts nonce_I with the responder publisher's decryption key (held by the responder Orchestrator's key service on behalf of delegated principals) and checks it against H(nonce_I), and verifies both Party Bindings (including that the responder-side email address is its own verified address), freshness/replay state, and request_instance_id before deliberate acceptance/countersignature at the required assurance level (§XVI.13.5).
8. On acceptance, the responder generates its own high-entropy nonce and produces the signed acceptance record over the capsule hash, H(nonce_R), and nonce_R encrypted to the initiator publisher's directory-registered encryption key, with its Delegation Certificate. The acceptance record is deposited with the Relay for recipient-bound delivery to the initiator, which the Relay performs without being able to decrypt nonce_R. The initiator verifies the signature and delegation chain, decrypts nonce_R and checks it against H(nonce_R); both parties compute the Establishment Commitment (§XVI.13.6), and the entry transitions to ESTABLISHED under the same C-class identifier.
9. Reprocessing the same message, reference, or claim for the same request_instance_id returns existing state and does not create a duplicate relationship.

The C-class reference remains the durable address of the relationship after establishment; the responder's runtime records it in its Handshake State Index under the lookup key of §XVI.7.3.

§XVI.7.6 Pre-delivery security boundary: what must pass before a capsule reaches the receiving Orchestrator

The receiving Orchestrator's trusted surface is the point at which a request becomes visible to a human and can be accepted. Nothing that has not passed every gate below is admitted to that surface; until then a request capsule is, from the receiver's point of view, either unreachable or opaque ciphertext. The gates are ordered, each is a hard stop, and a failure at any gate is reported as a capture, verification, or delivery failure — never as a request.

Gate 1 - Local syntax and submission (receiver device, offline-capable). The captured candidate is normalized, the class prefix is parsed to a terminal state, the body is framed by the class-fixed block lengths, the class-aware check validates, and the candidate is explicitly submitted (§XVI.5.8). No network access for the full candidate occurs before this gate passes; the only pre-submission lookup that may occur is the optional class-dependent namespace preview for P, SP, C, and SC.

Gate 2 - Namespace verification (Namespace Directory). For every Publisher Identifier in the reference — one for P, I, and sub-handshake classes, both for class C — the receiver obtains the signed directory record and confirms: valid dual signature (operator and publisher), status ACTIVE, verified account holder, and a valid DNS entry under the publisher's domain naming that Publisher Identifier. Superseded, revoked, unsigned, or DNS-mismatched namespaces stop here. The receiver also confirms that its own acting principal's SSO-verified email lies in its own publisher's DNS-verified domain; otherwise it cannot claim on the publisher's behalf.

Gate 3 - Entry verification (publisher resolver). The entry designated by the local block (or, for C, by the ordered pair) exists in the publisher's repository, matches the registered assignment and version exactly, and is in a state that admits a request (PENDING for an invitation; ACTIVE for an offering). For sub-handshake classes the combination code is expanded and both constituents verified. The resolver returns no pre-consent material at this gate.

Gate 4 - Self-match (receiver Orchestrator). The receiver confirms that the reference is addressed to it: for C, that the second block is its own current Publisher Identifier; for I and for every sub-handshake class, that the receiver's Party Identifier matches the receiving-party constituent of the combination code at the granularity the issuer bound; for other recipient-bound offerings, that the receiver's Party Identifier is admissible for the entry at the granularity the issuer bound — principal level (any paired device of the principal) or device level (this device's Device-Scoped Principal Identifier only). A reference not addressed to the receiver yields NOT_FOR_YOU, and a reference bound to another device of the same principal yields NOT_FOR_THIS_DEVICE; in either case no further step follows.

Gate 5 - Recipient-bound release (Relay). The receiver presents a claim signed with its Principal Key and, where it acts for a publisher, its Delegation Certificate with accept scope. The Relay releases the capsule only if: the claim signature verifies; the delegation chains to a directory-registered key of the Publisher Identifier named as recipient in the capsule (or, for an individual, the Principal Identifier matches); the capsule is not expired, withdrawn, or already terminal; and rate and replay limits are satisfied. The Relay releases ciphertext plus signed envelope; it can decrypt nothing.

Gate 6 - Capsule admission (receiver Orchestrator, before any UI). Before the capsule is admitted to trusted state, the Orchestrator verifies: the capsule signature against the initiator's directory-registered key or a valid Delegation Certificate chaining to it; the initiator's account-holder and DNS status again at admission time, and that the initiating principal's SSO-verified email recorded in the Delegation Certificate lies in the initiator's DNS-verified domain; that both Party Bindings are well-formed and that the receiver-side binding names the receiver's own Party Identifier and its own verified email address; that request_instance_id is new (or maps to existing state, in which case existing state is surfaced instead); freshness and expiry; that nonce_I decrypts under the receiver's key and hashes to the H(nonce_I) carried in the clear; that the requested profile/scope is admissible under receiver and tenant policy; and that no field of the capsule references a link, an external resource, or an unregistered carrier (P15).

Only after Gate 6 does the request exist for the user: it is recorded in the Handshake State Index as PENDING-RECEIVED, rendered by the trusted UI from validated state (P12), and offered for explicit acceptance (§XVI.13.5). Every gate is evaluated by the receiver or by an infrastructure component acting on the receiver's authenticated claim; none is delegated to the initiator, to the carrier, or to the content of the capsule itself.

Gate	Where	Stops
1 Local syntax	Receiver device	Typos, OCR/speech errors, malformed or unknown classes
2 Namespace verification	Directory	Spoofed, revoked, or domain-unproven publishers; unverified counterparties in C
3 Entry verification	Publisher resolver	Non-existent, reassigned, stale-version, or non-pending entries
4 Self-match	Receiver	References not addressed to this receiver
5 Recipient-bound release	Relay	Claimants without the right key/delegation; expired or withdrawn capsules; replay
6 Capsule admission	Receiver	Forged or mis-bound capsules, wrong email/party binding, stale or

Gate	Where	Stops
	Orchestrator	replayed requests, policy-inadmissible scope, embedded links

§XVI.7.7 Automatic detection, optional insertion, and WR Code® activation

Automatic candidate detection remains enabled in the reference Orchestrator profile. Detection may occur on any admitted capture surface: visible or selectable text, still images and video frames (OCR), live or recorded audio (speech transcription of microphone input, calls, voice assistants, video sound tracks, broadcasts), email and messaging content, documents and attachments, clipboard content, QR/Data Matrix, NFC, BLE/radio, fiducials, and any surface admitted later. Detection identifies a candidate and may wake the local WR processing surface, but it does not accept a handshake or execute an action.

Auto-insertion is a strict process. Automatic insertion of a detected candidate into the entry field is permitted only through capture methods that are safe by construction — methods in which the runtime obtains the textual reference without executing, rendering, or parsing untrusted content on the host. For websites, email, and BEAP™-packaged content the safe method is standardized: the reference is taken from the verified signed WR script block (§XVI.7.4a) or from the BEAP de-packaging step (§XVI.7.4), and nothing else on that surface qualifies for auto-insertion. References present on a page or in a message without such a block are left to manual capture, which is always available and yields an equally valid offering after verification.

Capture is channel-specific, always secure. Every transport channel has its own capture profile (§XVI.9.1) that states how the reference is obtained, which component performs the capture, and where that component runs. Channels are treated separately — a QR decoder, an audio transcriber, an NFC reader, a page-connector verifier, and a BEAP de-packager are different code paths with different attack surfaces — but every profile satisfies the same invariant: capture never gives untrusted content a code path on the host. Where a capture method carries inherent parsing risk — decoding third-party media containers, OCR over untrusted documents, frame extraction from arbitrary video, reading unknown tag formats, opening any original attachment — the runtime performs that capture only in the sandbox, never on the host, and makes this transparent in the trusted UI: the candidate is shown as a sandbox capture with its channel and risk class, the user is told that the content was processed in isolation, and such captures are never auto-inserted. A host-side capture path exists only for methods whose input is already text or a verified structured block; all other capture is sandbox capture by definition.

Capture risk classes. Every capture method is assigned to one of the following classes; a carrier profile declares the class of each method it uses (§XVI.9.1), and the class fixes where the method may run and whether its result may be auto-inserted.

Class	Input	Examples	Runs	Auto-insert	UI
0 - Plain text	Text already available to the runtime; no foreign parser	Keyboard, dictation transcript, plaintext paste, text from BEAP de-packaging, OCR over the runtime's own rendering or its own screenshot	Host	Yes (after check)	Ordinary candidate
1 - Verified structured block	Signed, schema-bound structure verified before parsing	Signed WR script block (§XVI.7.4a), directory/manifest records, registered NDEF/beacon record types	Host	Yes — the standard auto-insertion path for websites, email, BEAP	Ordinary candidate with provenance credit
2 - Standard decoder over self-acquired input	Bytes acquired by the runtime's own sensor, decoded by a standard, hardened decoder in a least-privilege process	Camera QR/Data Matrix, AprilTag, OCR of the runtime's own camera frame, microphone transcription, BLE token, NFC of known format	Isolated decoder process; host or sandbox per policy	Per policy	Labeled as sensor capture
3 - Foreign content as input	A complex parser (image, container, PDF, document, rich clipboard, unknown	OCR of a received image file, frame extraction from a	Sandbox only	Never	Labeled as sandbox capture with channel and

Class	Input	Examples	Runs	Auto-insert	UI
	tag format) over attacker-controllable bytes the runtime did not acquire itself	received video, audio from a received container, text from an office document, rich-format clipboard, unknown NFC/tag formats			risk class; user informed that content was processed in isolation
4 - Active or executable content	Anything that must be opened rather than read	Direct opening of an attachment, office files with active content, archives, executables, installers, scripts	Not a capture path	—	—

Class 4 is never a capture path. A WR Code® contained in such content reaches the pipeline only through the BEAP transformation (its text becomes Class 0) or, if a user opens the BEAP clone in the sandbox and sees a code there, through Class 0/2 capture of the sandbox's own rendering. Direct opening itself is governed exclusively by the BEAP clone mechanism and its policy restrictions (§XVI.7.4). The preferred way to capture a code from any foreign document is therefore not to parse the document but to capture the sandbox rendering of it — the pixels are the runtime's own, and the foreign parser never touches the host.

Standard textual form and the designation as wake cue. The standard way a reference appears on any textual or visual carrier is the WR Code® designation followed by the reference itself — WR Code P-WR7X4K-9B2M3C — with or without a colon after the designation (WR Code: P-WR7X4K-9B2M3C); both spellings, in any case and with any whitespace between designation and reference, are equivalent and are recognized as one wake cue. The designation announces that a reference follows; it is not part of the identifier, contributes nothing to the check, and confers no authority. A reference without the designation is still a valid candidate (a class prefix followed by a well-formed body is itself a cue); the designation raises detection confidence and is the form every conforming rendering, template, and publisher script emits.

Manual capture. Automatic detection is a convenience, never the only path. A conforming runtime provides an explicit manual capture method by which the user, at any time and on any surface, triggers a capture — for example a screenshot of the current screen region or window, a photo, a clipboard read, a selection, or a recording — and hands the result to the same capture pipeline (§XVI.7.7, §XVI.9.4). Manual capture is subject to the same risk classes, text-first verification, gates, and consent as automatic detection; it differs only in who opened the capture window. Where automatic detection is disabled by policy or by the user, manual capture remains available.

Wake cues. A wake cue is a modality-specific signal that a WR Code® reference follows or is present, and it lets the runtime open a bounded capture window instead of continuously parsing everything. The reference profile recognizes: the spoken activation phrase in audio (§XVI.5.9); the written WR Code® designation, with or without colon, in text and in OCR output; the Outer Shape and the visible designation in images and video (§XVI.9.4); a class prefix followed by a well-formed body in text surfaces; a registered NDEF/beacon record type on NFC and BLE; and a registered attachment or structured-field type in email and messaging. A wake cue is not part of the identifier and carries no authority; it is a detection optimization, and a reference captured without any cue (for example, pasted) is processed identically. Additional cues may be registered per carrier profile.

Candidate detection and field insertion are separate state transitions. A user setting may disable automatic insertion into the visible entry field while leaving candidate detection active. The runtime may therefore display a detected-candidate indicator and wait for review without copying the candidate into the editable field.

DETECTED

→ optional INSERTED / READY_FOR_REVIEW

→ COMPLETE (framed by class grammar; check validated locally)

→ SUBMITTED (explicit user act, or auto-insertion event for verified script-block candidates)

→ CHECK_VALID

→ RESOLVED / STATE_KNOWN

→ CONFIRMATION_REQUIRED or ACTIVE_EXISTING_STATE

→ ESTABLISHED / AUTHORIZED_USE, or explicit ERROR / TERMINAL state

The exact badge wording, color, or icon is non-limiting. The technical requirement is that recognition, insertion, completeness, submission, syntactic validation, authenticated resolution/state, acceptance, session creation, and execution remain distinguishable states, and that no network access for the full candidate occurs before SUBMITTED (§XVI.5.8).

§XVI.7.7a Element-scoped designation of embedded sub-handshake blocks

Where a page carries multiple credited sub-handshake blocks (§XVI.7.2a), detection identifies the candidates without surfacing them all; the user designates exactly one. Designation triggers form an open catalog; the reference profile recognizes at least:

- a defined WR Pointer™ gesture on the item's element or region;
- cursor dwell on the element or region for a profile-defined interval;
- selection from the hover-invoked adaptive WR Menu presenting the element's credited candidate;
- a direct tap or click on the locally rebuilt visual of the reference (P12);
- a Handshake Alias spoken through the assistant wake phrase (§XVI.5.9), where an alias is assigned;
- on gaze-capable devices, gaze dwell under the designation rules of the spatial annexes.

Additional triggers may be registered per carrier or device profile. A trigger is designation, never acceptance: it selects which credited candidate enters the pipeline and nothing else. Designation of a credited block is a Class 1 capture (verified structured block, §XVI.7.7); the registered-hash comparison of §XVI.7.2a(1) has already preceded any payload evaluation, and the visual shown to the user is rebuilt locally from the verified text, never taken from page pixels.

The designated candidate then proceeds through the unchanged state machine of §XVI.7.7 — detection, optional insertion, submission, check validation, resolution — and through the consent rules of the handshake hierarchy: inherited entry within the umbrella grant where every applicable policy permits it (§XVI.13.2); explicit confirmation where any policy requires it; and explicit acceptance as an own establishment ceremony at the required assurance (§XVI.13.1, §XVI.13.5) wherever the sub-handshake adds anything outside the umbrella grant or carries payment, subscription, license, or other entitlement terms, which are validated and released as ordinary hash-anchored grant items (§XVI.4.2a). No designation trigger, dwell interval, menu selection, or overlay presentation substitutes for any of these steps. After activation, umbrella and active sub-handshake are displayed simultaneously per §XVI.13.4.

Provisioning of the embedded blocks, the searchable catalog projection, and the local search and question-answering path over published sub-handshake catalogs are defined in Annex XVIII §XVIII.3.11; this annex governs capture, crediting, designation, and consent only.

§XVI.8 Entry and Request Lifecycle

§XVI.8.1 Entry status

State	Effect
ACTIVE	Resolution/validation may proceed and an offer or usable state may be presented.
INACTIVE	Temporarily withdrawn; no active offer is presented.
REVOKED	Permanently withdrawn; the durable identifier is not reassigned.
SUPERSEDED	Replaced by an explicitly identified successor; silent reinterpretation is prohibited.
COMPROMISED	Treated as terminal/revoked with an explicit warning or profile-equivalent safety behavior.
CLAIMED	(Use-limited entries only, §XVI.8.4.) Reserved for an identified party pending acceptance; other parties receive CLAIMED_BY_OTHER; reverts to ACTIVE on timeout or decline.
CONSUMED	(Use-limited entries only.) The declared number of uses has been taken; terminal; the identifier

State	Effect
	is not reassigned and a further capture reports CONSUMED.
EXHAUSTED	(Use-limited automations only.) The automation's execution allowance under its grant is spent; the governing handshake remains.

A known durable identifier resolves to its current state rather than silently becoming an unrelated referent. Unknown identifiers are reported as unresolved/capture or lookup failures according to profile, not invented as status records. The same state set applies to Publisher Identifiers at the Namespace Directory level.

§XVI.8.2 Identifier permanence and trust-material rotation

Publisher Identifiers and issued durable local/object references are not reassigned to unrelated referents after activation/publication; a Publisher Identifier is issued once and never reissued, and the address space (§XVI.5.3) is sized so that permanent non-reuse is inexpensive. Cryptographic keys, signatures, certificates, attestations, manifests, policy bindings, resolver endpoints, directory records, and carrier tokens may rotate without changing the durable textual identifier unless the entry itself is deliberately superseded. Rotation changes current validation material, not historical identity; superseded key generations remain retrievable from the directory for verification of historical signatures and commitments.

§XVI.8.3 Invitation lifecycle

A C-class Invitation Entry adds PENDING, CLAIMED, DECLINED, WITHDRAWN, and EXPIRED to the state set of §XVI.8.1 before it reaches ESTABLISHED (equivalent to ACTIVE for a relationship entry). All of these states are states of one identifier; the identifier is never reissued, and an established relationship is evidenced by its Establishment Commitment (§XVI.13.6).

§XVI.8.4 Use-limited entries, contexts, and automations (one-time use)

Declaration. When an issuer sets up a handshake offering — a WR Entry of any class, an addressable sub-handshake, or an automation within a grant — it may, as an advanced option in the handshake setup, declare a use limit. The declaration is part of the authoritative entry or grant record in the issuer's repository, never part of the reference; the WR Code® for a one-time entry is indistinguishable in form from any other. The reference profile declares:

Field	Values	Meaning
use_limit	unbounded (default), 1, or n	How many times the entry, context, or automation may be used.
use_scope	resolution, context, or automation	What counts as a use: a successful establishment through this entry; a session binding under the handshake; or an execution of the named automation.
consume_at	acceptance (default), execution, or resolution	The state transition at which a use is taken.
claim_timeout	duration	How long a CLAIMED reservation is held before reverting to ACTIVE.
require_live_resolution	true (forced for use-limited entries)	Cached directory/resolver state may not be used to present the entry; every capture resolves live.

Use-limited entries (use_scope = resolution). A use-limited entry starts ACTIVE. Successful passage through Gate 5 (§XVI.7.6) by an identified party moves it atomically, by compare-and-set on the resolver, to CLAIMED for that party; a concurrent claimant receives CLAIMED_BY_OTHER and no material. On the party's explicit acceptance the entry moves to CONSUMED (use_limit reached) or, for n > 1, decrements and returns to ACTIVE. If the party declines, the claim times out, or any gate of §XVI.7.6 fails after the claim, the entry reverts to ACTIVE; a failed verification never consumes a use. A subsequent capture of a CONSUMED entry resolves to the status CONSUMED, which the trusted UI shows as a warning: the code has already been used, and if the user did not use it, someone else did.

Use-limited contexts (use_scope = context). The handshake is established normally, but its grant permits exactly use_limit session bindings (§XVI.13.3). Once taken, the handshake remains ESTABLISHED as a record — it can still be evidenced by its Establishment Commitment and still appears in the Handshake State Index — but

no further session may be bound under it; the runtime reports `CONTEXT_EXHAUSTED`. This is the mechanism for "one look" access to a document, workspace, or device view.

Use-limited automations (use_scope = automation). Within an established umbrella or sub-handshake, a specific automation instruction carries `use_limit` in the grant. The finalizer enforces it against the execution record: after `use_limit` executions the automation is `EXHAUSTED`, further triggers are refused, and the governing handshake is unaffected. Combined with consent inheritance (§XVI.13.2), this yields sub-handshake automations that the user has consented to in the umbrella but that can fire exactly once — a safety property against double execution by retry, duplicate scan, or replay as much as a security property.

Consume point. `consume_at = acceptance` is the default and the recommended setting: an accidental scan, a wrong recipient, a policy refusal, or a transport failure must not spend the use. `consume_at = execution` is used for automation scopes. `consume_at = resolution` is permitted only for bearer-style entries that are deliberately not recipient-bound (first come, first served); a profile that enables it must mark the entry as bearer in the trusted UI, because any observer of the code can spend it.

Idempotency and retries. A retry of the same claim or acceptance with the same `request_instance_id` by the same party is not a second use. Only a distinct, completed acceptance (or execution) by a party counts.

Permanence. `CONSUMED` and `EXHAUSTED` are terminal states of a durable identifier (P11). The identifier is never reissued, and its terminal state remains resolvable, so that a code found later still tells the truth about itself.

Carriers. Because the limit is state on the entry rather than a property of the code, it applies identically across carriers: a printed code, its QR/Data Matrix, an NFC tag, and a BLE pointer to the same entry share one use counter. Copying or photographing a one-time code confers nothing once it is consumed.

Trusted UI. Before acceptance of a use-limited entry, the trusted UI displays the limit and the consume point so that the user knows the code will be spent by accepting. A `CONSUMED`, `CLAIMED_BY_OTHER`, or `CONTEXT_EXHAUSTED` status is displayed as such and never as an offer.

Scenarios. Physical handovers (parcels, keys, visitor access, vehicle release); tickets and vouchers without separate redemption infrastructure; single-use source contact or document handover for journalists and NGOs, where a later seized code leads nowhere; device pairing codes (I pairing handshakes with a Pairing Slot) that may bind exactly one device; high-impact automations that must not run twice. Publicly displayed P offerings intended for many parties simply leave the default (unbounded). C-class invitations are inherently single-use through their `PENDING` → `ESTABLISHED` lifecycle and need no declaration.

§XVI.9 Carrier Profiles and Visual Capture

§XVI.9.1 Profile obligations

Each admitted carrier profile declares whether it supports direct identifier transport, pointer/token transport, or both; its payload/framing limits; capture/discovery role; the capture method, its capture risk class (0-3, §XVI.7.7), the component that performs it, and whether that component runs on the host, in an isolated decoder process, or only in the sandbox; rendering constraints; security/privacy requirements; and prohibited trust, consent, or execution behavior. A carrier is admitted according to its actual technical properties rather than by name alone. Every carrier instance — one printed sign, one page, one message, one tag — presents at most one umbrella/parent handshake offering; sub-handshakes beneath it may be co-located.

§XVI.9.2 Reference carrier profiles

Carrier/profile	Reference role	Key boundary
Printed/displayed text	Direct human-readable textual reference of any class.	Supports manual entry, OCR, dictation, and relay; human readability is not authority.
QR / Data Matrix	Direct identifier or pointer; may also carry Bootstrap Payload where capacity permits.	Payload capacity does not make the carrier a trust root.
AprilTag / ArUco	Registered pointer/tag ID plus localization, tracking, orientation, or pose.	Reference profile does not transport the complete canonical textual reference.
NFC	Direct identifier or pointer according to	Tap selects a candidate; it does not

Carrier/profile	Reference role	Key boundary
	NDEF/tag profile and capacity.	establish a handshake.
BLE / radio	Reference discovery profile uses a compact pointer token; other profiles may transport more only if framing/capacity permit.	Proximity is not identity or entitlement.
Email / BEAP™	May carry checked textual references of any class (including PENDING C-class invitations) or carrier tokens.	Channel provenance supplements but does not replace resolver/request-signature validation.
Dynamic display	May show direct identifiers, SE ephemeral sub-handshakes, rotating pointer tokens, or durable addressable sub-handshakes.	Display state does not itself carry authority.
Spoken audio (live speech, voice assistant, phone/conference call)	Direct textual reference, dictated after the spoken wake cue; transcribed locally.	Speaker identity is not verified by the carrier; the transcript is a candidate only.
Recorded/streamed audio and video (video sound track, podcast, broadcast, webinar)	Direct textual reference spoken in the audio track, or shown as on-screen text/visual code in the video frame; captured by transcription or frame OCR.	The medium is a one-to-many carrier; every viewer captures the same reference and resolves it individually.
Messaging / chat / collaboration tools	Direct textual reference or embedded visual code in a message; may include structured fields analogous to email.	Platform provenance, where available, is creditable as channel provenance only.
Documents and attachments (PDF, office documents, images)	Direct textual reference or embedded visual code; captured by text extraction or OCR.	Document signatures, where present, are channel provenance, not WR authority.
Clipboard / share-sheet	Direct textual reference handed between applications as plain text.	Application origin is not verified; the payload is a candidate.

§XVI.9.3 Open carrier admission

The carrier list of §XVI.9.2 is illustrative and deliberately non-exhaustive. Any present or future mechanism capable of conveying, to a conformant client, either the complete normalized textual reference or a registered pointer to a WR Entry is admissible as a carrier once a carrier profile declares its properties per §XVI.9.1. This expressly includes mechanisms not enumerated here — for example, ultrasonic or optical data channels, subtitles and closed captions, screen-reader output, haptic or Braille output, in-vehicle or in-game displays, wearable and AR/VR surfaces, IoT/machine-to-machine messages, and physical marking technologies — and any mechanism developed after this disclosure. Admission never changes the reference: the same prefix, address blocks, and check are carried, and the same parsing, error-detection, directory, resolution, and consent boundaries apply. Where a new carrier can also supply a wake cue, that cue is registered in its profile per §XVI.7.7.

§XVI.9.4 Standardized visual WR Code® presentation and Outer Shape

Where the standardized visual WR Code® representation is used, the canonical class-prefixed textual identifier remains visibly presented within or immediately associated with the rendering. A conformant visual composite additionally displays the publisher's Displayed Responsible Domain (§XVI.2) — the normalized registrable domain, without scheme, subdomain, or path — within or immediately adjacent to the composite, so that the responsible party is human-readable before any capture; official templates and the WR Connect script emit it by default, and a publisher may make it mandatory for its own visual carriers by policy. The displayed domain is accountability text, not authority: it is never a link, never part of the identifier, and never enters the check. A scanner may use a distinctive Outer Shape, including colored or otherwise distinguishable line elements, as a region-of-interest and orientation cue — the visual wake cue of §XVI.7.6 — for locating the textual code and directing OCR, in still images and in video frames alike. The shape is capture assistance only; it is not identifier material, a credential, or a trust anchor.

Rendering profiles preserve the character size, module/cell size, quiet-zone or border behavior, contrast, print tolerance, camera distance, perspective, and other parameters required for the intended decoders. The visible textual field remains independently usable even where the machine-readable carrier contains only a pointer.

Text-first capture order for visual composites. When a conformant WR runtime captures a visual WR Code® composite (visible textual reference plus a co-located QR/Data Matrix or other machine-readable element), it

processes the visible text first: the Outer Shape locates the region, OCR yields the textual candidate, the candidate is parsed and check-validated, the Publisher Identifier is resolved through the Namespace Directory (including the DNS-based binding where that embodiment is used), and the namespace is verified. Only after this verification succeeds does the runtime decode or accept the machine-readable payload, and then only for the purposes its carrier profile declares — for example, a registered pointer, supplementary fields, or a bootstrap payload for non-WR devices. A payload that does not agree with the verified textual reference (different namespace, different entry, unregistered supplementary fields) is discarded and reported as a composite divergence (§XVI.12). The machine-readable element can therefore never introduce a namespace, entry, or destination that the verified text did not already establish, and any URL contained in it is never followed by the runtime (P15). A scan by a non-WR scanner is out-of-band: it sees the raw payload without any of these checks, and the system makes no trust claim about it. **Domain cross-check.** Where a Displayed Responsible Domain is present in the captured composite, the runtime OCR-captures it with the reference, normalizes it under the same rule (registrable domain, lowercase, punycode comparison form), and — after namespace verification (Gate 2) — compares it against the `display_origin` of the resolved publisher's Directory Record. A match adds nothing beyond confirmation; the trusted surface always re-renders the responsible domain from the verified Directory Record, never from the OCR text (P12). A mismatch is surfaced as an unsuppressible warning in the trusted UI, naming both values — "the carrier displays X; the verified responsible party is Y" — with abort as the default action; policy may only escalate, never suppress. Absence of a displayed domain is not a finding: transcribed text, audio, chat, and handwritten carriers legitimately carry none, and validity is decided by resolution, never by the carrier.

§XVI.9.5 Co-located QR/Data Matrix and fiducial functions

A sufficiently large visual composition may place an AprilTag, ArUco marker, or other admitted fiducial within or immediately adjacent to a QR/Data Matrix rendering provided that each decoder still receives the geometry, border/quiet-zone behavior, contrast, and resolution required by its own profile. The QR/Data Matrix may carry the complete textual reference or another payload permitted by its profile; the fiducial independently supplies its registered tag pointer and, where used, pose or localization information. Co-location does not merge their payload semantics.

§XVI.10 WR Code® Generator, Rendering, and Local Re-Rendering

The WR Code® Generator is an emission component operating from an authoritative WR Entry record. One logical entry may produce text, QR/Data Matrix, fiducial assignments, NFC provisioning, BLE/radio configuration, print, web, email, or dynamic-display artifacts without creating a separate identity per carrier. The Generator computes the check symbol from the registry quasigroup at emission time; a human-enterable rendering never carries an unchecked body.

Emitted or provisioned carrier artifacts may be registered with carrier type, direct/pointer mode, token or assignment, generation, deployment site, and spatial information where applicable. Rendering does not expose private keys, relationship secrets, protected content, nonces, or other trust material unless a separate profile expressly authorizes application data within a carrier.

Externally supplied artwork is treated as capture input. A trusted runtime presentation is preferably re-rendered locally from validated identifier/request state. Cross-device optical handoff transfers a reference only; the receiving device performs its own resolver or claim/verification process.

§XVI.11 Bootstrap and Non-WR Device Onboarding

A visual WR Code® composite is normally scanned by a conformant runtime, which reads the visible text first and treats the machine-readable element only as verified supplementary material (§XVI.9.4). The same composite may also be scanned by an ordinary camera or barcode application that knows nothing about WR. Such a scan is out-of-band: it lies outside the WR trust model, and nothing that happens in it is a WR handshake step.

To make that out-of-band case useful rather than dead, a QR or Data Matrix profile may include a Bootstrap Payload intended solely for non-WR scanners: an informational landing reference that explains the entry, shows the same textual WR Code® reference in plain text, and explains how to obtain or invoke a compatible runtime. The landing page may be hosted by the publisher on an origin listed in its directory record or by a central bootstrap service; both are embodiments. A conformant runtime never follows a Bootstrap Payload: it has already verified the textual reference through the directory and ignores any destination in the payload (P15, §XVI.9.4). The bootstrap path therefore exists only for devices that are not yet part of the system and cannot be reached by an attacker through a device that is.

The landing page does not emulate the trusted consent surface, establish the handshake, request unrelated credentials, or become the authority source. A C-class invitation may also be carried in a composite; its Bootstrap Payload explains the invitation without releasing any recipient-bound material.

§XVI.12 Composite Carrier Deployments and Carrier Binding Registry

A WR Entry may maintain a Carrier Binding Registry describing legitimate carrier artifacts or tokens associated with the entry. A record may include carrier type, direct/pointer mode, token or assignment, generation, deployment site, and spatial/pose information where applicable. Registry records are publisher-signed and verified against the directory-registered key (§XVI.6.5).

Within a declared composite or Deployment Site, each encountered component is decoded according to its own profile and resolved against authoritative entry state. Physical proximity alone does not make neighboring artifacts members of one site. A carrier presented as part of a registered composite but absent from the applicable registry is treated as unregistered or suspicious rather than silently accepted.

If multiple registered components are intended to identify the same entry or binding, divergent resolved identities are an alarm condition. For a visual composite scanned by a conformant runtime, the visible textual reference is captured and verified first and the machine-readable element second (§XVI.9.4); the two are equivalent only after the machine-readable element has been reconciled against the verified text and the registry. Registry checks can detect many substitution inconsistencies, but they do not physically prevent copying. A copied reference still requires the same relationship, entitlement, consent, and, where applicable, instance/spatial validation.

§XVI.13 Handshake Hierarchy, Sub-Handshakes, Sessions, and Establishment Evidence

§XVI.13.1 Handshake establishment model (summary)

A handshake is established by: (a) an initiator's signed request capsule naming the participants, profile, and scope and carrying an initiator nonce encrypted to the responder; (b) recipient-bound delivery of that capsule through the Relay; (c) the responder's independent verification of the capsule and delegation chain against the initiator's directory-registered key; (d) the responder's explicit, assurance-gated acceptance producing a signed acceptance record that carries a responder nonce encrypted to the initiator; and (e) both parties recording a Handshake State Record containing both Party Bindings (with verified email addresses), both signatures, the grant, the lifecycle state, and the Establishment Commitment. No step of the reference layer substitutes for any of (a)-(e). For a P-class handshake between an individual user and a publisher, the user is a party in its own right: its Party Identifier is its Principal Identifier, its Principal Key signs the acceptance record and receives the encrypted publisher nonce, and no Delegation Certificate is involved on the user side.

§XVI.13.2 Parent entries, addressable sub-handshakes, and ephemeral entries

A parent or umbrella handshake is not tied to a single code. Multiple durable entries may refer to publisher-, device-, internal-, cross-organizational-, object-, location-, or purpose-specific contexts under a common parent relationship. An independently text-addressable sub-handshake uses SP, SI, or SC according to its parent/domain class; its third block is the combination code of entry and receiving party of §XVI.5.10. A purely session-scoped dynamic context uses an SE ephemeral sub-handshake (§XVI.5.7) or, for machine-only paths, a short-lived carrier token.

Promotion of an SE ephemeral sub-handshake to a durable SP/SI/SC sub-handshake creates a new durable binding. It does not transfer consent, execution rights, or broader disclosure merely because the same object or workflow is referenced.

Consent inheritance from the umbrella. The umbrella handshake's grant is the consent boundary for everything beneath it. When the user accepted the umbrella, the grant enumerated the automations, contexts, object classes, capabilities, and data categories the publisher may operate under that relationship. A sub-handshake whose automation, context, and data use fall entirely within that enumerated grant may be entered automatically — on capture of its SP/SI/SC reference, on proximity, on page match, or on a publisher-side trigger — without a further consent dialogue, because the consent was given at the umbrella level, **to the extent the applicable policy permits**. Whether inherited entry is allowed, and for which sub-handshake kinds, is a policy decision layered on top of the grant: the publisher's policy may declare certain sub-handshakes as always requiring confirmation; the user's own policy may require confirmation for all sub-handshakes, for specific granularities (for example every device-directed sub-handshake), for specific triggers (for example anything entered on proximity rather than on a deliberate scan), or above a chosen assurance level; and a tenant or organizational policy may impose the same

for its principals. Where any applicable policy requires confirmation, the sub-handshake is presented for explicit acceptance even though the umbrella grant would cover it. Policy can therefore only narrow inherited entry, never widen the grant. The runtime still performs the full capture, check, directory, resolver, and lifecycle validation for the sub-handshake, still records it in the Handshake State Index, and still displays it as the active sub-handshake beneath its umbrella (§XVI.13.4); only the consent step is satisfied by inheritance. A sub-handshake that would add any automation, context, capability, or data category not enumerated in the umbrella grant is outside the inherited consent and requires explicit acceptance before it becomes active; a publisher cannot widen an umbrella grant by issuing sub-handshakes. Revocation of the umbrella revokes all inherited sub-handshakes. A sub-handshake automation may be declared use-limited in the umbrella grant (§XVI.8.4), so that it is consented to once and executable a bounded number of times.

§XVI.13.3 Handshake and session are separate state objects

A handshake represents a durable or separately managed relationship/consent context. A session is a temporary runtime context that may be started under an applicable handshake. An established handshake may exist with no active session; one or more sessions may be started, ended, timed out, disconnected, or later re-established without dissolving the handshake.

Ending a session invalidates transient session state and any ephemeral tokens expressly bound to that session, but does not by itself revoke, delete, dissolve, or supersede the parent handshake or durable SP/SI/SC references. Handshake revocation, dissolution, withdrawal, or deletion is a separate explicit transition governed by its own authority, confirmation, audit, and revocation rules.

If a Handshake State Record remains established but the prior session is absent or stale, a new session may be started or rebound after satisfying current authentication/assurance requirements. A new handshake is not required solely because the previous session ended.

§XVI.13.4 Active umbrella and sub-handshake context visibility

During an interaction governed by a hierarchical handshake structure, the trusted UI displays both the umbrella or parent handshake that currently governs the relationship context and the currently active sub-handshake, where one is active. The two levels are simultaneously recognizable from the active interaction surface and remain synchronized with authenticated runtime state.

An established handshake merely stored in the system is not necessarily the active governing context. If no sub-handshake is active, the UI distinguishes the parent-only state from a subordinate-active state. A hidden or separately opened details panel is insufficient as the sole indication of current governing context.

Changing sub-handshakes under the same umbrella updates the subordinate indication without implying that the umbrella changed. Changing the umbrella updates the parent indication and any subordinate indication as applicable. If multiple established relationships are known, the currently governing parent/subordinate pair is perceptibly distinguishable from inactive alternatives. Exact colors, icons, wording, and layout are presentation choices; the technical requirement is continuous synchronization of displayed hierarchical context with the state controlling resolution, consent, capability use, and subsequent actions.

Parallel docked sub-handshakes. More than one sub-handshake may be active at the same time beneath the same umbrella (or, where policy admits, beneath different umbrellas), so that several specialized disciplines — each an established sub-handshake with its own grant — are simultaneously available to the current interaction: for example a mathematics tutoring SP ("Calculus") docked alongside a plotting capability and a citation checker while working on one document. The set of currently docked sub-handshakes is runtime state of the active session context, managed by a **docking and undocking mechanism**:

- **Automatic, context-driven allocation.** The Orchestrator may dock and undock sub-handshakes as the governed context changes — entering a page region with a declared WR Link, opening a document type, starting a workflow step — under the policies of the umbrella and of the user. Automatic docking never widens any grant: it only activates what is already established and consented.
- **Fine-grained manual control.** The adaptive WR Menu presents the docked set and the dockable candidates (established sub-handshakes of the applicable umbrellas), and the user adds or removes individual sub-handshakes at any time — by selection in the menu, or by alias through the assistant: "OK Opty, add Calculus", "OK Opty, remove Calculus" (§XVI.5.9). Manual docking resolves the alias locally, verifies the handshake's state and grant validity (hash and grant checks of §XVI.4.2a apply to any assets it brings), and requires the assurance the grant demands for activation, where it demands any.
- **Visibility and precedence.** Every docked sub-handshake is individually visible in the trusted UI as part of the subordinate indication above; the umbrella indication is unaffected by docking changes beneath it.

Where docked grants overlap, the applicable policy resolves precedence; a docked sub-handshake never extends another's grant, and effects remain attributable to exactly one sub-handshake and its PoAE™ record.

- **Undocking.** Removal deactivates the sub-handshake for the current context immediately, ends any session state bound to that docking, and leaves the established handshake itself untouched (§XVI.13.3); it can be re-docked at any time. Revocation or expiry of a docked sub-handshake undocks it automatically and is surfaced in the UI.

§XVI.13.5 Explicit invitation acceptance and configurable assurance

A newly offered handshake requires an explicit acceptance transition after sufficient validation to identify the offering party and relevant scope. Detection, insertion, check success, resolver success, or mere possession of a runtime does not constitute acceptance.

The required assurance may be selected by handshake profile, publisher policy, user policy, or a combination. A lower-assurance profile may use a simple confirmation; a higher-assurance profile may require two-factor authentication, biometric confirmation, passkey, hardware key, device/authenticator attestation, or equivalent proof, and the issuer may make such proof mandatory for the accepting party, for itself, or for both parties of the handshake — and, by policy, for later resolutions and sessions beneath it. The set of admissible proofs is open; these are assurance requirements of the acceptance step rather than separate handshake classes.

§XVI.13.6 Establishment Commitment and optional anchoring

Purpose. Parties to an established handshake may later need to prove — to each other, to an auditor, or in a dispute — that a specific relationship with a specific scope was established at or before a specific time, without any third party learning, from the evidence artifact alone, who the parties are or that they are related.

Construction. At establishment, each party computes independently and identically:

```
EC = H( DOMAIN_TAG || commitment_version
      || initiator_party_id || responder_party_id[s]
      || request_instance_id
      || nonce_I || nonce_R[s]
      || H(capsule) || H(acceptance_record) )
```

where H is a registered collision-resistant hash function (the reference profile uses SHA-256); DOMAIN_TAG is a fixed byte string distinguishing establishment commitments from every other hash use in the system; party identifiers are Publisher Identifiers for organizations and Principal Identifiers for individual users (§XVI.2); nonce_I is the initiator nonce carried encrypted in the signed capsule; nonce_R is the responder nonce carried encrypted in the signed acceptance record; and, for multi-organization handshakes, each responder contributes a nonce and the identifiers and nonces are concatenated in capsule order. All fields are length-prefixed before concatenation. Both parties compare their computed EC; a mismatch is an establishment failure.

Two-sided nonces, end-to-end confidential. Because the commitment includes a nonce contributed by each party, neither party can compute a valid EC before the other has acted, and neither party can later fabricate a commitment for a relationship the other never accepted. Each nonce is at least 128 bits of entropy from a cryptographically secure generator. Each nonce travels only encrypted to the counterparty's directory-registered (or, for a user, principal) public encryption key, accompanied by H(nonce) in the clear so that the counterparty can verify what it decrypted; the Relay, the directory, the commitment service, and any ledger see at most H(nonce), the capsule hash, and the acceptance hash, which is insufficient to compute EC. Nonces are stored only in the parties' Handshake State Records.

Multi-organization establishment. Where a capsule names several responders, EC is computed once all named responders have accepted, over all responder identifiers and nonces in capsule order. If any responder declines or the request expires before all acceptances, the request instance is terminal and no EC exists; the initiator may issue a new request instance naming only the accepting parties. A profile may alternatively declare per-responder establishment, in which case each initiator-responder pair forms its own bilateral handshake and EC.

Privacy properties. Only EC, H(nonce) values, and signed-material hashes leave the parties. Without both nonces, EC cannot be linked to either Publisher Identifier, to the request instance, or to any other commitment of the same parties; an observer holding the complete directory of Publisher Identifiers cannot test whether two

known publishers are related. Anchoring or escrowing EC therefore discloses neither the parties nor the existence of a relationship between them.

Anchoring tiers. The `commitment_version` and handshake profile select an evidence tier, consistent with the verification tiers used for execution evidence elsewhere in the Canon:

Tier	Where EC is retained	Property
L (local)	In both parties' signed Handshake State Records only.	Mutual evidence; no third party involved.
C (central)	Additionally timestamped and stored by an Optirando™ or publisher-operated commitment service, which returns a signed timestamp receipt over EC.	Third-party time bound without content disclosure.
X (external)	Additionally anchored on a public or consortium ledger, either individually or as a leaf of a Merkle tree whose root is anchored in batches with a returned inclusion proof.	Publicly verifiable time bound; batching additionally hides the number and timing of individual establishments.

Anchoring is optional; a profile may require it, permit it at either party's option, or forbid it. Tier selection is recorded in the Handshake State Record.

Proof procedure. To prove establishment, a party discloses, to the verifier only, the preimage fields (identifiers, `request_instance_id`, both nonces, capsule, acceptance record) and both parties' signatures. The verifier (i) obtains both parties' directory records, including the key generations valid at the time of establishment; (ii) verifies the capsule and acceptance signatures; (iii) recomputes EC; and (iv) compares it with the locally held, centrally receipted, or externally anchored value and its timestamp/inclusion proof. Disclosure of a preimage to one verifier does not affect the unlinkability of any other commitment.

No authority. An Establishment Commitment, receipted or anchored, evidences that a handshake existed with the committed scope at the anchored time. It does not evidence that the handshake is currently active, does not extend or restore a revoked or expired handshake, and grants no capability. Current authority is determined solely by the Handshake State Record and its freshness conditions (§XVI.7.3). A revocation, dissolution, or scope change may be evidenced by a further commitment of the same construction over EC, the revocation record, and fresh nonces, anchored at the same tier.

Relation to class C references. A class-C or SC textual reference is the durable address of a relationship; EC is its cryptographic fingerprint. The two coexist. A resolver never returns EC, nonces, or preimage material in response to a C/SC reference; EC is available only within the parties' authenticated Handshake State Records.

§XVI.13.7 Device-directed sub-handshakes, device selection, and peer-to-peer session establishment

Scope. Device-level addressing exists inside a tenant, beneath an I umbrella, as an SI sub-handshake, and — as a high-assurance option — across organizations, beneath a C umbrella, toward a device that has registered itself by Device Pass. A tenant holds the signed Device Records of its own principals and nobody else's; an initiator holds a counterpart's Device Record only if that device presented and proved it. SP addresses a user as a principal, and SC addresses the responder organization; absent a Device Pass, the device that ends up handling the request is chosen on the receiving side, never by the issuer.

Tenant device list. The tenant's register lists every principal and every paired device with its Device Name, Device Class, and status, created by I pairing handshakes under the tenant. There is no discovery of foreign devices and no directory of devices; a record enters the list only through a pairing under this tenant.

Why the name is unambiguous. A Device Name is unique within its principal, the Device Record binding name → class → key fingerprint is signed by the tenant under the I umbrella in which the device was paired, and every holder of that umbrella verifies the signature against the tenant's verification key from its Directory Record. A name therefore resolves, for every holder of the umbrella, to exactly one key. The name is a selection handle only: the combination code binds the record, the resolver expands it to the fingerprint, and the target device must prove possession of the key. Renaming a device changes the record's display field and generation but not its identity; revoking the device revokes the record, and every SI reference bound to it resolves to REVOKED.

Selection (SI). In the issuing dialog the tenant issuer chooses the receiving party: Principal (any device) or a named device from the tenant's list (name, class, status shown). The default is Principal. Device-level selection is available only where the record is in the list; the UI never offers a free-text device field.

Receiving-side device binding (SP, SC, and principal-addressed SI). Where the receiving party is a principal or an organization, the recipient Orchestrator delivers the request according to the recipient's own policy — to the principal on the principal's devices, or, for an organization, to the principal or role the responder's policy assigns — and the device on which the request is accepted is recorded in the Acceptance Record by its Device-Scoped Principal Identifier. The sub-handshake and its session are bound to that device from acceptance onward. The issuer learns the accepting device's identifier through the Acceptance Record for the purpose of session establishment only; it does not thereby gain the ability to address that device in later codes, because the device is not in its list. The issuer may state an expected Device Class in the entry as an expectation, not an address; acceptance on another class is a consent-time warning under the recipient's policy, never a silent failure.

Role management within a C relationship. Which principals, departments, or devices of the responder organization handle SC sub-handshakes is defined by the responder's policy and executed by its Orchestrator. The responder may route by role ("procurement", "editorial desk"), by named principal, or by device class, and may change that routing at any time without the initiator's involvement; the initiator addresses the organization and sees only acceptances.

Device Pass (high-assurance device-bound SC). For confidential cross-organizational workflows in which exactly one device of the counterpart may act — the device of a named executive, a dedicated hardware-isolated sandbox PC, a source's single trusted phone — the responder may make one specific device addressable to the initiator:

1. *Precondition.* The device is paired in the responder's tenant by an I pairing handshake and therefore holds a tenant-signed Device Record (name, class, fingerprint, principal) and its own device key under the principal's SSO Identity.
2. *Registration.* Over the sealed channel of the C relationship — a peer-to-peer or Relay-sealed session established after this device accepted an SC, or a dedicated "register device" SC issued by the responder itself — the device presents its Device Record and proves possession of the device key. The act requires consent on that device.
3. *Verification.* The initiator verifies the tenant's signature on the record against the responder's verification key from the responder's Namespace Directory Record (Gate 2 assurance), checks the record's generation and status, and checks that the proving key equals the record's fingerprint. What the initiator now knows — "this fingerprint is device *ceo-laptop* of principal X in tenant Y" — rests on the responder's signature and the device's proof, not on any claim of the initiator.
4. *Effect.* The record is held by the initiator as a Registered Counterpart Device of this C relationship. The initiator may thereafter issue SC references whose combination code binds the entry to that record; such a reference resolves only on that device, and every other device, including other devices of the same principal, receives NOT_FOR_THIS_DEVICE. The responder's role management (above) is bypassed for these references by the responder's own act of registration, never by the initiator.
5. *Lifecycle.* The Device Pass is revocable by either side: the responder's tenant revoking the device invalidates the signature and, through the generation check performed at each resolution, every reference bound to it; the responder may also withdraw the pass without revoking the device; the initiator may drop the record. Passes expire and are renewed under the policies of both organizations. A Device Pass never extends to a second device or to a second identity; each device registers separately.

A Device Pass is the only route by which an organization's device becomes addressable outside its own tenant, and it is created exclusively by the device's own presented and proved record. This preserves the known-identifier rule: the initiator binds only what its repository holds, and the repository holds the counterpart's device only because the counterpart put it there.

Peer-to-peer session establishment. Once a sub-handshake is bound to a device on each side — by SI device selection, by an SC to a Registered Counterpart Device, or by receiving-side device binding — its session (§XVI.13.3) may be carried directly between the two devices:

1. **Rendezvous.** Each device publishes, recipient-bound over the Relay and signed with its device key, a connection offer: transport candidates and the transport profiles it supports (direct, relay-sealed, notification-only). The Relay sees only signed, sealed offers.
2. **Authenticated key agreement.** The devices run an authenticated key exchange directly against their device keys. Each side accepts the counterpart key only if its fingerprint equals the Device-Scoped Principal Identifier bound to the sub-handshake (from the combination code for an SI device selection or a Device-Pass-bound SC, otherwise from the Acceptance Record on the accepting side and from the

issuing device's own record on the issuing side). Neither the Relay nor the Directory holds a device private key, so a party in the path cannot interpose.

3. **Path selection.** Direct candidates are tried first; where no direct path exists, the session falls back to the sealed Relay transport. Both paths are end-to-end sealed; the direct path is a latency and data-locality optimization, never a security prerequisite.
4. **Binding.** Session keys are bound to the sub-handshake; the sub-handshake's establishment carries its own Establishment Commitment (§XVI.13.6); session termination does not dissolve the sub-handshake; device revocation on either side terminates the session and invalidates the sub-handshake.

Over the resulting channel run the governed BEAP™ exchanges permitted by the grant — message clones, document handover, automation, context or view sharing — between exactly the two named devices.

§XVI.14 BLE and Ambient Discovery

The reference BLE discovery profile uses a compact rotating or otherwise profile-controlled Carrier Token that identifies a candidate site, entry, or scope for resolver processing. In one embodiment the beacon token is derived as a truncated keyed hash over a per-deployment secret, the entry's registered token identifier, and a coarse time epoch; the issuing resolver, holding the same secret, recomputes the expected tokens for the current and adjacent epochs and maps a received token to its Carrier Binding Registry record. A token outside the acceptance window resolves to nothing. BLE proximity answers what may be nearby; it does not identify the user, create entitlement, or authorize a handshake.

The user device does not need to broadcast a stable public WR user identifier merely to discover nearby entries. Raw observations should be processed locally where practicable, and resolver requests should minimize unnecessary user-identifying telemetry. A future BLE/radio profile may carry the complete textual reference only where its admitted framing, capacity, and interoperability requirements support the complete normalized identifier.

§XVI.15 Offline and Degraded Operation

Textual references may be normalized and check-validated offline. Passing the check does not make the entry trusted; the candidate remains unresolved or provisionally cached until authoritative directory, resolver, or request-state validation is available. Cached directory records and signed material may support bounded pre-verification only within their generation, expiry, revocation, and scope limits.

A PENDING C-class invitation reference may likewise be captured and checked offline, but recipient-bound release, invitation existence, capsule-signature verification, and establishment cannot be inferred from local possession of the string. Any future offline delivery profile preserves the same separation between reference knowledge, initiator authenticity, consent/acceptance, and execution authority.

§XVI.16 Security and Privacy Considerations

Issue	Boundary / control
Capture error	Class-aware checks reject every single-symbol substitution and adjacent transposition, and many other typing, OCR, speech, and transcription errors, before resolver or Relay lookup. A valid check remains syntactic evidence only.
Cross-class confusion	Prefix symbols participate in error detection and the parser refuses unknown/incomplete terminal paths instead of falling through to another class. Because every reference begins with a prefix, no address body can be misread as a prefix.
Context mismatch	A syntactically valid reference can still be invalid for the current user, tenant, relationship, device, or target. Policy/state checks remain separate from capture validation.
Entry substitution or stale version	Entry verification requires the entry context to exist in the repository and to match the registered assignment and version exactly; sub-handshake combination codes are expanded and both constituents verified.
Unverifiable individual party	Individuals have no DNS; they are verified by their SSO Identity (the one verified email address per principal), and both parties' SSO Identities are bound into every handshake.

Issue	Boundary / control
Email/domain mismatch (personal or look-alike mailbox acting for an organization)	A principal acts for a publisher only if its SSO-verified email's domain is one of the publisher's DNS-verified domains; Delegation Certificates carry the email and are verified against DNS at signature time.
Device mis-targeting (reference meant for one device used on another)	The receiving-party constituent may be bound to a Device-Scoped Principal Identifier; other devices of the same principal receive NOT_FOR_THIS_DEVICE. Lost-device revocation invalidates all device-bound references without touching the principal-level identity.
Identity substitution across devices or logins	The SSO Identity is the verified email; Principal Keys, device pairings (I pairing handshakes), and Party Bindings are all issued under it. A second email identity cannot be introduced by pairing, by re-login, or by delegation, so a handshake bound to one email cannot be resumed or extended under another.
Carrier substitution and cloning	Registry/context validation can detect many inconsistent composite carriers. Copying a legitimate reference alone does not create authority.
Tampered connector script (XSS, compromised CMS, supply chain)	Connector code is hashed as served and compared with the canonical or dual-signed publisher-registered hash before any data block is read; mismatch blocks auto-surfacing and provenance credit regardless of block signature (§XVI.7.4a).
Risky capture methods (untrusted media, documents, unknown tag formats)	Channel-specific capture profiles; any parsing-risk capture runs only in the sandbox, is labeled as such in the trusted UI, and is never auto-inserted; host-side capture only for text or verified structured blocks (§XVI.7.7).
Malicious machine-readable payload (swapped QR sticker, poisoned NFC tag)	Text-first capture (§XVI.9.4): the runtime verifies the visible textual reference through the directory before decoding the payload, then accepts the payload only if it agrees with the verified reference; a payload naming another namespace, entry, or destination is discarded as composite divergence. Out-of-band scans by non-WR scanners carry no WR trust.
Publisher/namespace spoofing	Trusted presentation depends on the signed Namespace Directory Record, account-holder verification, DNS domain-control proof, the bidirectional origin binding, and directory-registered publisher keys rather than carrier text, page-supplied keys, or visual branding alone. For C references both organizations are verified this way, by each side independently.
Directory compromise or stale directory	Records are dual-signed (directory and publisher), carry generation and expiry, and superseded records remain retrievable; a client detects an unsigned, expired, or single-signed record and treats the namespace as unverified. Cached records are invalidated on any generation change, so a rotated key or revoked namespace takes effect on the next resolution, not at cache expiry.
Email spoofing and forwarding	SPF/DKIM/DMARC/DNS alignment against directory-listed origins may add provenance credit but does not replace signed-request or resolver validation. Forwarding that breaks provenance removes that credit without invalidating an independently captured reference.
Wrong recipient	An invitation reference does not prove that the holder is a named responder. Recipient-bound claim (delegation chaining to the directory-registered responder key) controls release; a non-responder learns at most that the reference is not for them. There is one C entry per ordered pair, so no selection ambiguity arises.
Fabricated relationship evidence	EC requires nonces from every party and both signatures; a party cannot produce a valid commitment unilaterally.
Intermediary linkability (Relay, directory, commitment service)	Nonces are encrypted end to end between the parties; intermediaries hold only H(nonce) and material hashes and cannot compute or link EC.
Key custody and delegation	Principals sign with Principal Keys under scoped Delegation Certificates; publisher root keys are not required on end-user devices; revoked or out-of-scope delegations invalidate signatures.

Issue	Boundary / control
Evidence linkability	EC is a domain-separated hash over high-entropy nonces; anchored values cannot be linked to parties, to each other, or tested against the directory. Batch anchoring additionally hides cardinality and timing.
Evidence misuse as authority	Anchored EC is historical evidence only; current authority comes from the Handshake State Record.
Replay and stale state	Request freshness, nonce/instance state, session epochs, expiry, revocation, and generation are profile state independent of the visible reference.
Reuse, copying, and double execution of one-time material	Use limits are resolver/grant state on a durable identifier, enforced atomically (compare-and-set), consumed only at acceptance or execution, immune to cached state, and shared across all carriers of the entry; a consumed code resolves to a visible CONSUMED warning.
Consent-surface spoofing	Carriers and externally supplied artwork cannot substitute for the trusted runtime state used for acceptance and authorization.
Link-based attacks (phishing, redirects, look-alike domains, link tracking)	Eliminated by construction (P15): no carrier delivers a link; a reference is plain checked text or a registered pointer, resolved only through the signed Namespace Directory by the runtime. Any URL encountered near a reference is ignored by the capture pipeline.
Malicious email content (attachments, embedded objects, links)	The host Orchestrator's trusted surface does not open originals; all mail is BEAP-transformed on receipt, attachments are shown in the sandbox or as reconstructed PDF/text within BEAP, and original binaries or non-WR links are by default opened via the BEAP clone mechanism in the sandbox Orchestrator (after an explicit warning in the standard tier; hardware-isolated in the high-assurance tier). Whitelisting verified URLs to open directly is an explicit super-admin policy deviation, applies to links only, and is never derived from content. Third-party binaries are treated as hostile irrespective of provenance; policy may restrict further. WR Codes are recognized during de-packaging as text only.
Session/handshake confusion	Session termination does not silently revoke the durable handshake, and a stored handshake is not treated as currently governing unless authenticated runtime state makes it active.
Hierarchical UI context	The UI keeps the active umbrella and active sub-handshake visible together to reduce wrong-scope operation in nested contexts.
Enumeration and metadata privacy	Directory, resolver, and Relay endpoints use anti-enumeration controls and minimize user-resolution telemetry. Basic public namespace verification does not require stable user identification where entitlement is unnecessary.

§XVI.17 Non-Limiting Technical Implementations and Specification Support

§XVI.17.1 Architecture-level generalization

The reference profile supplies concrete, enabled examples of the disclosed mechanisms without making every profile representation an architectural dependency. The literal glyphs P, I, C, S, SE; the use of a hyphen; a six-symbol publisher field; a five-symbol default local field; Crockford Base32; the Class Value Table; a Damm/order-32 check; SHA-256; particular tier names; particular badge text; and particular carrier products are embodiments or profile choices unless a functional dependency is expressly stated.

Equivalent implementations may represent class information through another registered symbol set or machine representation; implement prefix recognition through a trie, table, state machine, parser automaton, or equivalent logic; vary the number, order, role, and length of address fields and the number and placement of error-detection fields as set out in §XVI.5.11; use another human-recognizable field boundary; use a different class-aware error-detection algorithm; express relationships by pairs, discriminated pairs, collapsed identifiers, or participant lists; distribute or federate directory, resolver, Relay, and commitment services; use another collision-resistant hash or commitment scheme with two-sided randomness; anchor commitments on any timestamping or ledger substrate; and use different UI context indicators or authentication factors while preserving the disclosed state separations and technical acts.

§XVI.17.2 Representative computer-implemented method

1. monitor one or more admitted capture surfaces — including text, image, video-frame, audio, email, messaging, document, clipboard, and radio/proximity surfaces — for a modality-specific wake cue and for a prefixed textual reference or registered carrier token, while keeping detection separate from acceptance and execution;
2. on candidate detection, optionally wake a local presentation surface and independently apply a user-configurable rule controlling whether the candidate is inserted into a visible entry field;
3. normalize case and non-semantic presentation features and incrementally process a variable-length class/subclass prefix, beginning at the first symbol, against registered terminal, non-terminal, and invalid states;
4. withhold prefix-block completion while additional classification symbols are required and complete the block only after a terminal prefix is recognized;
5. optionally, on completion of the namespace block, obtain the signed directory record and display verified namespace identity without treating the unfinished full reference as trusted;
6. validate a terminal error-detection element computed over the class values of the terminal prefix and the complete normalized address body before remote resolver or request-state lookup;
7. select the resolver branch from the terminal prefix and, using the endpoints and keys of the directory record, obtain authenticated publisher, entry, object/binding, relationship, and lifecycle state as applicable, releasing pre-consent invitation material only to a recipient-bound claim;
8. determine whether a relevant handshake already exists independently of whether a temporary runtime session is active;
9. if a usable handshake exists, reuse it within its existing grant and establish or re-bind a session where necessary; otherwise present a verified invitation and obtain the assurance level required for explicit acceptance;
10. when hierarchical handshake state governs the interaction, display both the active umbrella/parent handshake and the active sub-handshake, where one is active, and update those indications as the authenticated governing state changes;
11. permit session termination without dissolving the underlying handshake and permit protected-data release or execution only within the validated grant and applicable authorization/finalization rules;
12. where the issuer declared a use limit as an advanced setup option, enforce it as atomic resolver or grant state — reserving on recipient-bound release, consuming on acceptance or execution, reverting on failure, refusing cached presentation, and resolving a consumed identifier to a visible terminal status;
13. on establishment, compute at each party an identical domain-separated commitment over the parties' identifiers, the request instance, a nonce contributed by each party and exchanged only in encrypted form between the parties, and the signed request and acceptance material, retain the commitment in the authenticated handshake state, optionally submit only the commitment to a timestamping or anchoring service according to the selected evidence tier, and, on later proof, verify a disclosed preimage against the retained or anchored commitment without treating the commitment as current authority.

The technical combination provides deterministic pre-resolution classification, local error detection across heterogeneous capture paths, deterministic and privacy-preserving namespace resolution, a common transition from capture into authenticated state, separation of durable relationship state from temporary session state, continuous human-visible alignment between hierarchical handshake state and the context in which subsequent actions are performed, and unforgeable, unlinkable evidence of establishment that carries no authority.

§XVI.17.3 External-technology references and non-admission

References to QR Code®, Data Matrix, NFC, Bluetooth®, AprilTag, ArUco, GS1 Digital Link, DNS, URLs, email authentication, hash functions, Merkle trees, public ledgers, and other external techniques provide interoperability and implementation context. They do not state that any cited feature or combination is prior art, material to patentability, or the source of the disclosed WR architecture. The described technical allocation of capture, parsing, directory resolution, validation, handshake state, consent, session, evidence, and execution functions stands independently of any particular carrier brand or standard.

§XVI.18 Version History

Version	Consolidated change
v1.95 — 30 August 2026	Check-quasigroup definition in §XVI.5.4 and Appendix A.1 corrected to $x * y = (\alpha \cdot x) \oplus y$ (Registry Material v1.4 operation). The previously printed operation $\alpha \cdot (x \oplus y)$ is commutative and therefore not totally anti-symmetric (the second condition $x * y \neq y * x$ fails for all $x \neq y$; a sweep over 32^3 triples exercises only the first condition, which is why the error passed verification). Both operations produce identical check symbols and identical accept/reject decisions for every input, so all published references, test vectors, and deployed verifiers are unaffected. Zero-diagonal claim removed; check-symbol extraction restated as $k = \alpha \cdot \text{interim}$; A.1 table regenerated; A.2 interim column updated.
v1.94 — 30 August 2026	New §XVI.7.7a "Element-scoped designation": open trigger catalog for selecting one credited sub-handshake block among many — defined WR Pointer™ gesture, cursor dwell, hover-invoked adaptive WR Menu selection, tap/click on the locally rebuilt visual, Handshake Alias by voice, gaze dwell on gaze-capable devices; triggers are designation, never acceptance; designated capture is Class 1 with hash pre-check before payload evaluation; unchanged §XVI.7.7 state machine and §XVI.13.2/§XVI.13.5 consent scale, entitlement-bearing activations established as own ceremonies with items released per §XVI.4.2a; cross-reference to Annex XVIII §XVIII.3.11 for provisioning and catalog projection.
v1.94 — 30 August 2026	New §XVI.7.2a "Catalog pages": beneath the single umbrella offering of §XVI.7.2, a page may carry any number of signed sub-handshake-class script blocks (SP/SI/SC/SE), e.g. one per catalog item; per-block crediting conditions (page-level connector-hash pre-check per §XVI.7.4a, block signature against the directory-registered key, Publisher Identifier match to verified origin and mapped umbrella, declaration of the referenced sub-handshake in the currently valid manifest/catalog-commitment generation); generation supersession retires embedded blocks without touching the umbrella; failed blocks fall back to manual capture; single-parent rule unchanged; credited candidates never bulk-surfaced.
v1.93 - 24 August 2026	New §XVI.5.12 "Design rationale and encompassed simplified variants": rationale for the fixed-alphabet Publisher Identifier versus a domain name (local error behaviour, lifetime decoupling, DNS as one channel among several); seven simplified variants (no class prefix, no check, flat token, machine-readable-only, larger alphabet, namespace-only short form, existing identifier systems) stated neutrally as embodiments encompassed by §XVI.5.11, each with the property it trades away; section marked for review (green).
v1.92 - 24 August 2026	§XVI.4.2a: brief paragraph on paid offerings — publisher entries may offer entitlement-gated commercial items (pro library/LoRA versions, premium capabilities, subscription tiers) via the WR Store™ or other governed channels; payment/entitlement mechanisms open, paid items remain ordinary hash-anchored grant items. WR Store™ added to the Trademark Notice.
v1.91 - 24 August 2026	End-of-document License Notice finalized: governance wording ("subject to the same governance and license terms ... restated below and apply without modification"), plain copyright line; restated terms (restrictive documentation license, no further rights) unchanged beneath.
v1.90 - 24 August 2026	Displayed Responsible Domain added: publisher-declared display_origin in the Directory Record (registrable base domain, no scheme/subdomain/path, lowercase, punycode comparison form) shown on visual composites before capture; OCR cross-check after Gate 2 against the resolved record; mismatch → unsuppressible UI warning naming both values, abort default, policy may only escalate; absence on non-visual carriers has no effect. §XVI.2, §XVI.9.4, A.4.1, §XVI.19.1 (numeral 130).
v1.89 - 24 August 2026	End-of-document legal block made internally consistent: restrictive documentation license (citation with attribution) expressly granted; all further rights reserved (no patent, trademark, or implementation rights).
v1.88 - 24 August 2026	End-of-document legal block completed: copyright line, no-license-granted clause, and permitted-use clause added beneath the License Notice.
v1.87 - 24 August 2026	Trademark Notice precised: WR Code® is a trademark registered in the European Union (EUIPO).
v1.86 - 24 August	§XVI.19 retitled "Brief Description of the Drawings" and production/planning language removed;

Version	Consolidated change
2026	License Notice repeated at the end of the document.
v1.85 - 24 August 2026	License Notice added to the front matter: Annex is an integral part of the Optirando™ specification, licensed under the same terms as BEAP™, WR Desk™, and PoAE™.
v1.84 - 24 August 2026	Parallel docked sub-handshakes (§XVI.13.4): several established sub-handshakes may be simultaneously active beneath the umbrella; docking/undocking mechanism with automatic context-driven allocation and fine-grained manual control via adaptive WR Menu and alias voice commands ("OK Opty, add/remove Calculus"); docking never widens grants, per-sub-handshake visibility and PoAE™ attribution, undocking leaves the handshake untouched. §XVI.5.9 aligned.
v1.83 - 24 August 2026	Handshake Alias added (§XVI.2, §XVI.5.9): user-assigned local names for established handshakes and sub-handshakes, resolvable by voice via the assistant wake phrase ("OK Opty, load Calculus"); local lookup only, never on a carrier, no authority, no network access on unknown alias; recall yields the canonical reference and proceeds through the normal gates and grant.
v1.82 - 24 August 2026	§XVI.19.1 added: reference-numeral list for FIG. 1 (100–128) with section cross-references; FIG. 1 examples fixed to Appendix A.2 test vectors.
v1.81 - 24 August 2026	§XVI.7.7: standard textual form defined as the WR Code® designation followed by the reference, with or without colon, both recognized as one wake cue; manual capture (screenshot, photo, clipboard, selection, recording) mandated as always-available path through the same pipeline, gates, and consent.
v1.80 - 24 August 2026	§XVI.4.2a: cryptographic versioning and hash anchoring of every grant item made explicit; execution/rendering/loading only after local hash check against the grant and grant validity check, hard stop on failure; inventory declared illustrative — any securely transportable data may be made available as a grant item.
v1.79 - 24 August 2026	§XVI.4.2a grant inventory rebuilt from the corpus: parametric, procedural, declarative (incl. Context Graph, Context Frames, Publisher Semantic Layer), spatial context (Scope Manifest, Spatial Anchor Registry, Virtual Replica, recognition descriptors), presentation assets (normalized media incl. video where admitted, frames, Mini-App compositions, derived assets), capability requests/offers, automation triggers, configurations and policy; message clones removed (payloads under the grant, not grant items); case-dependence stated. Phase 8 aligned.
v1.78 - 24 August 2026	Validation phase 8 expanded and new §XVI.4.2a "Content of a grant and governed release": grant items (hashed model artifacts/adapters, configurations, AI instructions in text form, WCR recordings, capability requests/offers, documents and clones, automation triggers, scope/limits/lifecycle), PoAE™ and PoAC™ proof regimes defined self-containedly, all transport as BEAP™ capsules beneath the handshake.
v1.77 - 24 August 2026	Validation phase 7: consent-free reuse and consent inheritance made explicitly policy-dependent in both directions.
v1.76 - 24 August 2026	Authentication assurance made explicit without narrowing: validation phase 6 and §XVI.13 now state that the issuer or profile may require proof such as passkey, hardware key, biometric, or multi-factor from either or both parties; set of admissible proofs open.
v1.75 - 24 August 2026	Device Pass added as optional high-assurance mechanism: a device of the responder registers itself to the initiator beneath a C umbrella by presenting its tenant-signed Device Record and proving key possession over the sealed channel; the initiator verifies the signature via the responder's Directory Record and holds the record as a Registered Counterpart Device, enabling SC references that resolve on that device only. Opt-in per device, revocable by either side, expiry/renewal by policy. §XVI.2, §XVI.5.8, §XVI.5.10, known-identifier rule, §XVI.13.7, A.4.4 aligned.
v1.74 - 24 August 2026	Device selection restricted to SI: only a tenant holds signed Device Records, so device granularity of the receiving-party constituent exists only beneath an I umbrella. SP addresses the user as principal; SC addresses the responder organization, whose role management (routing to principals, departments, devices) is policy-defined and invisible to the initiator. Cross-organization device register and issuer-side learning of counterpart devices removed; receiving-side device binding via the Acceptance Record retained for session establishment only. §XVI.2, §XVI.5.8, §XVI.5.10, known-identifier rule, §XVI.13.7 aligned.

Version	Consolidated change
v1.73 - 24 August 2026	Appendix A completed: Damm order-32 quasigroup defined algebraically ($x * y = \alpha \cdot (x \oplus y)$ in $GF(2^5)$, polynomial $0x25$), exhaustively verified totally anti-symmetric with zero diagonal, full table and reference implementation; positive test vectors for all seven classes and negative vectors; Class Value Table; DOMAIN_TAG and encoding rule; field schemas for Directory Record, Delegation Certificate, Capsule, Acceptance Record, Device Record. §XVI.5.4 algorithm paragraph made self-contained; §XVI.5.5 examples replaced by registry-derived references; placeholder check symbols removed.
v1.72 - 24 August 2026	Class SD removed (Option A): the receiving party's granularity — organization, principal, or named device — is a property of the combination code and the Device Record, not of the prefix; device-directed sub-handshakes are SP, SI, or SC of the umbrella's class. Final class set: P, I, C, SP, SI, SC, SE. Prefix grammar, form tables, check examples, §XVI.5.8, §XVI.5.10, known-identifier rule, §XVI.13, §XVI.16, Appendix A aligned.
v1.71 - 24 August 2026	Class D removed. Device onboarding is an I pairing handshake (context constituent = Pairing Slot, receiving party = principal); machines/servers/robots are paired devices with a Device Class, offered externally as P/SP entries. P is now the only unaddressed form (check attached). Class Value Table, prefix grammar, block-length table, §XVI.5.8, §XVI.5.10, known-identifier rule, §XVI.8.4, §XVI.13.7, §XVI.17, Drawing Plan and Appendix A aligned.
v1.70 - 24 August 2026	Device model completed: device onboarding is a D pairing (Pairing Slot, principal-bound internally); pairing assigns a Device Name (unique per principal) and creates a signed Device Record; SD redefined as a sub-handshake directed to one specific named device selected from the umbrella's device list, combination code binds the Device Record; known-identifier rule updated; new §XVI.13.7 (device list, why the name is unambiguous, selection, device binding by acceptance, P2P session establishment).
v1.69 - 24 August 2026	SD corrected: the issuer knows nothing about the device to be paired, so the SD combination code binds a Pairing Slot (issuer-side entry, optional expected Device Class and expiry) + the principal; the device's D entry, Device Class, and Device-Scoped Principal Identifier are created as the result of the pairing; slot single-use. §XVI.2, §XVI.5.8, §XVI.5.10 aligned.
v1.68 - 24 August 2026	Class I moved to the addressed form I-III-III-BBBBBB-X (combination code of internal context + receiving principal or device, separate check), replacing the earlier "tenant identifier twice" residue; two rendering families stated (P/D attached check; I, C, S-classes separate check). Known-identifier rule added: the receiving-party constituent may bind only identifiers registered with the issuer (publisher IDs always; principals after a handshake; devices only after SD pairing under the issuer's context), with per-class consequences incl. SC limited to the responder organization or relationship-registered identifiers. §XVI.1.1, §XVI.2, §XVI.5.1, §XVI.5.2, §XVI.5.3, §XVI.5.8, §XVI.5.10, §XVI.7.6 Gate 4, figure plan aligned.
v1.67 - 24 August 2026	§XVI.5.8 rewritten: completeness determined by grammar alone (per-class block lengths table); submission as explicit act that first triggers network access (SUBMITTED state); progressive namespace verification defined per class — default on for P/SP and both blocks of C/SC (public records, P10), never for D/I/SI/SE/SD; per-class entry experience spelled out. §XVI.7.7 state chain and Gate 1 aligned.
v1.66 - 24 August 2026	Directory-declared local-field length removed: the check is always the last symbol and candidates are framed by class-fixed length and submission framing, so no directory lookup is needed for parsing; progressive verification stated as optional (§XVI.2, §XVI.5.1, §XVI.5.3, §XVI.5.8, §XVI.5.11, §XVI.6.4, §XVI.17.2, figure plan).
v1.65 - 24 August 2026	Repository links clarified: they open the publisher record on the Optirando™ WRC (external, but pinned and runtime-constructed from the validated identifier); P15 scoped to content-derived links (§XVI.4.3, P15).
v1.64 - 24 August 2026	D and I disambiguated (D = device as referent with registered Device Class, I = tenant-internal resource/context, never a device). SD defined as the pairing that issues a device its own Principal Key under the SSO Identity, yielding a Device-Scoped Principal Identifier. Device targeting via receiving-party granularity (principal vs. specific device), NOT_FOR_THIS_DEVICE, Device-Class policy, device-key revocation. §XVI.2, §XVI.4.2, §XVI.5.10, §XVI.6.5, §XVI.7.6 Gate 4, §XVI.16 aligned.
v1.63 - 24 August 2026	Device examples extended to smartphone, laptop, smart glasses, and wearable (§XVI.2, §XVI.5.10).
v1.62 - 24	Isolation model reframed as policy-dependent: recommended default is out-of-band opening of all links

Version	Consolidated change
August 2026	and attachments in the sandbox Orchestrator (warned VM in standard tier, hardware-isolated in high assurance); whitelisted verified URLs may open directly as an explicit super-admin policy deviation (links only, never content-derived); policy may always tighten. P15, §XVI.2, §XVI.7.4, §XVI.16 aligned.
v1.61 - 24 August 2026	Absolute "never" wording removed; isolation tiers defined: standard/free tier opens links and attachments in a host-resident sandbox VM only after an explicit warning; high-assurance tier requires a hardware-isolated sandbox Orchestrator, prohibits host-side opening, and allows policy-whitelisted verified URLs (links only). Attachments across tiers: sandbox display or reconstructed PDF/text-only within BEAP. P15, §XVI.2, §XVI.7.4, §XVI.16 aligned.
v1.60 - 24 August 2026	Host Orchestrator and sandbox Orchestrator distinguished: external links and original attachments are redirected by the BEAP clone mechanism to the sandbox Orchestrator (recommended hardware-isolated sandbox) and opened there; the host Orchestrator never opens them. P15, §XVI.2, §XVI.7.4, §XVI.16 corrected.
v1.59 - 24 August 2026	§XVI.1.2 reworded: a handshake request is human-enterable as the C-class WR Code® itself; the statement now says only that no second code format exists for requests.
v1.58 - 24 August 2026	Uniform sub-handshake form restored and made explicit for all of SP/SI/SC/SE: block 1 prefix, block 2 publisher identifier, block 3 six-symbol combination code of entry + receiving-party identifier, block 4 check (separate). The v1.57 SC exception is withdrawn. Terminology "combination code" replaces "collapsed local block"; §XVI.1.1, §XVI.2, §XVI.5.1, §XVI.5.3, §XVI.5.6, §XVI.5.7, §XVI.5.10 and verification passages aligned.
v1.57 - 24 August 2026	SC form defined: SC-IIIIII-RRRRRRLLLLL-X — third block is the responder Publisher Identifier in the clear followed by the individual entry symbols, check separate, so that the responder sees its identifier before retrieval and every sub-handshake beneath a C umbrella has its own entry (§XVI.2, §XVI.5.1, §XVI.5.3, §XVI.5.6, §XVI.5.10).
v1.56 - 24 August 2026	Capture risk classes 0-4 defined as a table in §XVI.7.7 (class 4 never a capture path; sandbox rendering as preferred path for foreign documents); carrier profiles must declare the class (§XVI.9.1). Connector integrity: registered WR Connect script hash (canonical platform list or dual-signed publisher record), compared before any data block is read (§XVI.6.4, §XVI.7.2, §XVI.7.4a, §XVI.16).
v1.55 - 24 August 2026	§XVI.7.7: auto-insertion restricted to capture methods safe by construction (standardized for websites, email, BEAP via signed script block / de-packaging); capture defined as channel-specific with a common invariant — no untrusted content path on the host; parsing-risk capture only in the sandbox, labeled in the trusted UI, never auto-inserted. §XVI.9.1 profile obligations and §XVI.16 aligned.
v1.54 - 24 August 2026	Framing corrected: validity of an offering is decided by resolution, never by carrier (a code from a video, podcast, sign, or screenshot is equally valid once verified); the signed WR Connect script block is required only for automatic detection and insertion from websites and email; manual capture always available. P12, §XVI.7.2, §XVI.7.4, §XVI.7.4a, §XVI.7.7 aligned.
v1.53 - 24 August 2026	§XVI.7.4: attachments are either BEAP-transformed or opened only as BEAP clones in the sandbox; third-party binaries treated as potentially hostile regardless of provenance; policy may restrict or forbid direct attachment processing for high-assurance deployments; no host-side open path. §XVI.16 aligned.
v1.52 - 24 August 2026	P12 rebalanced: external artwork is legitimate capture input (OCR, decode, direct capture, Orchestrator screenshots) but never a source of trust; trusted visuals are always rebuilt locally; the strict signed-script-block procedure is specific to website and email handshake offerings, where it is a zero-effort integrated proof. §XVI.7.4 and §XVI.7.4a aligned.
v1.51 - 24 August 2026	Legacy figure appendix removed; §XVI.19 reduced to the drawing plan for newly produced figures.
v1.50 - 24 August 2026	§XVI.7.4 introduction: Orchestrator is not a conventional mail client — all mail BEAP-transformed on receipt, originals viewable only in the sandbox, original binaries and non-WR links opened only via the BEAP clone mechanism into the sandbox, WR Codes recognized during BEAP de-packaging. P15 and §XVI.16 aligned.
v1.49 - 24 August 2026	Email capture corrected: messages and attachments are transformed into BEAP format, the reference is extracted as text (normally from the signed WR script block), message graphics are never decoded; one umbrella offering per carrier instance; the visual code is rebuilt locally and can be OCR-scanned

Version	Consolidated change
	from that rendering. New §XVI.7.4a signed WR script block for email and web. P12 sharpened to a firm rule. §XVI.7.2 step 5 and §XVI.9.1 aligned.
v1.48 - 24 August 2026	Ephemeral references reclassified as the SE sub-handshake class (was a standalone E class): same three-block collapsed form as SP/SI/SC, bounded session lifecycle; defined in summary, §XVI.2, §XVI.5.1, §XVI.5.2, §XVI.5.4 ([25,14]), §XVI.5.7, §XVI.5.10, §XVI.9.2, §XVI.13.2, §XVI.17.1, figure plan, Appendix A.
v1.47 - 24 August 2026	P7 lists websites (discovery and on-page references), video-frame OCR, audio, messaging, and documents among the converging capture paths and states always-on detection with toggleable insertion.
v1.46 - 24 August 2026	WR Link definition extended: context-dependent presentation of agreed automation links in the WR Overlay and the adaptive WR Menu, rendered by the trusted runtime from validated handshake state.
v1.45 - 24 August 2026	Signed Site Manifest clarified as the page-to-entry map only, distinct from repository context (adapters, agent instructions, capabilities) and from WR Links; WR Link defined as site-owner-declared anchors for WCR recordings and governed automation routing, consumed only after entry verification and under an active handshake; §XVI.7.2 step 5 aligned.
v1.44 - 24 August 2026	Directory caching clarified: records are cacheable, but any change increments the generation and invalidates all cached copies immediately; expiry is an upper bound, not a validity promise (§XVI.2, §XVI.6.4, §XVI.16).
v1.43 - 24 August 2026	§XVI.2 Local/Object/Binding Reference: parent and object constituents of the collapsed block defined explicitly per sub-handshake class (SP, SI, SC).
v1.42 - 24 August 2026	Email-domain agreement added as a third verification condition: the acting principal's SSO-verified email must lie in the publisher's DNS-verified domain (or a directory-registered domain with its own DNS proof); Delegation Certificates carry the email and are valid only while that holds. §XVI.1.3, §XVI.2, §XVI.6.5, §XVI.7.6 gates 2 and 6, §XVI.16 aligned.
v1.41 - 24 August 2026	SSO Identity defined as the one verified email address per principal, to which Principal Keys, Party Bindings, and device pairings are bound; SD clarified as device pairing under the same SSO Identity; §XVI.6.5 B2C paragraph and §XVI.16 aligned so that no reader can infer a login identity separate from the handshake-bound email.
v1.40 - 24 August 2026	§XVI.2 Local/Object/Binding Reference: clarified that SP/SI/SC keep the ordinary local-block form (5 + attached check) and that only the parent C form has a 6-symbol second block with separate check; wording no longer uses "class" ambiguously.
v1.39 - 24 August 2026	Use-limited (one-time / n-time) entries, contexts, and automations added as an issuer-declared advanced setup option (§XVI.8.4): use_limit, use_scope, consume_at, claim_timeout, require_live_resolution; states CLAIMED, CONSUMED, EXHAUSTED (§XVI.8.1); atomic claim/consume with reversion on failure; consent-inherited single-use automations (§XVI.13.2); §XVI.5.11 variant, §XVI.16 row, method step 12.
v1.38 - 24 August 2026	New §XVI.7.6 "Pre-delivery security boundary": six ordered hard-stop gates (local syntax, namespace verification, entry verification, self-match, recipient-bound release at the Relay, capsule admission at the receiver Orchestrator) that must all pass before a request capsule is admitted to the receiver's trusted surface; former §XVI.7.6 renumbered to §XVI.7.7.
v1.37 - 24 August 2026	Relay restored as the sealed store-and-forward transport of the architecture (had been demoted to optional in v1.21): defined in §XVI.2, named in the directory record, and included in the end-to-end sequence (§XVI.1.3), the processing stages (§XVI.4.1 stage 5a), §XVI.7.5, and §XVI.13.1.
v1.36 - 24 August 2026	Entry verification made explicit: entry context must exist in the repository and match assignment and version exactly; sub-handshake combination codes are expanded and both constituents verified. Individual (B2C) parties verified as account holders with verified email (no DNS). Party Binding defined; both parties' verified email addresses bound into every handshake, capsule, acceptance record, state record, and (via hashes) the Establishment Commitment. §XVI.1.3, §XVI.2, §XVI.4.1, §XVI.6.5, §XVI.7.5, §XVI.13.1, §XVI.16 aligned.
v1.35 - 24 August	Two-sided verification for C references: account-holder and DNS domain-control verification is performed for both Publisher Identifiers, by each party independently (initiator before issuing,

Version	Consolidated change
2026	responder on receipt and at acceptance, later resolvers for both). §XVI.1.3, §XVI.4.1, §XVI.5.6, §XVI.6.5, §XVI.7.5, §XVI.16 aligned.
v1.34 - 24 August 2026	Inherited sub-handshake entry made policy-dependent: publisher, user, and tenant policies may require explicit confirmation for all or selected sub-handshakes (by class, trigger, or assurance level) even where the umbrella grant covers them; policy can only narrow inherited entry. §XVI.1.3, §XVI.4.1, §XVI.13.2 aligned.
v1.33 - 24 August 2026	Consent model refined: explicit consent is mandatory for every umbrella/parent handshake; sub-handshakes whose automation, context, and data use are fully covered by the umbrella's consented grant may be entered without a new consent step (consent inheritance, §XVI.13.2); anything beyond the umbrella grant requires explicit acceptance; a publisher cannot widen a grant via sub-handshakes. §XVI.1.3, §XVI.4.1, §XVI.7.3 aligned.
v1.32 - 24 August 2026	End-to-end sequence (§XVI.1.3) corrected: explicit DNS domain-control verification step added alongside account-holder verification; consent stated as mandatory for every new handshake, with reuse of an already-consented handshake as the only consent-free path. Stage 4 of §XVI.4.1 and §XVI.6.5 aligned.
v1.31 - 24 August 2026	§XVI.1.1 definition reworded so that the check character is unambiguously stated as present in every class, with the class C difference limited to rendering.
v1.30 - 24 August 2026	Class semantics table (§XVI.5.10) and reference-form table (§XVI.5.1) now state the terminal check character explicitly for P/D/I and SP/SI/SC, so that no class reads as unchecked.
v1.29 - 24 August 2026	Text-first capture order for visual composites defined (§XVI.9.4): OCR → parse/check → directory/DNS verification → only then machine-readable payload, reconciled against the verified text. §XVI.11 rewritten: Bootstrap Payload is for out-of-band non-WR scanners only; a conformant runtime never reads it before verification nor follows it afterwards. P15, §XVI.12, and §XVI.16 aligned.
v1.28 - 24 August 2026	Repository links restored and defined (§XVI.4.3): runtime-internal links from completed publisher blocks to the authoritative directory/repository record for user verification; P15 renamed "No external links" and scoped so that runtime-internal repository links remain permitted.
v1.27 - 24 August 2026	Links removed as a transport: deep-link/URL carrier row deleted, in-runtime previews clarified as non-hyperlinks, P15 "No links" added with security-table row; §XVI.11 Bootstrap flagged for revision (still describes a URL payload).
v1.26 - 24 August 2026	Audio (live speech, calls, video sound tracks, broadcasts), on-screen video text, messaging, documents, and clipboard/deep-link added as carriers (§XVI.9.2); wake cue generalized across modalities (spoken activation phrase, Outer Shape, text pattern, NDEF/beacon type, mail/message field type) in §XVI.7.6 and §XVI.5.9; open carrier admission clause §XVI.9.3; P1 and method step 1 widened; summary updated.
v1.25 - 24 August 2026	Opening summary restructured into reference forms / carriers / resolution so that email is presented as one carrier for any class rather than alongside the class definitions; §XVI.7.4 retitled and introduced as a carrier section; §XVI.7.5 step 3 lists email among the carriers.
v1.24 - 24 August 2026	Added §XVI.5.11 "Generalized reference structure" stating the general classification/address/error-detection model and the admissible variations in field count, role, length, check placement, classification encoding, alphabet, algorithm, relationship addressing, and lifecycle binding, so that later profile changes remain within the disclosure; drafting convention and §XVI.17.1 aligned.
v1.23 - 24 August 2026	Class C body defined as the ordered pair initiator/responder Publisher Identifiers with a separately rendered check block (both IDs stay clean six-symbol blocks); one C entry per ordered pair, PENDING → ESTABLISHED under the same identifier; declared local length not applicable to C; mirror-entry option removed; examples and §XVI.7.5 aligned.
v1.22 - 24 August 2026	P consistently defined as the Publisher class. Local block of SP/SI/SC references defined as a single binding identifier that collapses receiving party/parent context and object identifier into one block, expanded only by the resolver (§XVI.2, §XVI.5.10, §XVI.13.2).
v1.21 - 24 August 2026	Removed the separate pairing-reference form (class R) and request selectors (class Q) entirely: every carrier, including email, conveys an ordinary WR Code® Entry reference. Cross-organizational invitations are C-class entries in the initiator's namespace with a PENDING lifecycle state and

Version	Consolidated change
	recipient-bound resolution; the same identifier becomes the ESTABLISHED relationship entry. request_instance_id and the Establishment Commitment are internal protocol values, never carried or checked. Relay demoted to an optional embodiment of the initiator resolver's capsule-holding function. §XVI.1.2, §XVI.5.1, §XVI.5.6, §XVI.5.7, §XVI.6.3, §XVI.7.5, §XVI.8.3, P13, §XVI.16, method step 7, and figure plan revised accordingly.
v1.20 - 24 August 2026	Nonces made end-to-end confidential (encrypted to counterparty directory/principal key, H(nonce) in the clear) so that Relay and anchoring services cannot compute or link EC; Party Identifier generalization (Publisher Identifier or Principal Identifier) so that user-publisher handshakes carry an EC; Principal Key / Delegation Certificate model for signatures on behalf of a publisher; E/Q extension classes and class-R 6k+1 framing registered; directory trust anchor, key rollover, and initial registration stated; multi-organization establishment rule; class-awareness clarification in §XVI.5.4; cross-references corrected.
v1.19 - 24 August 2026	Full rewrite for specification support. Every human-enterable textual reference now begins with a class prefix; pairing references become class R with request-state branch (replaces prefix-less IIIII-RRRRRR-X form, a residue of earlier drafts); participant order made semantic. Check input defined precisely (Class Value Table + address body values; Damm order-32; detected error classes stated). Namespace Directory (§XVI.6.4) and bidirectional publisher key/origin binding (§XVI.6.5) added as the concrete namespace-resolution mechanism; local-field length declared per namespace. Establishment Commitment with two-sided nonces, evidence tiers L/C/X, proof procedure, and no-authority rule added (§XVI.13.6, P14, method step 12). Handshake establishment model summarized (§XVI.13.1) and Relay, Orchestrator, Handshake, Grant, Acceptance defined for self-containedness. BLE rotating-token mapping embodiment added. "Current profile" replaced by "reference profile"/embodiment language.
v1.18 - 24 August 2026	Reorganized the complete annex into a consolidated technical disclosure: positive technical definition first; duplicate review/supersession text removed; parser/check logic centralized; website/email/detection flows consolidated; independent repository verification clarified; handshake/session lifecycle and active umbrella/sub-handshake UI state retained; carrier and security sections de-duplicated; non-limiting implementation language aligned to specification support.
v1.17 - 24 August 2026	Standardized visible textual use of the WR Code® designation.
v1.16 - 24 August 2026	Created the clean editable structure and placed all figures at the end; clarified simultaneous active parent/sub-handshake UI context.
v1.14- v1.15 - 24 August 2026	Added always-on candidate detection with optional auto-insertion, activation without automatic effect, explicit invitation acceptance, handshake/session separation, and persistent active-context visibility.
v1.11- v1.13 - 23 August 2026	Established SP/SI/SC sub-handshake grammar, case-insensitive/transcriber capture, website/email end-to-end state flows, extensible terminal/non-terminal prefix parsing, universal checks for human-enterable references, and separate terminal check block for pairing references.

§XVI.19 Brief Description of the Drawings

Drawing	Content	Supports
FIG. 1	Reference format: prefix block, Publisher Identifier block, local block with terminal check; identical four-block structure for all classes; attached check for P, separate check block for I, C, and sub-handshake classes.	§XVI.5.1, §XVI.5.6
FIG. 2	Prefix Grammar state machine: TERMINAL / NON_TERMINAL / INVALID transitions for P, I, C, S, SP-SE; UI hyphen insertion events.	§XVI.5.2
FIG. 3	Check computation: class values body values → Damm state → check symbol; detected error classes.	§XVI.5.4

Drawing	Content	Supports
FIG. 4	Capture pipeline: heterogeneous sources (text, image/video OCR, audio transcription, email/messaging, documents, radio) with wake cues → Capture Record → normalization → grammar → check → branch selection.	§XVI.7.1, P7
FIG. 5	Namespace Directory: Publisher Identifier → signed record (keys, endpoints, origins, status, generation); DNS and federated embodiments.	§XVI.6.4
FIG. 6	Website discovery sequence with bidirectional origin binding and manifest verification.	§XVI.7.2, §XVI.6.5
FIG. 7	Cross-organizational invitation sequence: PENDING C-entry, capsule, recipient-bound claim, acceptance, commitment computation, transition to ESTABLISHED.	§XVI.7.5
FIG. 8	Handshake/session state diagram showing independence of session termination from handshake lifecycle.	§XVI.13.3
FIG. 9	Trusted UI with simultaneous umbrella and sub-handshake indication; parent-only vs. subordinate-active states.	§XVI.13.4
FIG. 10	Establishment Commitment construction with end-to-end-encrypted nonces, tiers L/C/X, batch Merkle anchoring, and proof procedure.	§XVI.13.6
FIG. 11	Composite carrier deployment with Carrier Binding Registry and divergence alarm.	§XVI.12
FIG. 12	BLE rotating token derivation and resolver-side window matching.	§XVI.14

§XVI.19.1 Reference numerals — FIG. 1

FIG. 1A shows a P-class parent-entry composite (100); FIG. 1B shows an SP-class sub-handshake composite (120). Example references are registry-derived test vectors of Appendix A.2 and verify under §XVI.5.4. Numerals 118 and 128 are optional.

Numeral	Element	Specification
100	P-class composite carrier (parent entry)	§XVI.5.1, §XVI.9.4
102	Outer Shape — region-of-interest and orientation cue for OCR; capture assistance only, no identifier material, no authority	§XVI.7.7, §XVI.9.4
104	Machine-readable element (QR/Data Matrix or similar) — pointer or supplementary payload; decoded only after text-first verification of 108, must agree with the verified text	§XVI.9.4, P12
106	WR Code® designation mark — visual wake cue; with or without colon in text form; not part of the identifier	§XVI.7.7
108	Visible textual reference — the identifier proper; OCR-captured, parsed, check-validated, namespace-verified before 104 is read	§XVI.5, §XVI.9.4
110	Class prefix (one symbol, P) — terminal state of the prefix grammar	§XVI.5.2
112	Publisher Identifier — six symbols; resolves the namespace via the Directory	§XVI.5.3, §XVI.6.4
114	Local entry block — five symbols, publisher-local; no counterparty	§XVI.5.1
116	Check symbol — attached to the local block (unaddressed form); Damm over class values + body	§XVI.5.4
118	Optional companion carriers co-located with the composite (NFC, BLE, touch) — same reference, same verification	§XVI.9.2, §XVI.14
120	SP-class composite carrier (sub-handshake)	§XVI.5.1, §XVI.5.10
122	Two-symbol class prefix (S non-terminal, then P terminal)	§XVI.5.2
124	Combination code — six symbols encoding entry + receiving party; expanded by the	§XVI.5.10

Numeral	Element	Specification
	resolver, not user-interpretable	
126	Check symbol — separate block (addressed form)	§XVI.5.4
128	Optional admitted carrier paths from the original mark (print, display, email, messaging, radio) — validity decided by resolution, never by carrier	§XVI.9.1, §XVI.9.5
130	Displayed Responsible Domain — normalized registrable domain shown above/beside the composite; display text only, cross-checked after resolution	§XVI.2, §XVI.9.4

Appendix A - Registry Material of Grammar Version 2

A.1 Check quasigroup (Damm, order 32)

Definition (§XVI.5.4): symbol values 0–31 are the Crockford Base32 values; $x * y = (\alpha \cdot x) \oplus y$ in $GF(2^5)$, reduction polynomial $x^5 + x^2 + 1$, $\alpha = 2$. Properties verified exhaustively: quasigroup (every row and column a permutation, all 32^2 pairs), and total anti-symmetry in both conditions — $(c * x) * y \neq (c * y) * x$ for all c and all $x \neq y$ (all 32^3 triples), and $x * y \neq y * x$ for all $x \neq y$ (all 32^2 pairs). The diagonal is $x * x = (\alpha \oplus 1) \cdot x$, not zero; accordingly the check symbol is $k = \alpha \cdot \text{interim}$, the unique solution of $\text{interim} * k = 0$, and verification remains "interim over $v \parallel [k]$ equals 0". Row = left operand (current interim), column = right operand (next value). The table is shown in reference-alphabet symbols.

*		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	G	H	J	K	M	N	P	Q	R	S	T	V	W	X	Y	Z
0		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	G	H	J	K	M	N	P	Q	R	S	T	V	W	X	Y	Z
1		2	3	0	1	6	7	4	5	A	B	8	9	E	F	C	D	J	K	G	H	P	Q	M	N	T	V	R	S	Y	Z	W	X
2		4	5	6	7	0	1	2	3	C	D	E	F	8	9	A	B	M	N	P	Q	G	H	J	K	W	X	Y	Z	R	S	T	V
3		6	7	4	5	2	3	0	1	E	F	C	D	A	B	8	9	P	Q	M	N	J	K	G	H	Y	Z	W	X	T	V	R	S
4		8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7	R	S	T	V	W	X	Y	Z	G	H	J	K	M	N	P	Q
5		A	B	8	9	E	F	C	D	2	3	0	1	6	7	4	5	T	V	R	S	Y	Z	W	X	J	K	G	H	P	Q	M	N
6		C	D	E	F	8	9	A	B	4	5	6	7	0	1	2	3	W	X	Y	Z	R	S	T	V	M	N	P	Q	G	H	J	K
7		E	F	C	D	A	B	8	9	6	7	4	5	2	3	0	1	Y	Z	W	X	T	V	R	S	P	Q	M	N	J	K	G	H
8		G	H	J	K	M	N	P	Q	R	S	T	V	W	X	Y	Z	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
9		J	K	G	H	P	Q	M	N	T	V	R	S	Y	Z	W	X	2	3	0	1	6	7	4	5	A	B	8	9	E	F	C	D
A		M	N	P	Q	G	H	J	K	W	X	Y	Z	R	S	T	V	4	5	6	7	0	1	2	3	C	D	E	F	8	9	A	B
B		P	Q	M	N	J	K	G	H	Y	Z	W	X	T	V	R	S	6	7	4	5	2	3	0	1	E	F	C	D	A	B	8	9
C		R	S	T	V	W	X	Y	Z	G	H	J	K	M	N	P	Q	8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7
D		T	V	R	S	Y	Z	W	X	J	K	G	H	P	Q	M	N	A	B	8	9	E	F	C	D	2	3	0	1	6	7	4	5
E		W	X	Y	Z	R	S	T	V	M	N	P	Q	G	H	J	K	C	D	E	F	8	9	A	B	4	5	6	7	0	1	2	3
F		Y	Z	W	X	T	V	R	S	P	Q	M	N	J	K	G	H	E	F	C	D	A	B	8	9	6	7	4	5	2	3	0	1
G		5	4	7	6	1	0	3	2	D	C	F	E	9	8	B	A	N	M	Q	P	H	G	K	J	X	W	Z	Y	S	R	V	T
H		7	6	5	4	3	2	1	0	F	E	D	C	B	A	9	8	Q	P	N	M	K	J	H	G	Z	Y	X	W	V	T	S	R
J		1	0	3	2	5	4	7	6	9	8	B	A	D	C	F	E	H	G	K	J	N	M	Q	P	S	R	V	T	X	W	Z	Y
K		3	2	1	0	7	6	5	4	B	A	9	8	F	E	D	C	K	J	H	G	Q	P	N	M	V	T	S	R	Z	Y	X	W
M		D	C	F	E	9	8	B	A	5	4	7	6	1	0	3	2	X	W	Z	Y	S	R	V	T	N	M	Q	P	H	G	K	J
N		F	E	D	C	B	A	9	8	7	6	5	4	3	2	1	0	Z	Y	X	W	V	T	S	R	Q	P	N	M	K	J	H	G

P | 9 8 B A D C F E 1 0 3 2 5 4 7 6 S R V T X W Z Y H G K J N M Q P
Q | B A 9 8 F E D C 3 2 1 0 7 6 5 4 V T S R Z Y X W K J H G Q P N M
R | N M Q P H G K J X W Z Y S R V T 5 4 7 6 1 0 3 2 D C F E 9 8 B A
S | Q P N M K J H G Z Y X W V T S R 7 6 5 4 3 2 1 0 F E D C B A 9 8
T | H G K J N M Q P S R V T X W Z Y 1 0 3 2 5 4 7 6 9 8 B A D C F E
V | K J H G Q P N M V T S R Z Y X W 3 2 1 0 7 6 5 4 B A 9 8 F E D C
W | X W Z Y S R V T N M Q P H G K J D C F E 9 8 B A 5 4 7 6 1 0 3 2
X | Z Y X W V T S R Q P N M K J H G F E D C B A 9 8 7 6 5 4 3 2 1 0
Y | S R V T X W Z Y H G K J N M Q P 9 8 B A D C F E 1 0 3 2 5 4 7 6
Z | V T S R Z Y X W K J H G Q P N M B A 9 8 F E D C 3 2 1 0 7 6 5 4

Reference implementation (normative behaviour, illustrative code):

```
ALPHABET = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"

def mul2(x): x <= 1; return (x ^ 0x25) & 0x1F if x & 0x20 else x

def op(x, y): return mul2(x) ^ y

def interim(values): st = 0; for d in values: st = op(st, d); return st

# v = class values || body values

check = ALPHABET[mul2(interim(v))]

valid = interim(v + [value(check)]) == 0
```

A.2 Test vectors

Check input v = class values of the consumed terminal prefix (Class Value Table, A.3) followed by the Crockford values of the normalized body; presentation separators and the check symbol are excluded. Positive vectors (one per class):

Positive vectors (one per class):

Class	Reference	Check input v (integer values)	Interim after v	Check
P	P-WR7X4K-9B2M3C	[22, 28, 24, 7, 29, 4, 19, 9, 11, 2, 20, 3]	6	C
I	I-K4T9M2-7QFA3X-S	[1, 19, 4, 26, 9, 20, 2, 7, 23, 15, 10, 3, 29]	30	S
C	C-ABC123-XYZ789-H	[12, 10, 11, 12, 1, 2, 3, 29, 30, 31, 7, 8, 9]	26	H
SP	SP-WR7X4K-H2N5V8-7	[25, 22, 28, 24, 7, 29, 4, 19, 17, 2, 21, 5, 27, 8]	17	7
SI	SI-K4T9M2-3ZDQ7E-H	[25, 1, 19, 4, 26, 9, 20, 2, 3, 31, 13, 23, 7, 14]	26	H
SC	SC-ABC123-M8W2R4-0	[25, 12, 10, 11, 12, 1, 2, 3, 20, 8, 28, 2, 24, 4]	0	0
SE	SE-WR7X4K-6TCJ9F-P	[25, 14, 28, 24, 7, 29, 4, 19, 6, 26, 12, 18, 9, 15]	22	P

Negative vectors (each must fail local verification):

Case	Candidate	Verifies
Single substitution ($X \rightarrow Z$ at body position 4)	P-WR7Z4K-9B2M3C	no
Adjacent transposition (9B $\rightarrow$ B9)	P-WR7X4K-B92M3C	no
Prefix substitution (SP body presented as P)	P-WR7X4K-H2N5V8-7	no

Case	Candidate	Verifies
Transposed Publisher Identifiers in a C reference	C-XYZ789-ABC123-H	no
Check symbol altered	C-ABC123-XYZ789-J	no
Alias normalization (lower case, presentation separators, body alias l for 1)	p-wr7x4k-9b2m3c, PWR7X4K9B2M3C, C-ABC123-XYZ789-H	yes

A.3 Class Value Table and DOMAIN_TAG

Class symbol	Value
P	22
I	1
C	12
S	25
E (second symbol of SE only)	14

Class value sequences: P [22]; I [1]; C [12]; SP [25, 22]; SI [25, 1]; SC [25, 12]; SE [25, 14]. The values coincide with the Crockford input values of the class symbols, so the computation equals running the algorithm over the complete alias-normalized string; the prefix is parsed before alias normalization (§XVI.5.2).

DOMAIN_TAG for the Establishment Commitment (§XVI.13.6): the 24-byte ASCII string OPTIRANDO-WRC-EC-v1\x00\x00\x00\x00\x00 (the 19-character tag OPTIRANDO-WRC-EC-v1 followed by five zero bytes to a fixed 24-byte width). commitment_version is the single byte 0x01. All concatenated fields are length-prefixed (two-byte big-endian length) so that the encoding is injective.

A.4 Field schemas

A.4.1 Namespace Directory Record (§XVI.6.4; dual-signed).

Field	Type	Description
publisher_id	6 symbols	Publisher Identifier
generation	uint	Incremented on every change; cache validity key
status	enum	ACTIVE, INACTIVE, REVOKED, SUPERSEDED, COMPROMISED
grammar_version	uint	Grammar version the namespace issues
verification_keys[]	key, generation, valid_from, valid_to	Publisher signing keys (current and retained predecessors)
encryption_keys[]	key, generation, valid_from, valid_to	Publisher encryption keys; predecessors retained for decryption of historical nonces
resolver_endpoints[]	URI	Resolver endpoints for the namespace
relay_endpoints[]	URI	Relay endpoints for capsule and acceptance delivery
origins[]	domain / origin	DNS-verified domains and web origins
display_origin	domain	The Displayed Responsible Domain (§XVI.2): normalized registrable base domain shown on visual composites; must correspond to one of origins[]
connector	script_version, code_hash	Registered WR Connect script version and, if customized, connector code hash (§XVI.7.4a)
verified_holder	opaque	Operator attestation of account-holder verification
expiry	timestamp	Upper bound on cache life
operator_signature	signature	Directory operator (pinned trust anchor)
publisher_signature	signature	Publisher countersignature

A.4.2 Delegation Certificate (§XVI.2, §XVI.6.5; publisher-signed).

Field	Type	Description
publisher_id	6 symbols	Delegating publisher
principal_key	public key	Principal Key being certified
principal_email	string	SSO-verified email address; domain must be one of the publisher's DNS-verified domains
device_scoped_id	fingerprint, optional	Present for a device-bound Principal Key (§XVI.13.7)
scope[]	enum	initiate, accept, revoke
generation	uint	Certificate generation
valid_from / valid_to	timestamp	Validity window
publisher_key_generation	uint	Generation of the signing publisher key
signature	signature	Publisher signature

A.4.3 Pending Handshake-Request Capsule (§XVI.2, §XVI.7.5; initiator-signed, recipient-bound).

Field	Type	Description
request_instance_id	128-bit	Idempotency identifier, never carried on a reference
entry_reference	reference	The C (or other) entry the capsule is attached to
initiator_party	Publisher or Principal Identifier	Initiating party
responder_parties[]	Publisher or Principal Identifier	Named responders, in order
requested_profile	opaque	Requested scope, grant profile, use-limitation options (§XVI.8.4)
nonce_I_enc[]	ciphertext per responder	Initiator nonce encrypted to each responder's encryption key
nonce_I_commitment	hash	H(nonce_I) in the clear
issued_at / expires_at	timestamp	Freshness and expiry
delegation_certificate	by value or reference	Where signed by a delegated principal
signature	signature	Initiator signature

A.4.4 Acceptance Record (§XVI.7.5, §XVI.13.5; responder-signed).

Field	Type	Description
request_instance_id	128-bit	Matches the capsule
capsule_hash	hash	H(capsule)
responder_party	Publisher or Principal Identifier	Accepting party
accepting_device	fingerprint, optional	Device-Scoped Principal Identifier of the accepting device (§XVI.13.7)
device_pass	Device Record + proof, optional	Tenant-signed Device Record presented for registration as a Registered Counterpart Device, with key-possession proof (§XVI.13.7 Device Pass)
accepted_grant	opaque	Grant as accepted (may narrow the requested profile)
nonce_R_enc	ciphertext	Responder nonce encrypted to the initiator

Field	Type	Description
nonce_R_commitment	hash	H(nonce_R) in the clear
accepted_at	timestamp	Acceptance time
delegation_certificate	by value or reference	Where signed by a delegated principal
signature	signature	Responder signature

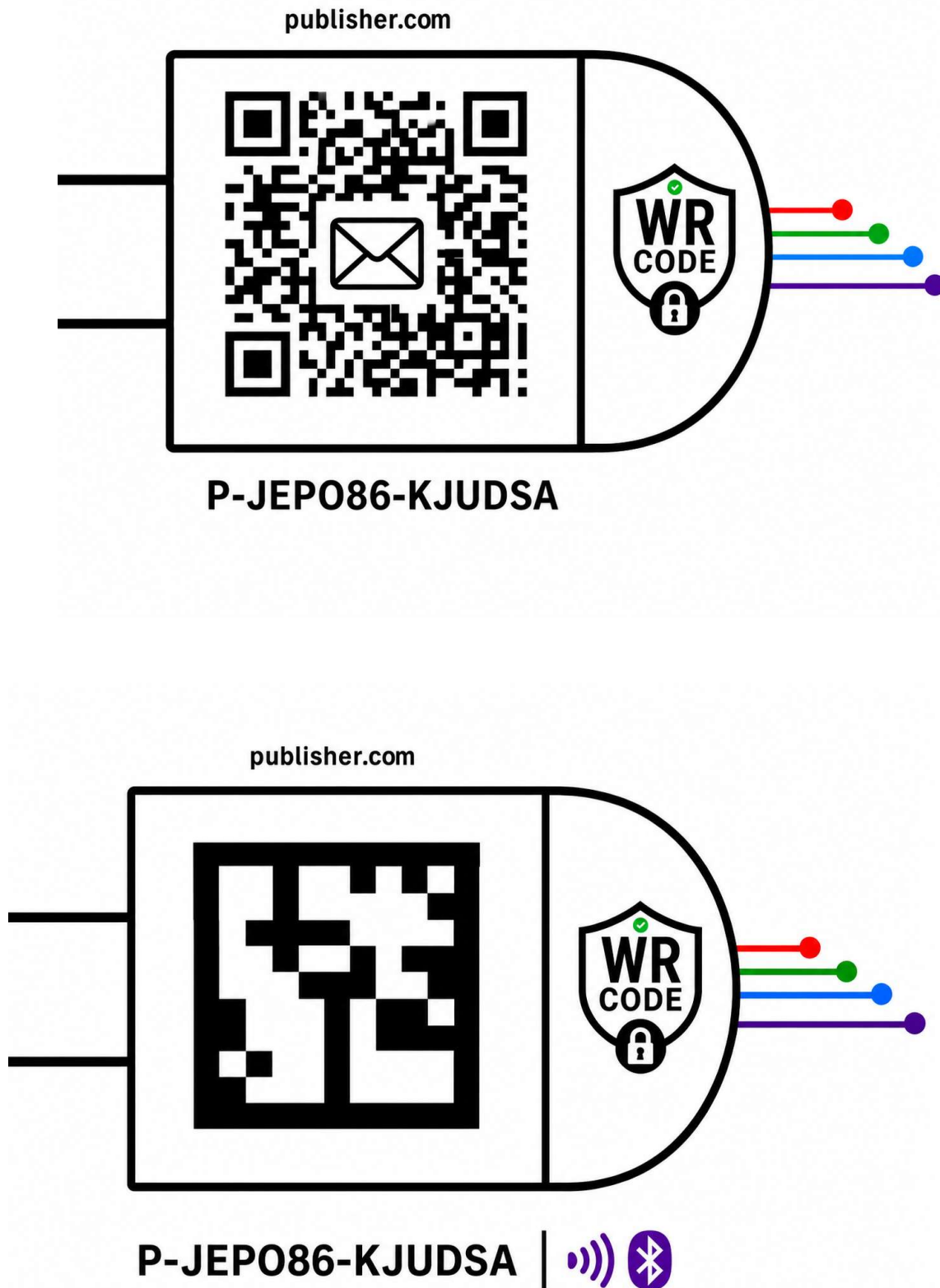
A.4.5 Device Record (§XVI.2, §XVI.13.7; signed by the tenant or publisher under the pairing umbrella).

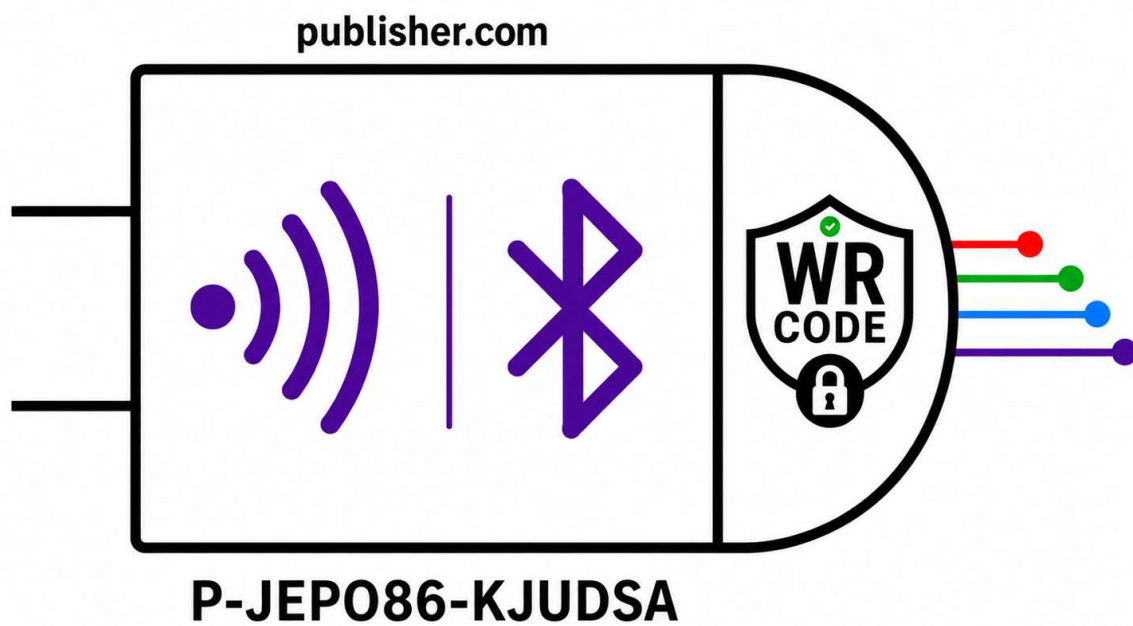
Field	Type	Description
principal_email	string	Owning principal (SSO Identity)
device_name	string	Unique within the principal
device_class	enum	Registered Device Class
device_scoped_id	fingerprint	Fingerprint of the device's Principal Key
status	enum	ACTIVE, REVOKED
generation	uint	Incremented on rename or status change
paired_at	timestamp	Result of the I (or P) pairing handshake
signature	signature	Issuer signature

License Notice. This Annex forms an integral part of the Optirando™ specification and is subject to the same governance and license terms as the specification's other components, including BEAP™, WR Desk™, and PoAE™. Those terms are restated below and apply to this Annex without modification.

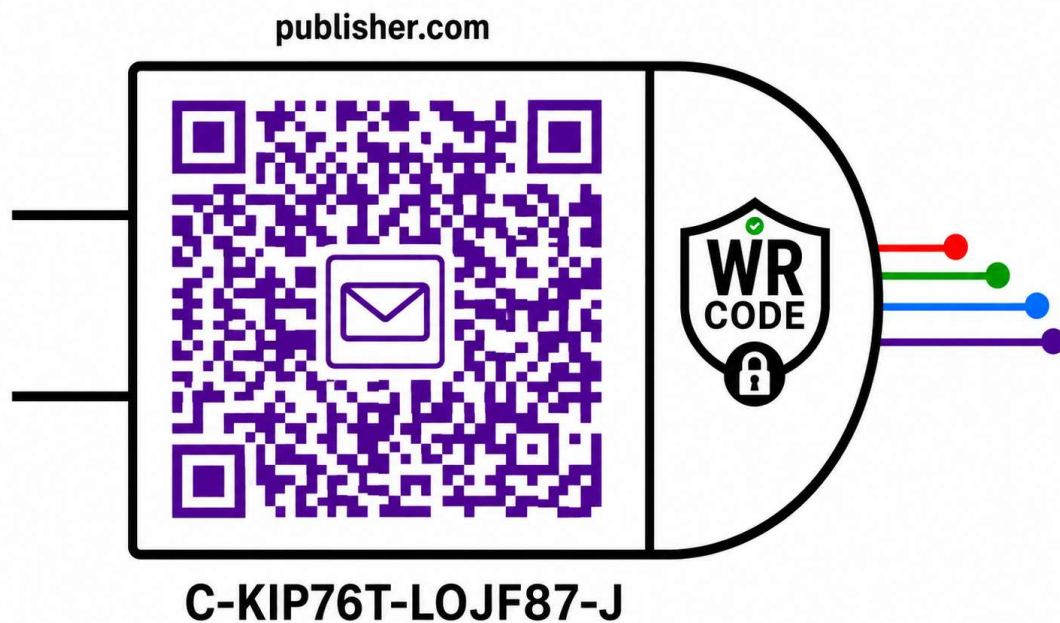
Copyright © 2026 Oscar Schreyer. All rights reserved.

Illustrative Representation of a WR Code® for Initiating a Public Handshake

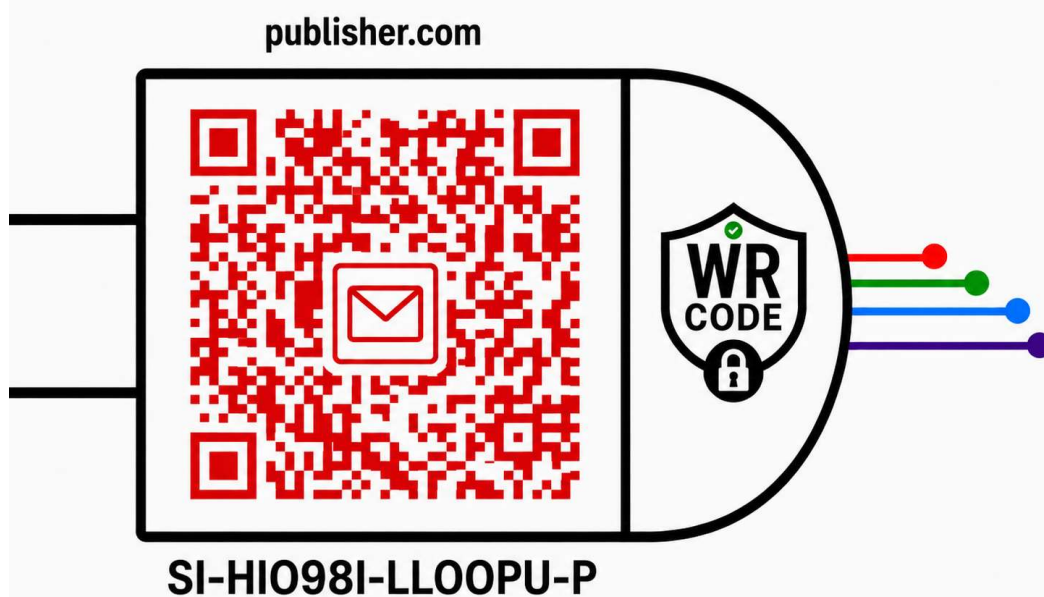




Cross-Org Handshake and Sub-Handshake Initiator (color coded purple)



Initiator Types for Internal Handshakes and Sub-Handshakes (color coded red)



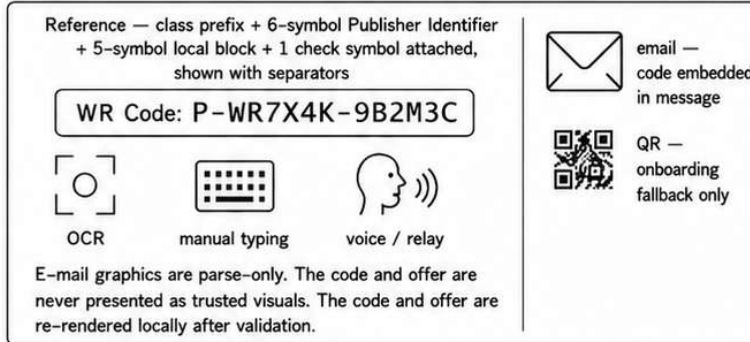
WR Code® — Public Handshake via Email

Example path: e-mail-based verification. Other public entry routes exist and converge into the same trust and consent model.

Scope of this figure

This figure explains the e-mail-based public handshake path. It does not represent all public entry paths. Other valid entry routes include visiting a verified website, scanning a WR Code® via OCR/manual/voice, and combined carrier paths such as QR + OCR, NFC, AprilTag, or BLE token.

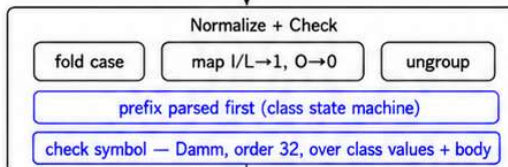
1 Email Capture



Other public entry paths (not expanded here)

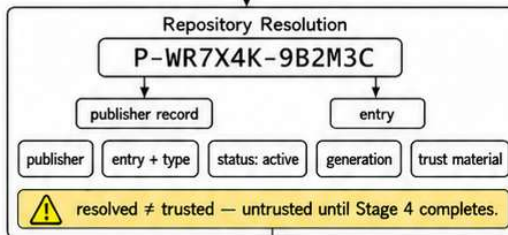


2 Local Validation

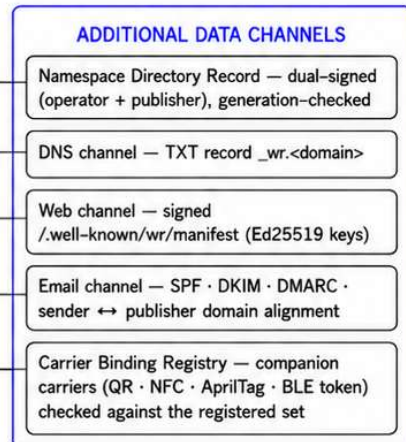


Check fails — capture error — resolution never triggered.

3 Repository Resolution



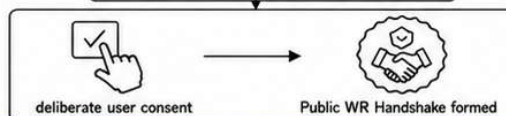
4 Trust Validation



5 Validated Offer



6 Consent → Handshake

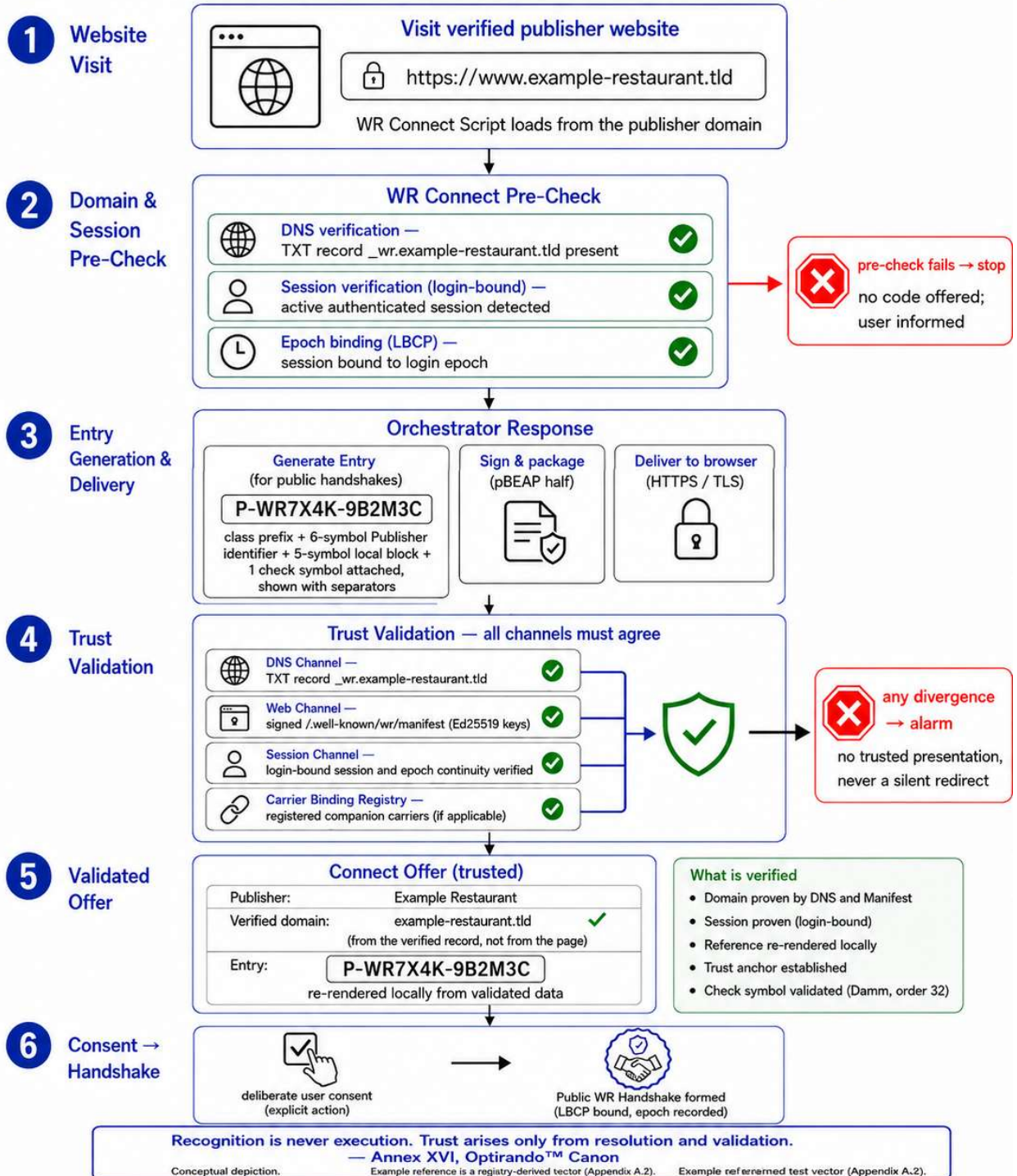


Recognition is never execution. This figure shows the e-mail path for a public handshake. Other entry paths exist, but trust still arises only from resolution and validation.
Annex XVI, OptirandoSM Canon

Conceptual depiction. Example reference is a registry-derived test vector (Appendix A.2).

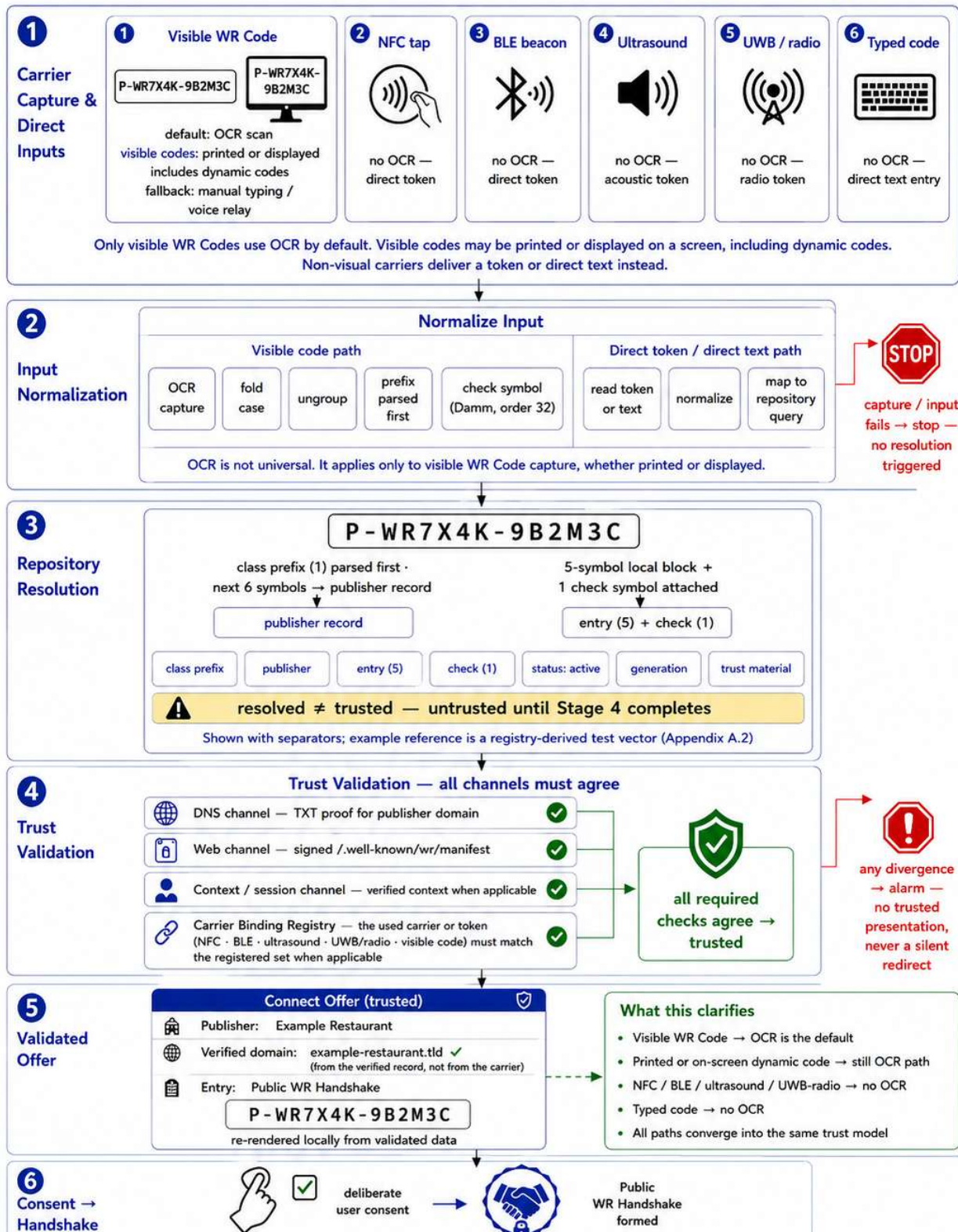
WR Code® — Public Handshake via Website Visit

Website path only — focused depiction of the verified web entry route.



WR Code® — Combined Carrier Paths

One identifier, multiple carriers — OCR is the default only for visible WR Codes, including printed and on-screen dynamic codes.



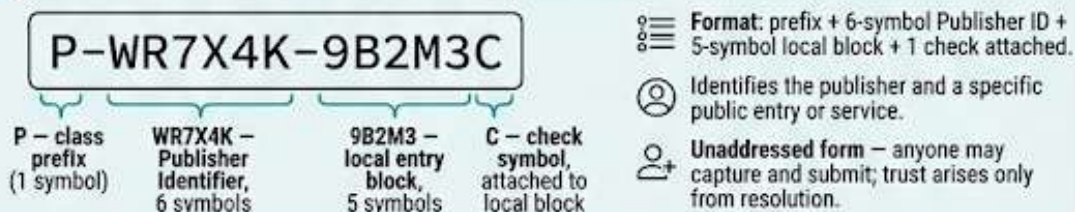
Recognition is never execution. Carriers transport identifiers or tokens — trust arises only from resolution and validation. — Annex XVI, Optirando™ Canon
Conceptual depiction. Example reference is a registry-derived test vector (Appendix A.2).

WR Code® — Handshake Initiation Types

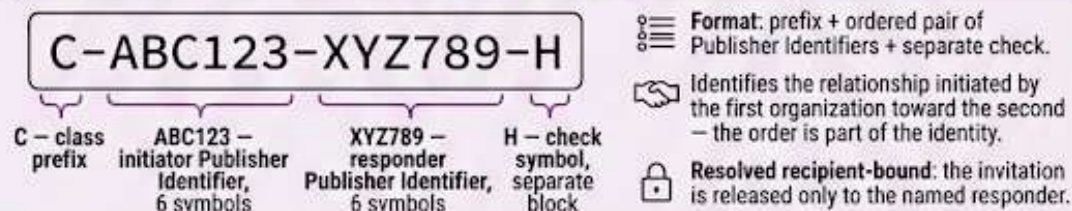
Three parent handshake classes, one grammar — **class prefix**, 6-symbol **Publisher Identifier**, third block, **check symbol**.

i Every reference starts with a class prefix and ends with a check symbol. **P** attaches the check to the local block; all other classes carry it as a separate block.

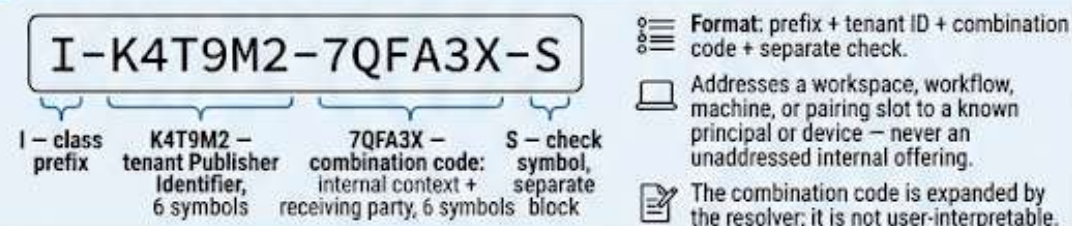
1 PUBLIC WR CODE® — class P. For publisher-to-endpoint public handshakes.



2 CROSS-ORG WR CODE® — class C. Between different organizations.



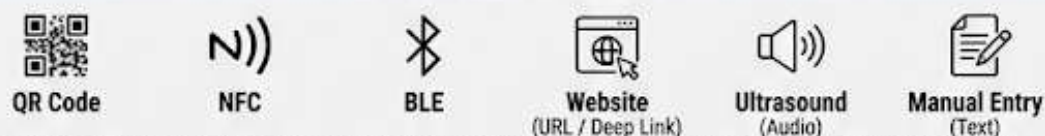
3 INTERNAL WR CODE® — class I. Within the same organization / tenant.



SUB-HANDSHAKE CLASSES

Beneath an established parent handshake, sub-handshakes use two-symbol prefixes: **SP · SI · SC · SE** — same structure, 6-symbol combination code, separate check. Example: **SP-WR7X4K-H2N5V8-7**.

CARRIERS / TRANSPORTS



The WR Code® is carried by these transports. The reference itself stays the same — validity is decided by resolution, never by the carrier.

AFTER CONNECTION

- 1** **WR Code® entered or scanned** — user or system enters or scans via any supported carrier.
- 2** **Connection established and verified** — authenticated resolution and validation succeed.
- 3** **Establishment Commitment created** — a private cryptographic commitment binds both parties' contributions to this handshake.
- 4** **Optional ledger anchoring** — in high-assurance environments the commitment may be anchored on a ledger in a privacy-preserving way (optional).

i The Establishment Commitment is not the same thing as the public-facing WR Code®. It is private and not publicly shown.

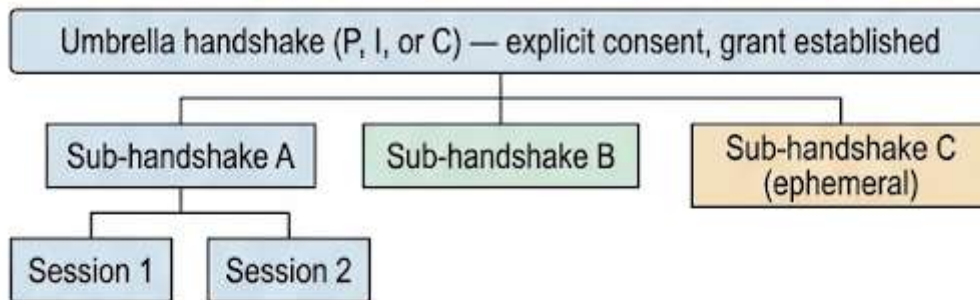
IMPORTANT NOTES

- Every class carries a check symbol (Damm, order 32, over class values + body).
- **P** attaches the check to the local block; **C**, **I**, and all sub-handshake classes render it as a separate block.
- This infographic shows public-facing references only; Establishment Commitments are private and not shown.
- Example references are registry-derived test vectors (Appendix A.2). — **Annex XVI, Optirando™ Canon**

WR Code[®] — Sub-Handshake Classes

Work happens beneath an umbrella — four sub-handshake classes, one grammar.

1 — The hierarchy: umbrella → sub-handshake → session



Sub-handshakes inherit their trust from the umbrella; sessions run beneath sub-handshakes. Ending a session does not end the sub-handshake; ending a sub-handshake does not end the umbrella.

2 — The four classes

SP — beneath a publisher umbrella

SP-WR7X4K-H2N5V8-7

- Publisher offers a specific object, workflow, or capability to a known principal
- Receiving party: the principal — any of their paired devices; the accepting device is recorded
- Typical uses: document handover, tutoring module, support workflow

SI — beneath an internal umbrella

SI-K4T9M2-3ZDQ7E-H

- Tenant-internal delivery: workspace, workflow, machine, or resource to a principal
- Only class with device selection: the tenant may bind one named device from its device list
- Any other device answers: NOT_FOR_THIS_DEVICE

SC — beneath a cross-org umbrella

SC-ABC123-M8W2R4-0

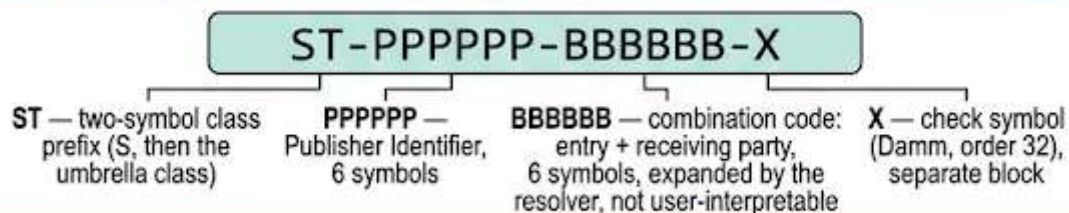
- Addressed to the responder organization — never to a foreign principal or device
- Which principal or device handles it is the responder's own role management, by policy
- Exception: a device registered by Device Pass may be bound directly (high assurance)

SE — ephemeral, session-scoped

SE-WR7X4K-6TCJ9F-P

- Short-lived sub-handshake for one bounded interaction or hand-off
- Expires with its session; leaves no standing relationship
- Same grammar, same gates, same consent rules as every other class

3 — One grammar for all four



4 — Shared rules

- ✓ An issuer can bind only what its repository already knows (known-identifier rule)
- ✓ Consent: policy decides — a sub-handshake fully covered by the umbrella's consented grant may proceed without a new consent step, or policy may require fresh consent
- ✓ Every resolution passes the same gates as a parent handshake — no shortcuts beneath an umbrella
- ✓ A sub-handshake never widens the umbrella's grant
- ✓ Several sub-handshakes may be active in parallel and can be docked and undocked as context requires

Sub-handshakes are where governed work happens — scoped, revocable, and always beneath a consented umbrella. — Annex XVI, Optirando[™] Canon

Conceptual depiction. Example references are registry-derived test vectors (Appendix A.2).

WR Code® – Convergent Trust Across Transport Media

One trust model. Many carriers. Same trusted outcome.

1 – Untrusted carriers – trusted outcome

A Multiple ways in. Not trusted by itself.

1	 Visible WR Code – printed or displayed	6	 Ultrasound
2	 QR code – optional companion	7	 UWB / radio
3	 AprilTag / fiducial – optional companion	8	 Website visit
4	 NFC tap – optional companion	9	 E-mail path
5	 BLE beacon	10	 Typed code

Each of these is an entry or token – never trusted by itself.

B WRC – Workflow Ready Cloud™ by Silicon Cargo

- 1 Publisher trust anchor
- 2 Namespace Directory Record (dual-signed)
- 3 Carrier Binding Registry
- 4 Repository resolution
- 5 Manifest / key checks
- 6 Channel validation

The transport medium is not trusted by itself. It becomes trustworthy only after WRC-based resolution and validation.

C Trusted local connect offer



Context-aware, policy-controlled re-presentation – re-rendered locally from validated data.

P-WR7X4K-9B2M3C

OCR is the default capture method for Visible WR Codes only.

2 – Cryptographic attachment to verified publisher identity



Visible WR Code · QR code · AprilTag
NFC tap · BLE beacon · Website visit



Verified publisher identity

- verified domain / trust anchor
- cryptographic binding
- repository-backed resolution
- carrier association can be registered



One identity across many transport methods

Known carriers gain a stronger trust model when attached to a cryptographically verifiable publisher identity through WRC.

3 – Companion carriers and convergent verification

Example combinations (illustrative only)

- 1 Website visit + QR code
- 2 E-mail path + QR code
- 3 Visible WR Code + AprilTag
- 4 QR code + NFC tap + BLE beacon
- 5 AprilTag + NFC tap + BLE beacon

Convergent validation – all required channels must agree

- 1 DNS / publisher proof
- 2 Web manifest / keys
- 3 Email sender alignment (when applicable)
- 4 Session / epoch (when applicable)
- 5 Carrier Binding Registry
- 6 Repository status

If channels diverge:
alert – no trusted presentation – never a silent redirect.

Different carriers can reinforce one another. Trust comes from convergence, not from any single presentation layer.

4 – Best carrier for the situation

1	 Visible WR Code – human-readable, printable / displayed	6	 Ultrasound – local acoustic signals
2	 QR code – compact machine-readable companion carrier	7	 UWB / radio – precise local ranging
3	 AprilTag / fiducial – pose / spatial alignment	8	 Website visit – broad public entry surfaces
4	 NFC tap – intentional near-field tap	9	 E-mail path – broad public entry surfaces
5	 BLE beacon – proximity and discovery	10	 Typed code – manual entry when no other carrier is available

5 – Trusted local re-presentation after validation



After handshake: constrained p2p BEAP™ channel

- Allowed (only): real-time text information
- text plus normalized public media from WRC / repository
- Not allowed: arbitrary binaries
- unbounded media exchange · automatic execution

The user does not rely on the raw presented carrier. The user relies on the validated local re-presentation.

6 – Core architectural strengths

- ✓ One trust model across heterogeneous entry paths
- ✓ Cryptographically verifiable publisher attachment
- ✓ Multi-carrier combinations improve verifiability and context
- ✓ Validation is based on convergence, not raw presentation
- ✓ Same architecture across QR, visual, website, NFC, BLE, AprilTag, ultrasound, UWB, radio
- ✓ Sensitive actions can later be authorized separately if needed

The carrier transports the entry. WRC resolves identity and context. Convergent validation establishes trust. The user receives a trusted local offer. – Annex XVI, Optirando™ Canon

Conceptual depiction. Example reference is a registry-derived test vector (Appendix A 2).

WR Code® – Public Handshake via Website Visit

Website path **only** – focused depiction of the verified web entry route.

1 Website Visit



<https://www.example-restaurant.tld>

WR Connect Script loads from the publisher domain

2 Domain & Session Pre-Check

WR Connect Pre-Check

- ✓ DNS verification – TXT record `_wr.example-restaurant.tld` present
- ✓ Session verification (login-bound) – active authenticated session detected
- ✓ Epoch binding (LBCP) – session bound to login epoch

pre-check fails → **stop**
– no code offered;
user informed

3 Entry Generation & Delivery

Orchestrator Response

Generate Entry (for public handshakes)

P-WR7X4K-9B2M3C

class prefix + 6-symbol Publisher Identifier +
5-symbol local block + 1 check symbol
attached, shown with separators

Sign & package (pBEAP half)



Deliver to browser (HTTPS / TLS)



4 Trust Validation

Trust Validation – all channels must agree

- ✓ DNS Channel – TXT record `_wr.example-restaurant.tld`
- ✓ Web Channel – signed `/.well-known/wr/manifest` (Ed25519 keys)
- ✓ Session Channel – login-bound session and epoch continuity verified
- ✓ Carrier Binding Registry – registered companion carriers (if applicable)

any divergence → **alarm**
– no trusted presentation,
never a silent redirect



5 Validated Offer

Connect Offer (trusted)

Publisher: Example Restaurant

Verified domain: `example-restaurant.tld` ✓
(from the verified record, not from the page)

Entry: Public WR Handshake

P-WR7X4K-9B2M3C

re-rendered locally from validated data

What is verified

- Domain proven by DNS and Manifest
- Session proven (login-bound)
- Reference re-rendered locally
- Trust anchor established
- Check symbol validated (Damm, order 32)

6 Consent → Handshake



deliberate user consent
(explicit action)



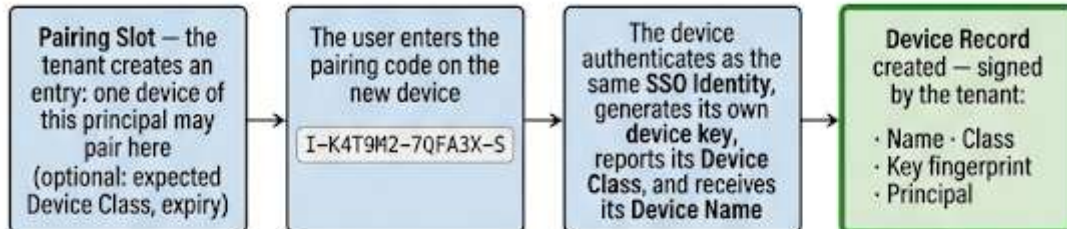
Public WR Handshake formed
(LBCP bound, epoch recorded)

Recognition is never execution. Trust arises only from resolution and validation.
– Annex XVI, Optirando™ Canon

WR Code® — The Device Model

How devices become known, named, selectable, and — where needed — the only permitted endpoint.

1 — Pairing: how a device is born



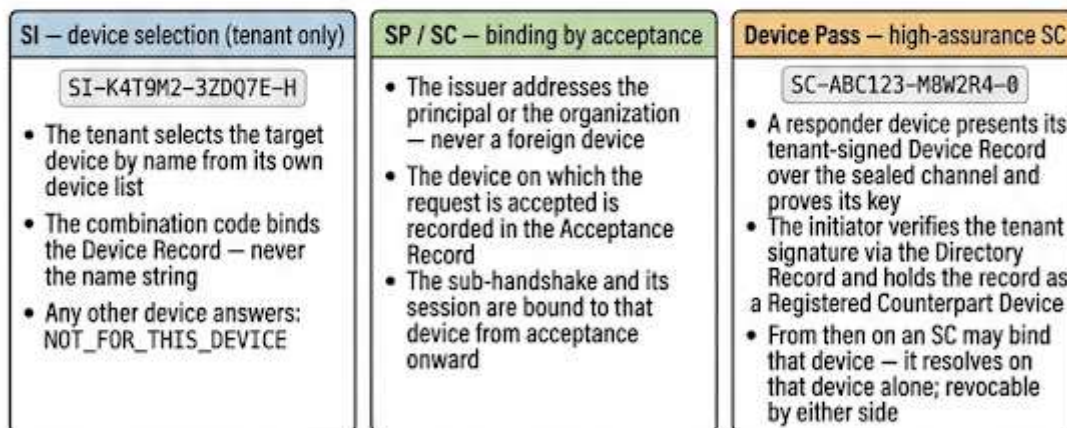
The issuer knows nothing about the device before pairing — the Device Record is the result of the pairing, never an input to the code. The slot is single-use.

2 — The Device Record

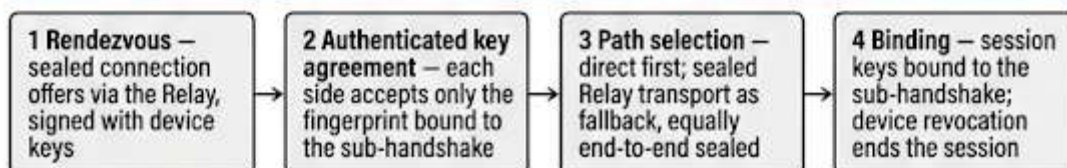
Device Name: laptop-2 (unique within the principal)
Device Class: workstation
Key fingerprint: Device-Scoped Principal Identifier
Principal: the owning SSO Identity
Status · Generation: revocable, renaming never breaks issued references

Signed by the tenant under the pairing umbrella — every holder of the umbrella can verify that a name resolves to exactly one key.

3 — Reaching a device



4 — Peer-to-peer session between two named devices



A device is addressable only where a signed **Device Record exists** — created by pairing, presented by the device itself, never asserted by a stranger. — Annex XVI, Optirando™ Canon

"WR Code® — Anatomy of a Grant

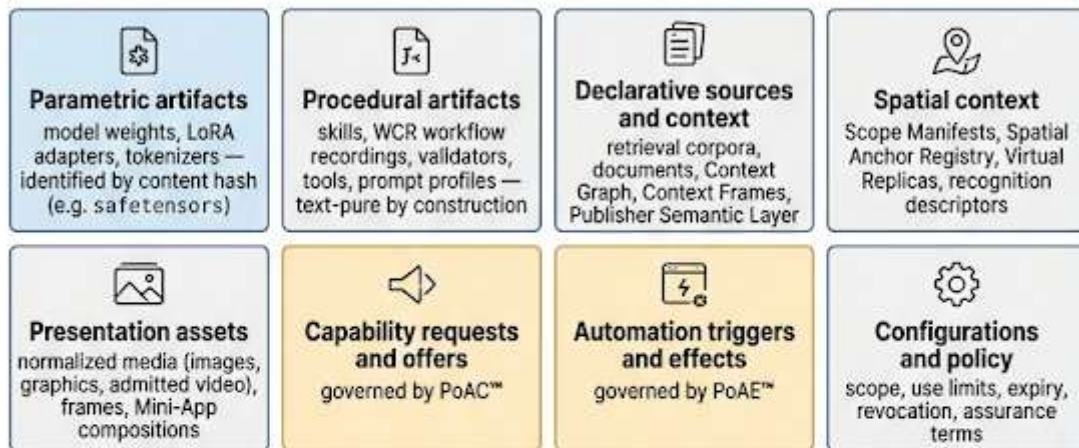
The grant is the concrete content of a handshake — enumerated, consented, hash-anchored.

1 — Where the grant lives



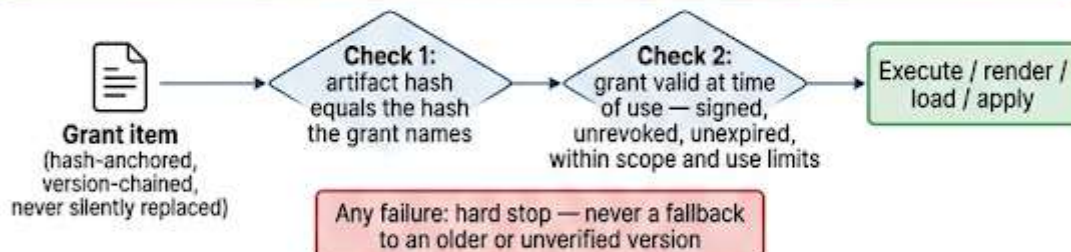
Established at acceptance, narrowed by the responder or by policy, inherited and refined by sub-handshakes.

2 — What a grant can contain



The inventory is illustrative — any data that can be transported securely under the specification's invariants may be a grant item.

3 — Nothing runs before two checks pass



Both checks run locally on the consuming device.

4 — Two proof regimes

PoAE™ — Proof of Authoritative Execution

- Effects execute only after human authorization of the grade the grant demands
- Authorization, artifact hashes, instructions, and outcome are recorded as cryptographic proof
- AI prepares; a human authorizes; the system records

PoAC™ — Proof of Authoritative Capability

- A capability enters an orchestrator only under a grant, backed by evidence of need
- Propose, never silently apply — provenance (hash, issuer, grant) is verified first
- Applied only after authorization

5 — Transport and commerce

- ✓ Every grant item, proof, request, and release travels as a **BEAP™ capsule** beneath the handshake — sealed, recipient-bound, verified on every read path
- ✓ Nothing outside the grant is released at all; no side channels
- ✓ Paid offerings — pro libraries, pro adapters, subscription tiers — are distributed through the **WR Store™** and remain ordinary hash-anchored grant items

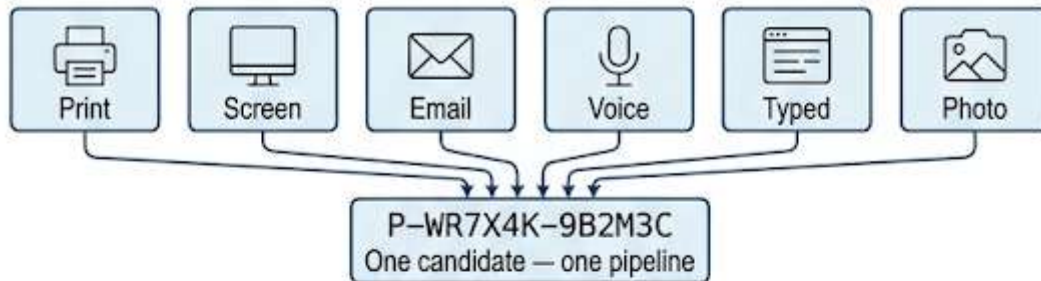
The reference points, the handshake trusts, the grant delivers — exactly what was consented, exactly as hashed, and nothing else. — Annex XVI, Optirando™ Canon

Conceptual depiction.

WR Code® — The Six Gates

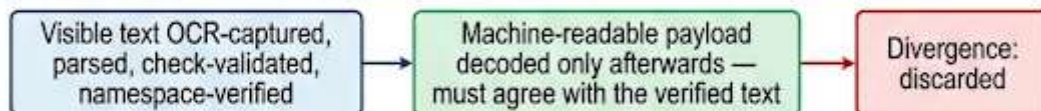
One cryptographic resolution for every carrier — six hard stops between capture and the user.

1 — Every capture enters the same pipeline

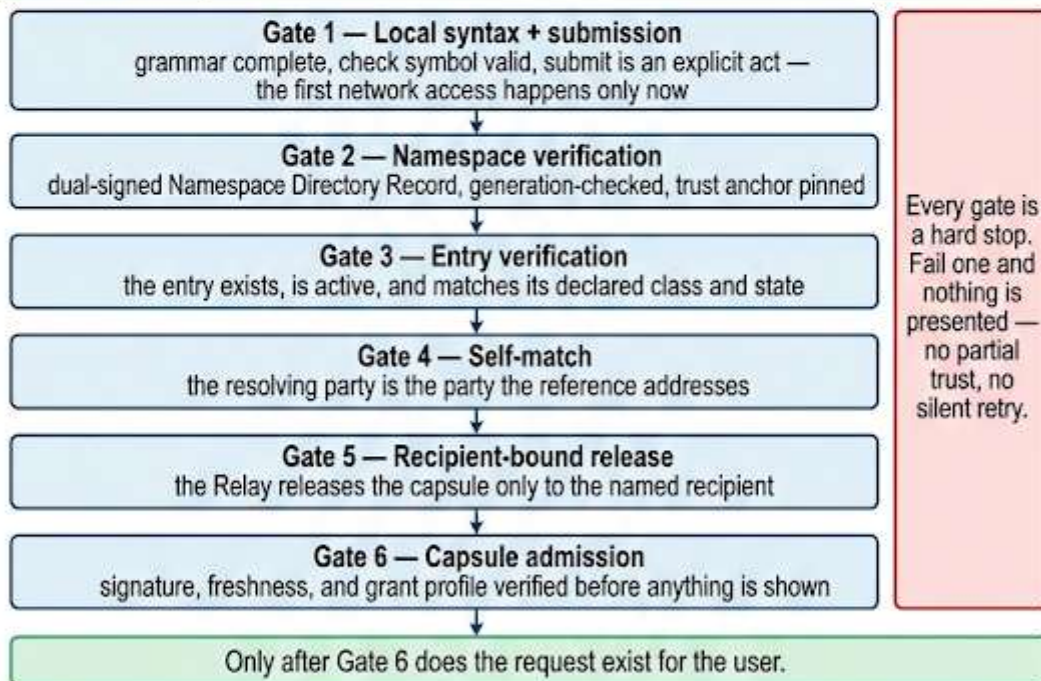


The carrier never matters. However a reference was captured, it is resolved in exactly the same cryptographic way — there are no privileged carriers and no shortcut paths.

2 — Text first



3 — The six gates



4 — What this defeats

- ✓ A forged or tampered reference fails locally at the check symbol — before any network access
- ✓ A fake namespace or fake entry fails at Gates 2 and 3 against the signed Directory Record
- ✓ Misdelivery is impossible by construction: recipient-bound release gives the wrong party nothing
- ✓ A manipulated machine-readable payload can never override the verified text

Recognition is never execution. Capture opens a window; the gates decide what, if anything, comes through — identically for every carrier. — Annex XVI, Optirando™ Canon

WR Code® – The Establishment Commitment

A private cryptographic proof that a handshake was formed —
visible to nobody who was not part of it.

1 – Both sides contribute a secret

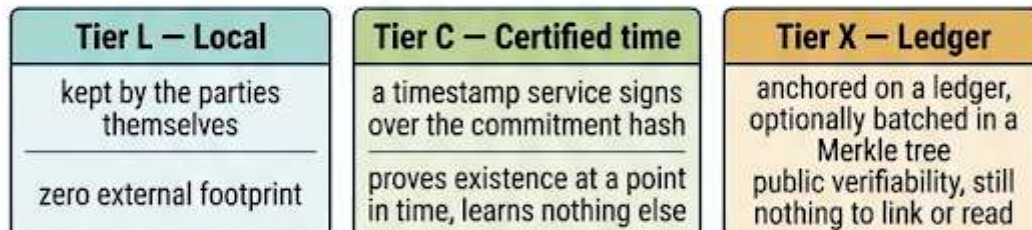


2 – The commitment

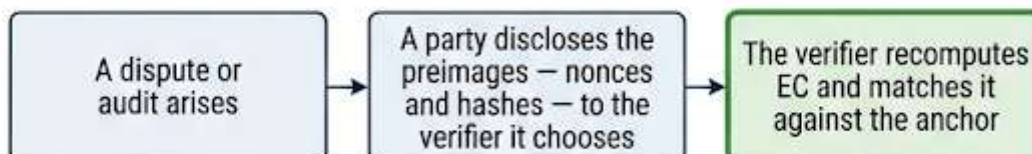
$$EC = H \left(\begin{array}{l} \text{domain tag} \parallel \text{parties} \parallel \text{request instance} \parallel \text{nonce A} \\ \parallel \text{nonce B} \parallel \text{capsule hash} \parallel \text{acceptance hash} \end{array} \right)$$

- Domain-separated and length-prefixed — the encoding is injective
- Both parties can compute it; nobody else can
- In a multi-party handshake the commitment exists only after every acceptance

3 – Three anchoring tiers



4 – Proving it later



Disclosure is selective and voluntary; without the preimages the anchor reveals nothing.

5 – What it is not

- ⊗ **Not an authority:** the commitment authorizes nothing and grants nothing
- ⊗ **Not a public record of who talked to whom:** anchors are unlinkable
- ⊗ **Not carried on any reference or carrier:** it exists only inside the protocol

Two secrets, one hash, zero exposure — proof of establishment without a trace.
— Annex XVI, Optirando™ Canon

WR Code® — The Displayed Responsible Domain

See who is responsible before you scan — verified after you scan.

1 — On the carrier

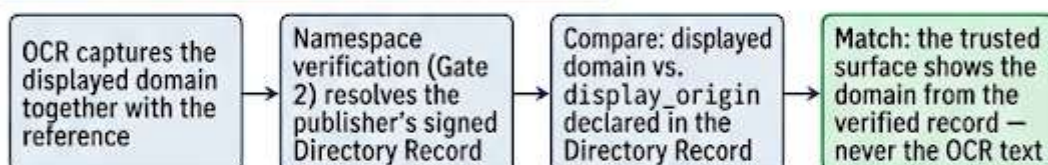


2 — One strict normalization

Written on a carrier	Normalized
https://www.example-restaurant.tld/menu	example-restaurant.tld
shop.example-restaurant.tld	example-restaurant.tld
WWW.EXAMPLE-RESTAURANT.TLD	example-restaurant.tld
example-restaurant.tld:8080	example-restaurant.tld

Registrable base domain only — no scheme, no subdomain, no path, no port; lowercase; internationalized names compared in punycode form.

3 — The cross-check after resolution



4 — When the carrier lies



5 — Absence is not a finding

- ✓ Transcribed text, audio, chat, and handwritten carriers legitimately carry no domain
- ✓ Validity is decided by resolution, never by the carrier
- ✓ Official templates and the WR Connect script emit the domain on visual composites by default

The printed domain is a claim; the verified record is the truth — and the user always sees the truth. — Annex XVI, Optirando™ Canon

Conceptual depiction. Example reference is a registry-derived test vector (Appendix A.2).

WR Code[®] — Aliases and Docking

Call your handshakes by name — and run several at once.

1 — A Handshake Alias is a local name

Alias table (local)

Calculus → SP-WR7X4K-H2N5V8-7

Plotting → (established sub-handshake)

- A user-assigned name for an established handshake or sub-handshake
- Lives only in your own Orchestrator — never on a carrier, never part of a reference, no check, no authority
- Resolved locally: an unknown alias fails with no network access at all
- An alias can never establish a new handshake, reach a foreign namespace, or bypass a gate

2 — Three voice commands

“OK Opty, load Calculus”

recalls the aliased sub-handshake as the primary context

“OK Opty, add Calculus”

docks it alongside the sub-handshakes already active

“OK Opty, remove Calculus”

undocks it from the current context

The assistant **resolves the alias** in the local table, verifies handshake state and grant validity, and requires whatever assurance the grant demands.

3 — Parallel docking

Active context — e.g. working on one document

Calculus
(tutoring SP)

Plotting
(capability)

Citation
checker

Docked set — managed in the adaptive WR Menu

- Several established sub-handshakes can be active at the same time
- The Orchestrator may dock and undock automatically as the governed context changes
- You add and remove individually — by menu selection or by alias

4 — Docking never widens anything

- ✓ Docking only activates what is already established and consented
- ✓ A docked sub-handshake never extends another's grant
- ✓ Every docked sub-handshake stays individually visible in the trusted UI, and every effect remains attributable to exactly one of them
- ✓ Undocking ends the docked state, not the handshake — revocation or expiry undocks automatically

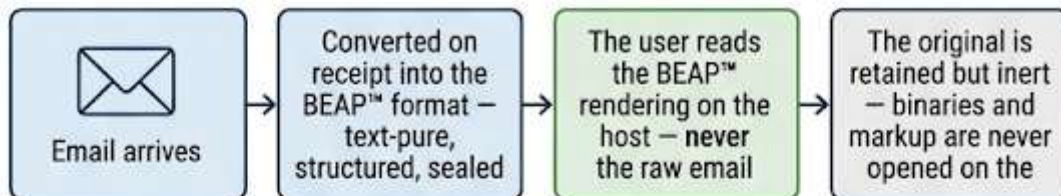
The alias is convenience; the handshake is the authority — and the two never trade places. — Annex XVI, Optirando™ Canon

Conceptual depiction. Example reference is a registry-derived test vector (Appendix A.2).

WR Code® — The Email Security Model

The host Orchestrator is not a mail client — every email becomes **governed BEAP™ content** before anyone reads it.

1 — Ingest: every email is converted



2 — Opening originals: the BEAP™ Clone in the Sandbox Orchestrator

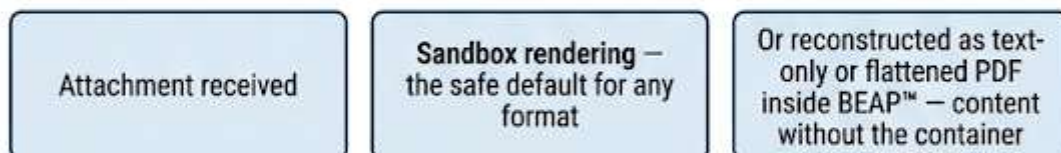
Standard tier

- Originals open only as a BEAP™ Clone inside a sandbox VM on the host
- An explicit warning precedes every opening
- Nothing from the sandbox writes back to the host

High-assurance tier

- The Sandbox Orchestrator runs hardware-isolated on a separate machine
- Opening originals on the host is prohibited entirely
- Capture the sandbox rendering — never parse the file on the host

3 — Attachments



The container format is the attack surface; the reconstruction carries the content and leaves the container behind.

4 — Links

Default: links never open on the host — a link in an email is text, not an action

Sole exception: whitelisted, verified URLs may open directly — an explicit superadmin deviation, links only, never attachments, never content-derived

5 — References in email

- ✓ A WR Code® reference in an email is parse-only text — the email graphic is never a trusted visual
- ✓ One umbrella offer per carrier instance — a single email cannot smuggle a second offer
- ✓ The offer the user sees is re-rendered locally after full validation, exactly as on every other carrier

Email delivers text and tokens — **never execution, never trust.**
What you read is BEAP™; what you open, opens in the sandbox.

— Annex XVI, Optirando™ Canon

WR Code® — Use-Limited Entries

An entry can be valid exactly once — or exactly as often as its issuer declares.

1 — Declared at issue, enforced at resolution

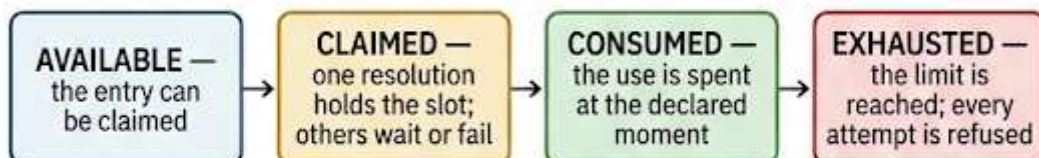
Advanced options (set by the issuer)

```
use_limit: 1
use_scope: resolution | context | automation
consume_at: acceptance | execution
claim_timeout: 120 s
require_live_resolution: true
```

P-WR7X4K-9B2M3C

The limits are properties of the entry in the issuer's namespace — **never carried on the reference, never inferable from it.**

2 — The lifecycle



A failed or abandoned claim falls back to AVAILABLE after the claim timeout — nothing is lost to a crashed scan.

3 — Atomic by construction

- ✓ Claiming is an atomic compare-and-set in the issuer's namespace — two simultaneous scans can never both win
- ✓ The counter spans all carriers: the printed poster, the email, and the spoken code count against the same limit
- ✓ `consume_at` decides what spends the use — accepting the offer, or executing what it grants

4 — What this enables



One-time invitations — a cross-org invitation that dies the moment it is accepted



Pairing Slots — one device may pair, exactly once, then the slot is superseded



Single-use vouchers and tickets — an offer that cannot be shared, forwarded, or replayed



Bounded automation an action the counterpart may trigger a declared number of times, no more

Copying the reference copies nothing — the limit lives in the namespace, and the namespace counts.

— Annex XVI, Optirando™ Canon

Conceptual depiction. Example reference is a registry-derived test vector (Appendix A.2).

